

Official
Release

Andes DSP Library

V3 User Manual

Document Number UM119-22

Date Issued 2019-01-11

Copyright © 2014-2019 Andes Technology Corporation.
All rights reserved.



Copyright Notice

Copyright © 2014–2019 Andes Technology Corporation. All rights reserved.

AndesCore™, AndeShape™, AndeSight™, AndESLive™, AndeSoft™, AndeStar™ and Andes Custom Extension™ are trademarks owned by Andes Technology Corporation. All other trademarks used herein are the property of their respective owners.

This document contains confidential information pertaining to Andes Technology Corporation. Use of this copyright notice is precautionary and does not imply publication or disclosure. Neither the whole nor part of the information contained herein may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language in any form by any means without the written permission of Andes Technology Corporation.

The product described herein is subject to continuous development and improvement. Thus, all information herein is provided by Andes in good faith but without warranties. This document is intended only to assist the reader in the use of the product. Andes Technology Corporation shall not be liable for any loss or damage arising from the use of any information in this document, or any incorrect use of the product.

Contact Information

Should you have any problems with the information contained herein, you may contact Andes Technology Corporation through

- email – support@andestech.com
- Website – <https://es.andestech.com/eservice/>

Please include the following information in your inquiries:

- the document title
- the document number
- the page number(s) to which your comments apply
- a concise explanation of the problem

General suggestions for improvements are welcome.

Revision History

Rev.	Revision Date	Revised Content
2.2	2019/01/11	1. Modified Section 1.3
2.1	2018/09/25	1. Added Section 1.3 2. Modified the example for nds32_bq_stereo_df2T_f32 function. (Section 2.4.2.3)
2.0	2018/05/21	1. Updated the function list in Andes DSP library. (Table 1) 2. Added nds32_fir_fast_q31 (Section 2.4.6.3) 3. Added nds32_fir_fast_q15 (Section 2.4.6.5) 4. Added nds32_liir_fast_q31 (Section 2.4.9.3) 5. Added nds32_liir_fast_q15 (Section 2.4.9.5) 6. Refined the description of linking options (Chapter 1)
1.9	2018/03/06	1. Corrected the intrinsic function prototype of “khm16” (Section 3.2.6) 2. Corrected the intrinsic function prototypes of “khmbb”, “khmbt”, “khmtb” and “khmtt” (Section 3.5.2)
1.8	2017/10/5	1. Modified the definition of the parameter “*coeff” for FIR Filter functions (Section 2.4.6.1~2.4.6.6) 2. Modified the definition of the parameter “*coeff” for Decimation FIR Filter functions (Section 2.4.7.1~2.4.7.3) 3. Modified the definition of the parameters “*rcoeff” and “*lcoeff” for Lattice IIR Filter functions (Section 2.4.9.1~2.4.9.4) 4. Modified the definition of the parameter “*coeff” for LMS Filter functions (Section 2.4.10.1~2.4.10.3) 5. Modified the definition of the parameter “*coeff” for Normalized LMS Filter functions (Section 2.4.11.1~2.4.11.3) 6. Modified the definition of the parameter “*coeff” for Sparse FIR Filter functions (Section 2.4.12.1~2.4.12.4) 7. Modified the definition of the parameter “*coeff” for Upsampling FIR Filter functions (Section 2.4.13.1~2.4.13.3) 8. Modified the array declaration for FFT functions in examples (Section 2.7.4.1, 2.7.5.1, 2.7.6.1) 9. Modified the definition of the parameter “*src” for Radix-2 Complex FFT functions (Section 2.7.1.1~2.7.1.6) 10. Modified the definition of the parameter “*src” for Radix-4

Rev.	Revision Date	Revised Content
		Complex FFT functions (Section 2.7.2.1~2.7.2.6) 11. Modified the definition of the parameter “*src” for Complex FFT functions (Section 2.7.3.1~2.7.3.6) 12. Modified the definition of the parameter “*src” for DCT Type II FFT functions (Section 2.7.4.1~2.7.4.6) 13. Modified the definition of the parameter “*src” for DCT Type IV FFT functions (Section 2.7.5.1~2.7.5.6) 14. Modified the definition of the parameter “*src” for Real FFT functions (Section 2.7.6.1~2.7.6.6) 15. Added nds32_div_u64_u32 (Section 2.1.3.3) 16. Added nds32_div_s64_u32 (Section 2.1.3.4) 17. Modified the Note for Matrix Power 2 function (Section 2.5.7.1)
1.7	2017/06/17	1. Noted that DSP-compatible chip profiles in AndeSight v3.0 or later versions have been specified with DSP-related linking options. (Section 1.1) 2. Updated the function list in Andes DSP library and added a column to indicate the BSP version since which each function is supported. (Table 1) 3. Added nds32_add_u8_u16 (Section 2.1.2.5) 4. Added nds32_dprod_u8 and nds32_dprod_u8xq15 (Section 2.1.4.5~2.1.4.6) 5. Added nds32_mul_u8_u16 (Section 2.1.5.5) 6. Added nds32_offset_u8 (Section 2.1.7.5) 7. Added nds32_scale_u8 (Section 2.1.8.5) 8. Added nds32_shift_u8 (Section 2.1.9.4) 9. Added nds32_sub_u8_q7 (Section 2.1.10.5) 10. Added nds32_cdprod_typ2_f32, nds32_cdprod_typ2_q31 and nds32_cdprod_typ2_q15 (Section 2.2.2.4~2.2.2.6) 11. Modified the example for nds32_bq_df1_f32 function. (Section 2.4.1.1) 12. Added an example for nds32_bq_df1_q31 function. (Section 2.4.1.2) 13. Added nds32_bq_df1_32x64_q31 (Section 2.4.1.4) 14. Added nds32_bq_df2T_f32, nds32_bq_df2T_f64 and nds32_bq_stereo_df2T_f32 (Section 2.4.2 and subsections) 15. Added nds32_conv_partial_f32, nds32_conv_partial_q31,

Rev.	Revision Date	Revised Content
		nds32_conv_partial_q15 and nds32_conv_partial_q7 (Section 2.4.4 and subsections) 16. Added nds32_nlms_q31 and nds32_nlms_q15 (Section 2.4.11.2~2.4.11.3) 17. Added nds32_mat_inv_f64 (Section 2.5.2.2) 18. Added nds32_mat_mul_f64 (Section 2.5.3.2) 19. Added nds32_mat_trans_u8 (Section 2.5.6.4) 20. Added nds32_mat_pwr2_cache_f64 (Section 2.5.7 and subsection) 21. Added nds32_max_u8 (Section 2.6.1.5) 22. Added nds32_mean_u8 (Section 2.6.2.5) 23. Added nds32_min_u8 (Section 2.6.3.5) 24. Added nds32_std_u8 (Section 2.6.6.4) 25. Added nds32_cfft_f32, nds32_cifft_f32, nds32_cfft_q31, nds32_cifft_q31, nds32_cfft_q15, nds32_cifft_q15 and nds32_atan2_f32. (Section 2.7.3 and subsections) 26. Modified the value range of the parameter “m” for Real FFF functions (Section 2.7.6.1~2.7.6.6)
1.6	2016/04/12	1. Added how to link software and hardware libdsp with glibc toolchains and enable zero overhead loop feature in linux kernel. (Section 1.1 and 1.2)
1.5	2015/12/14	1. Corrected the base of logarithm for transform functions. (Section 2.7) 2. Summarized intrinsic functions related to DSP application programming in Chapter 3.
1.4	2015/11/12	1. Added the value range for the shift parameter in scale functions. 2. Corrected the order of dst and size parameters in complex-multiply-complex functions. 3. Removed the const keyword in convolution and correlation functions. 4. Corrected the order of coeff and state in nds32_dcmfir_f32_t, nds32_dcmfir_q31_t and nds32_dcmfir_q15_t for decimation FIR filter functions. 5. Corrected the order of coeff and state in nds32_upspltir_f32_t, nds32_upspltir_q31_t and nds32_upspltir_q15_t for upsampling FIR filter functions. 6. Added more descriptions for the matrix inverse function.

Rev.	Revision Date	Revised Content
		7. Corrected the figure number referred in the description of matrix multiplication functions.
1.3	2015/07/29	1. Refined the description of linking options (Chapter 1)
1.2	2015/05/15	1. Appended the compilation options. 2. Corrected the code examples. 3. Modified the content of biquad df1 functions. 4. Modified the description of arc tangent 2 functions and added nds32_atan2_q31 function. 5. Modified the description of correlation functions. 6. Refined the description of cosine and sine functions.
1.1	2015/01/14	1. Noted that an arithmetic right shift may be needed before Q31 and Q15 transform functions are called. 2. Added tables about the input and output formats for Q31 and Q15 transform functions.
1.0	2014/10/31	Initial release

Table of Contents

COPYRIGHT NOTICE.....	II
CONTACT INFORMATION.....	II
REVISION HISTORY.....	III
LIST OF TABLES	X
LIST OF FIGURES	XI
1. OVERVIEW.....	1
1.1. LINKING OPTIONS FOR NEWLIB AND MCULIB TOOLCHAINS	3
1.2. LINKING OPTIONS FOR GLIBC TOOLCHAINS.....	4
1.3. DATA ALIGNMENT FOR ANDES DSP LIBRARY FUNCTIONS.....	5
2. DESCRIPTIONS OF FUNCTIONS	7
2.1. BASIC FUNCTIONS	17
2.1.1. Absolute Functions.....	17
2.1.2. Addition Functions.....	22
2.1.3. Division Functions.....	28
2.1.4. Dot Product Functions.....	33
2.1.5. Multiplication Functions.....	40
2.1.6. Negate Functions.....	46
2.1.7. Offset Functions.....	51
2.1.8. Scale Functions.....	57
2.1.9. Shift Functions.....	63
2.1.10. Subtraction Functions.....	68
2.2. COMPLEX FUNCTIONS.....	74
2.2.1. Complex Conjugate Functions.....	74
2.2.2. Complex Dot Product Functions	78
2.2.3. Complex Magnitude Functions	85
2.2.4. Complex Magnitude-Squared Function.....	89
2.2.5. Complex-Multiply-Complex Functions	93
2.2.6. Complex-Multiply-Real Functions.....	97
2.3. CONTROLLER FUNCTIONS.....	101
2.3.1. Clarke Transform Functions	101
2.3.2. Inverse Clarke Transform Functions.....	104
2.3.3. PID Functions	107
2.4. FILTERING FUNCTIONS	115
2.4.1. Biquadratic Cascade Filter (Direct Form I) Functions	115

2.4.2.	<i>Biquadratic Cascade Filter (Direct Form II Transposed) Functions</i>	121
2.4.3.	<i>Convolution Functions</i>	125
2.4.4.	<i>Partial Convolution Functions</i>	130
2.4.5.	<i>Correlation Functions</i>	135
2.4.6.	<i>Finite Impulse Response (FIR) Filter Functions</i>	140
2.4.7.	<i>Decimation FIR Filter Functions</i>	147
2.4.8.	<i>Lattice FIR Filter Functions</i>	152
2.4.9.	<i>Lattice Infinite Impulse Response (IIR) Filter Functions</i>	156
2.4.10.	<i>Least Mean Square (LMS) Filter Functions</i>	163
2.4.11.	<i>Normalized Least Mean Square (NLMS) Filter Function</i>	167
2.4.12.	<i>Sparse FIR Filter Functions</i>	171
2.4.13.	<i>Upsampling FIR Filter Functions</i>	176
2.5.	MATRIX FUNCTIONS	180
2.5.1.	<i>Matrix Addition Functions</i>	181
2.5.2.	<i>Matrix Inverse Function</i>	185
2.5.3.	<i>Matrix Multiplication Functions</i>	188
2.5.4.	<i>Matrix Scale Functions</i>	193
2.5.5.	<i>Matrix Subtraction Functions</i>	197
2.5.6.	<i>Matrix Transpose Functions</i>	201
2.5.7.	<i>Matrix Power 2 Function</i>	206
2.6.	STATISTICS FUNCTIONS	208
2.6.1.	<i>Maximum Functions</i>	208
2.6.2.	<i>Mean Functions</i>	214
2.6.3.	<i>Minimum Functions</i>	220
2.6.4.	<i>Root Mean Square (RMS) Functions</i>	226
2.6.5.	<i>Power Functions</i>	230
2.6.6.	<i>Standard Deviation Functions</i>	235
2.6.7.	<i>Variance Functions</i>	240
2.7.	TRANSFORM FUNCTIONS	244
2.7.1.	<i>Radix-2 Complex FFT Functions</i>	245
2.7.2.	<i>Radix-4 Complex FFT Functions</i>	252
2.7.3.	<i>Complex FFT Functions</i>	259
2.7.4.	<i>DCT Type II Functions</i>	266
2.7.5.	<i>DCT Type IV Functions</i>	273
2.7.6.	<i>Real FFT Functions</i>	280
2.8.	UTILS FUNCTIONS	287
2.8.1.	<i>Arc Tangent Functions</i>	287
2.8.2.	<i>Arc Tangent 2 Functions</i>	291
2.8.3.	<i>Cosine and Sine Functions</i>	295

2.8.4.	<i>Conversion Functions</i>	302
2.8.5.	<i>Duplicate Functions</i>	315
2.8.6.	<i>Set Functions</i>	320
2.8.7.	<i>Square Root Functions</i>	325
3.	INTRINSIC FUNCTIONS FOR PROGRAMMING DSP APPLICATIONS	329
3.1.	DEFINITIONS OF SHORTHANDS FOR EXPLAINING FUNCTION OPERATION	330
3.2.	SIMD DATA PROCESSING INTRINSIC FUNCTIONS	331
3.2.1.	<i>16-bit Addition & Subtraction Intrinsic Functions</i>	331
3.2.2.	<i>8-bit Addition & Subtraction Intrinsic Functions</i>	336
3.2.3.	<i>16-bit Shift Intrinsic Functions</i>	340
3.2.4.	<i>16-bit Compare Intrinsic Functions</i>	343
3.2.5.	<i>8-bit Compare Intrinsic Functions</i>	345
3.2.6.	<i>16-bit Miscellaneous Intrinsic Functions</i>	347
3.2.7.	<i>8-bit Miscellaneous Intrinsic Functions</i>	351
3.2.8.	<i>8-bit Unpacking Intrinsic Functions</i>	353
3.3.	NON-SIMD DATA PROCESSING INTRINSIC FUNCTIONS	355
3.3.1.	<i>32-bit Addition/Subtraction Intrinsic Functions</i>	355
3.3.2.	<i>32-bit Shift Intrinsic Functions</i>	356
3.3.3.	<i>16-bit Packing Intrinsic Functions</i>	357
3.3.4.	<i>Most Significant Word "32x32" Multiply & Add Intrinsic Functions</i>	358
3.3.5.	<i>Most Significant Word "32x16" Multiply & Add Intrinsic Functions</i>	360
3.3.6.	<i>Signed 16-bit Multiply with 32-bit Add/Subtract Intrinsic Functions</i>	362
3.3.7.	<i>Signed 16-bit Multiply with 64-bit Add/Subtract Intrinsic Functions</i>	366
3.3.8.	<i>Miscellaneous Intrinsic Functions</i>	367
3.4.	64-BIT PROFILE INTRINSIC FUNCTIONS	372
3.4.1.	<i>64-bit Addition & Subtraction Intrinsic Functions</i>	372
3.4.2.	<i>32-bit Multiply with 64-bit Add/Subtract Intrinsic Functions</i>	375
3.4.3.	<i>Signed 16-bit Multiply with 64-bit Add/Subtract Intrinsic Functions</i>	377
3.5.	SATURATION ISA EXTENSION INTRINSIC FUNCTIONS	380
3.5.1.	<i>Q31 saturation Intrinsic Functions</i>	380
3.5.2.	<i>Q15 saturation Intrinsic Functions</i>	382
3.5.3.	<i>Overflow status manipulation Intrinsic Functions</i>	384

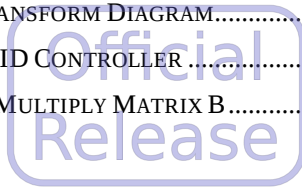
List of Tables

TABLE 1, ANDES DSP LIBRARY FUNCTION LIST	7
--	---



List of Figures

FIGURE 1. CLARKE TRANSFORM DIAGRAM.....	101
FIGURE 2. STATE OF PID CONTROLLER.....	107
FIGURE 3. MATRIX A MULTIPLY MATRIX B.....	188



Typographical Convention Index

Document Element	Font	Font Style	Size	Color
Normal text	Georgia	Normal	12	Black
Command line, source code or file paths	Lucida Console	Normal	11	Indigo
VARIABLES OR PARAMETERS IN COMMAND LINE, SOURCE CODE OR FILE PATHS	LUCIDA CONSOLE	BOLD + ALL - CAPS	11	INDIGO
Note or warning	Georgia	Normal	12	Red
<u>Hyperlink</u>	Georgia	<u>Underlined</u>	12	Blue

1. Overview



Andes DSP library V3 user manual contains information about DSP function specifications in Andes DSP library for ISA V3 toolchains and describes how to use these functions.

Andes DSP library provides comprehensive functions to easily and quickly develop digital signal processing systems with high code efficiency and streamlined code size. These functions include basic operations for vectors, matrices and complex vectors and functions which compute more complicated algorithms such as filtering and transform functions. Designed and developed with toolchains from Andes BSP v4.0 and later versions, Andes DSP library provides easy-to-use APIs to encapsulate the complexity of calculations when processing signals. It benefits you to save development resources, focus on your system and reduce development time.

The functions in Andes DSP library can be classified into several categories: basic functions, complex functions, controller functions, filtering functions, matrix functions, statistics functions, transform function and utility functions. You may refer to Chapter 1.3 for detailed descriptions about each category.

To call the library functions, just include header files provided by Andes DSP library. These header files define prototypes and instance structures of functions. They are named according to function categories as listed below:

- `nds32_basic_math.h`
- `nds32_complex_math.h`
- `nds32_controller_math.h`
- `nds32_filtering_math.h`
- `nds32_matrix_math.h`
- `nds32_statistics_math.h`
- `nds32_transform_math.h`
- `nds32_utils_math.h`

For example, if you would like to use basic functions for some operations with vectors, you can include `nds32_basic_math.h` in your C files.

Andes DSP Library V3 User Manual

Andes DSP library defines several data types to properly store signal data for later processing. These data types, defined in `nds32_math_types.h` and quoted below, are `q7_t` for 8-bit integer, `q15_t` for 16-bit integer, `q31_t` for 32-bit integer, `q63_t` for 64-bit integer, `float32_t` for 32-bit single-precision floating-point and `float64_t` for 64-bit double-precision floating-point respectively:

```
typedef int8_t q7_t;           /* q7 type */
typedef int16_t q15_t;        /* q15 type */
typedef int32_t q31_t;        /* q31 type */
typedef int64_t q63_t;        /* q63 type */
typedef float float32_t;      /* single-precision floating-point type */
typedef double float64_t;     /* double-precision floating-point type */
```

1.1. Linking Options for Newlib and MCULib Toolchains

In addition to the above header files, different linking options are required depending on the toolchain in use. For Newlib or MCULib toolchains, `-ldsp` is the option needed to build an application that links Andes DSP library (`libdsp.a`). If your application requires DSP ISA Extension to accelerate the DSP library and achieve better performance, please use linking options `-ldsp -mext-dsp`. To use Andes DSP library functions that involve math functions from the libm library (that is, `nds32_std_f32`, `nds32_rms_f32`, `nds32_cmag_f32`, `nds32_sqrt_f32` and `nds32_atan_f32`), the linking option `-lm` must be applied. These linking options for Newlib and MCULib toolchains are summarized below:

- `-ldsp` links Andes DSP library (`libdsp.a`).
- `-mext-dsp` enables DSP ISA Extension.
- `-lm` links the libm math library.

If you are using AndeSight v3.0.0 or later versions, you can save the trouble of manually specifying these DSP-related linking options by selecting a DSP-compliant chip profile (such as D1088 or D15) for your projects.

1.2. Linking Options for Glibc Toolchains

For glibc toolchains, the required linking options differ between static linking and dynamic linking. The following table summarized the required linking options according to the linking type and whether or not the application need the DSP ISA Extension support.

Linking Type	DSP ISA Extension	Linking Options
Static Linking	Without	<code>-static -ldsp</code>
	With	<code>-static -ldsp_hw</code>
Dynamic Linking (default)	Without	<code>-ldsp</code>
	With	<code>-ldsp_hw</code>

If your application is statically linked and requires Andes DSP library functions that involve math functions from the libm library (that is, `nds32_std_f32`, `nds32_rms_f32`, `nds32_cmag_f32`, `nds32_sqrt_f32` and `nds32_atan_f32`), the linking option `-lm` MUST be applied too.

These linking options for glibc toolchains are explained individually below:

- `-static` uses static linking.
- `-ldsp` links Andes DSP library without DSP ISA Extension.
- `-ldsp_hw` links Andes DSP library with DSP ISA Extension.
- `-lm` links the libm math library.

Andes DSP library with DSP ISA Extension delivers better performance with the support from both DSP instructions and hardware zero overhead loop (ZOL). That is, the ZOL feature MUST be enabled in the Linux kernel for applying the option `-ldsp_hw`; otherwise, unpredictable errors may occur. To turn on the feature, please refer to BSP User Manual, Section 7.6 “Building Linux Kernel System Image” for the instruction.

By contrast, Andes DSP library without DSP ISA Extension requires no ZOL support. There is no need to enable the ZOL feature in the Linux kernel for applying the option `-ldsp`.

1.3. Data alignment for Andes DSP library functions

For performance consideration, you can link your application to Andes DSP library and enable DSP ISA Extension using the linking option `-mext-dsp` or `-ldsp_hw`, but make sure that input arrays are accessed at aligned addresses.

Andes DSP library functions are designed to use 4-byte aligned data. That is, they assume the base memory addresses of input arrays, including input source arrays and output destination arrays, are aligned to 4-byte boundaries. These functions are implemented to load/store one word regardless of the type of input pointer, then process with SIMD DSP instructions to deliver high performance. Accessing a word from an unaligned memory location is called an unaligned memory access, which causes a Data Alignment Check exception on a CPU that doesn't allow unaligned accesses. For a CPU that supports unaligned accesses and has the feature enabled, an unaligned memory access doesn't result in an exception but a slight performance degradation. You may refer to *AndeStar™ System Privilege Architecture Version 3 Manual* for detailed descriptions of how Andes CPUs handle unaligned accesses.

Among the data types supported by Andes DSP library functions, Q31 and F32 are 4-byte data types and thus naturally aligned at 4-byte addresses when they are declared. On the other hand, Q15 and Q7 are 2-byte and 1-byte data types, which are aligned at 2-byte and 1-byte addresses by default respectively. To avoid unaligned access exceptions, you need to explicitly declare that input arrays are accessed at 4-byte aligned addresses for these two data types. For examples,

```
q7_t inputA[blocksize] __attribute__((aligned(4)));  
q15_t outputvec[blocksize] __attribute__((aligned(4)));
```

The following are functions that do not require explicit declaration of aligned accesses:

- Functions using input arrays of F32 data type
- Functions using input arrays of Q31 data type
- Controller functions
- Conversion functions
- Arc tangent functions
- Arc tangent 2 functions
- Cosine functions

Andes DSP Library V3 User Manual

- Sine functions
- Square root functions



2. Descriptions of Functions

Table 1 lists all functions in Andes DSP library grouped into several categories. The “Supported since” column in this table uses the following numbers to denote the BSP version since which each function is first available:

1 = BSP v4.0.0

2 = BSP v4.0.2

3 = BSP v4.1.0

4 = BSP v4.2.0 and BSP v4.2.1

5 = BSP v4.2.2

Table 1. Andes DSP Library Function List

Description	Function name	Supported since	Page
Basic Functions			
Absolute Functions	nds32_abs_f32	1	18
	nds32_abs_q31	1	19
	nds32_abs_q15	1	20
	nds32_abs_q7	1	21
Addition Functions	nds32_add_f32	1	23
	nds32_add_q31	1	24
	nds32_add_q15	1	25
	nds32_add_q7	1	26
	nds32_add_u8_u16	4	27
Division Functions	nds32_div_f32	1	29
	nds32_div_q31	1	30
	nds32_div_u64_u32	4	31
	nds32_div_s64_u32	4	32
Dot Product Functions	nds32_dprod_f32	1	34
	nds32_dprod_q31	1	35

Andes DSP Library V3 User Manual

Description	Function name	Supported since	Page
Official Release	nds32_dprod_q15	1	36
	nds32_dprod_q7	1	37
	nds32_dprod_u8	4	38
	nds32_dprod_u8xq15	4	39
Multiplication Functions	nds32_mul_f32	1	41
	nds32_mul_q31	1	42
	nds32_mul_q15	1	43
	nds32_mul_q7	1	44
	nds32_mul_u8_u16	4	45
Negate Functions	nds32_neg_f32	1	47
	nds32_neg_q31	1	48
	nds32_neg_q15	1	49
	nds32_neg_q7	1	50
Offset Functions	nds32_offset_f32	1	52
	nds32_offset_q31	1	53
	nds32_offset_q15	1	54
	nds32_offset_q7	1	55
	nds32_offset_u8	4	56
Scale Functions	nds32_scale_f32	1	58
	nds32_scale_q31	1	59
	nds32_scale_q15	1	60
	nds32_scale_q7	1	61
	nds32_scale_u8	4	62
Shift Functions	nds32_shift_q31	1	64
	nds32_shift_q15	1	65
	nds32_shift_q7	1	66
	nds32_shift_u8	4	67

Andes DSP Library V3 User Manual

Description	Function name	Supported since	Page
Subtraction Functions	nds32_sub_f32	1	69
	nds32_sub_q31	1	70
	nds32_sub_q15	1	71
	nds32_sub_q7	1	72
	nds32_sub_u8_q7	4	73
Complex Functions			
Complex Conjugate Functions	nds32_cconj_f32	1	75
	nds32_cconj_q31	1	76
	nds32_cconj_q15	1	77
Complex Dot Product Functions	nds32_cdprod_f32	1	79
	nds32_cdprod_q31	1	80
	nds32_cdprod_q15	1	81
	nds32_cdprod_typ2_f32	1	82
	nds32_cdprod_typ2_q31	1	83
	nds32_cdprod_typ2_q15	1	84
Complex Magnitude Functions	nds32_cmag_f32	1	86
	nds32_cmag_q31	1	87
	nds32_cmag_q15	1	88
Complex Magnitude Squared Functions	nds32_cmag_sqr_f32	1	90
	nds32_cmag_sqr_q31	1	91
	nds32_cmag_sqr_q15	1	92
Complex Multiply Complex Functions	nds32_cmul_f32	1	94
	nds32_cmul_q31	1	95
	nds32_cmul_q15	1	96
Complex Multiply Real Functions	nds32_cmul_real_f32	1	98
	nds32_cmul_real_q31	1	99
	nds32_cmul_real_q15	1	100

Description	Function name	Supported since	Page
Controller Functions			
Clarke Transform Functions	nds32_clarke_f32	1	102
	nds32_clarke_q31	1	103
Inverse Clarke Transform Functions	nds32_inv_clarke_f32	1	105
	nds32_inv_clarke_q31	1	106
PID Functions	nds32_init_pid_f32	1	108
	nds32_pid_f32	1	110
	nds32_init_pid_q31	1	111
	nds32_pid_q31	1	112
	nds32_init_pid_q15	1	113
	nds32_pid_q15	1	114
Filtering Functions			
Biquadratic Cascade Filters (Direct Form I) Functions	nds32_bq_df1_f32	1	116
	nds32_bq_df1_q31	1	117
	nds32_bq_df1_q15	1	119
	nds32_bq_df1_32x64_q31	3	120
Biquadratic Cascade Filters (Direct Form II) Functions	nds32_bq_df2T_f32	3	122
	nds32_bq_df2T_f64	3	123
	nds32_bq_stereo_df2T_f32	3	124
Convolution Functions	nds32_conv_f32	1	126
	nds32_conv_q31	1	127
	nds32_conv_q15	1	128
	nds32_conv_q7	1	129
Partial Convolution Functions	nds32_conv_partial_f32	3	131
	nds32_conv_partial_q31	3	132
	nds32_conv_partial_q15	4	133
	nds32_conv_partial_q7	3	134

Description	Function name	Supported since	Page
Correlation Functions	nds32_corr_f32	1	136
	nds32_corr_q31	1	137
	nds32_corr_q15	1	138
	nds32_corr_q7	1	139
Finite Impulse Response (FIR) Filter Functions	nds32_fir_f32	1	141
	nds32_fir_q31	1	142
	nds32_fir_fast_q31	5	143
	nds32_fir_q15	1	144
	nds32_fir_fast_q15	5	145
	nds32_fir_q7	1	146
Decimator FIR Filter Functions	nds32_dcmfir_f32	1	148
	nds32_dcmfir_q31	1	150
	nds32_dcmfir_q15	1	151
Lattice FIR Filter Functions	nds32_lfir_f32	1	153
	nds32_lfir_q31	1	154
	nds32_lfir_q15	1	155
Lattice Infinite Impulse Response (IIR) Filter Functions	nds32_liir_f32	1	157
	nds32_liir_q31	1	159
	nds32_liir_fast_q31	5	160
	nds32_liir_q15	1	161
	nds32_liir_fast_q15	5	162
Least Mean Square (LMS) Filters Functions	nds32_lms_f32	1	164
	nds32_lms_q31	1	165
	nds32_lms_q15	1	166
Normalized LMS (NLMS) Filter Functions	nds32_nlms_f32	1	168
	nds32_nlms_q31	4	169
	nds32_nlms_q15	4	170

Andes DSP Library V3 User Manual

Description	Function name	Supported since	Page
Sparse FIR Filter Functions	nds32_spafir_f32	1	172
	nds32_spafir_q31	1	173
	nds32_spafir_q15	1	174
	nds32_spafir_q7	1	175
Upsampling FIR Filter Functions	nds32_upsplfir_f32	1	177
	nds32_upsplfir_q31	1	178
	nds32_upsplfir_q15	1	179
Matrix Functions			
Matrix Addition Functions	nds32_mat_add_f32	1	182
	nds32_mat_add_q31	1	183
	nds32_mat_add_q15	1	184
Matrix Inverse Functions	nds32_mat_inv_f32	1	186
	nds32_mat_inv_f64	4	187
Matrix Multiplication Functions	nds32_mat_mul_f32	1	189
	nds32_mat_mul_f64	4	190
	nds32_mat_mul_q31	1	191
	nds32_mat_mul_q15	1	192
Matrix Scale Functions	nds32_mat_scale_f32	1	194
	nds32_mat_scale_q31	1	195
	nds32_mat_scale_q15	1	196
Matrix Substraction Functions	nds32_mat_sub_f32	1	198
	nds32_mat_sub_q31	1	199
	nds32_mat_sub_q15	1	200
Matrix Transpose Functions	nds32_mat_trans_f32	1	202
	nds32_mat_trans_q31	1	203
	nds32_mat_trans_q15	1	204
	nds32_mat_trans_u8	4	205
Matrix Power 2 Function	nds32_mat_pwr2_cache_f64	4	207

Description	Function name	Supported since	Page
Statistics Functions			
Maximum Functions	nds32_max_f32	1	209
	nds32_max_q31	1	210
	nds32_max_q15	1	211
	nds32_max_q7	1	212
	nds32_max_u8	4	213
Mean Functions	nds32_mean_f32	1	215
	nds32_mean_q31	1	216
	nds32_mean_q15	1	217
	nds32_mean_q7	1	218
	nds32_mean_u8	4	219
Minimum Functions	nds32_min_f32	1	221
	nds32_min_q31	1	222
	nds32_min_q15	1	223
	nds32_min_q7	1	224
	nds32_min_u8	4	225
Root Mean Square(RMS) Functions	nds32_rms_f32	1	227
	nds32_rms_q31	1	228
	nds32_rms_q15	1	229
Power Functions	nds32_pwr_f32	1	231
	nds32_pwr_q31	1	232
	nds32_pwr_q15	1	233
	nds32_pwr_q7	1	234
Standard Deviation Functions	nds32_std_f32	1	236
	nds32_std_q31	1	237
	nds32_std_q15	1	238
	nds32_std_u8	4	239
Variance Functions	nds32_var_f32	1	241
	nds32_var_q31	1	242

Description	Function name	Supported since	Page
	nds32_var_q15	1	243
Transform Functions			
Radix-2 Complex FFT Functions	nds32_cfft_rd2_f32	1	246
	nds32_cfft_rd2_q31	1	248
	nds32_cfft_rd2_q15	1	250
	nds32_cifft_rd2_f32	1	247
	nds32_cifft_rd2_q31	1	249
	nds32_cifft_rd2_q15	1	251
Radix-4 Complex FFT Functions	nds32_cfft_rd4_f32	1	253
	nds32_cfft_rd4_q31	1	255
	nds32_cfft_rd4_q15	1	257
	nds32_cifft_rd4_f32	1	254
	nds32_cifft_rd4_q31	1	256
	nds32_cifft_rd4_q15	1	258
Complex FFT Functions	nds32_cfft_f32	3	260
	nds32_cfft_q31	3	262
	nds32_cfft_q15	3	264
	nds32_cifft_f32	3	261
	nds32_cifft_q31	3	263
	nds32_cifft_q15	3	265
DCT Type II Functions	nds32_dct_f32	1	267
	nds32_dct_q31	1	269
	nds32_dct_q15	1	271
	nds32_idct_f32	1	268
	nds32_idct_q31	1	270
	nds32_idct_q15	1	272
DCT Type IV Functions	nds32_dct4_f32	1	274
	nds32_dct4_q31	1	276
	nds32_dct4_q15	1	278

Description	Function name	Supported since	Page
Official Release	nds32_idct4_f32	1	275
	nds32_idct4_q31	1	277
	nds32_idct4_q15	1	279
Real FFT Functions	nds32_rfft_f32	1	281
	nds32_rfft_q31	1	283
	nds32_rfft_q15	1	285
	nds32_rifft_f32	1	282
	nds32_rifft_q31	1	284
	nds32_rifft_q15	1	286
Utils Functions			
Arc Tangent Functions	nds32_atan_f32	1	288
	nds32_atan_q31	1	289
	nds32_atan_q15	1	290
Arc Tangent 2 Functions	nds32_atan2_f32	4	292
	nds32_atan2_q31	2	293
	nds32_atan2_q15	1	294
Cosine Functions	nds32_cos_f32	1	296
	nds32_cos_q31	1	297
	nds32_cos_q15	1	298
Sine Functions	nds32_sin_f32	1	299
	nds32_sin_q31	1	300
	nds32_sin_q15	1	301
Conversion Functions	nds32_convert_f32_q31	1	303
	nds32_convert_f32_q15	1	304
	nds32_convert_f32_q7	1	305
	nds32_convert_q31_f32	1	306
	nds32_convert_q31_q15	1	307
	nds32_convert_q31_q7	1	308
	nds32_convert_q15_f32	1	309

Description	Function name	Supported since	Page
	nds32_convert_q15_q31	1	310
	nds32_convert_q15_q7	1	311
	nds32_convert_q7_f32	1	312
	nds32_convert_q7_q31	1	313
	nds32_convert_q7_q15	1	314
Duplicate Functions	nds32_dup_f32	1	316
	nds32_dup_q31	1	317
	nds32_dup_q15	1	318
	nds32_dup_q7	1	319
Set Functions	nds32_set_f32	1	321
	nds32_set_q31	1	322
	nds32_set_q15	1	323
	nds32_set_q7	1	324
Square Root Functions	nds32_sqrt_f32	1	326
	nds32_sqrt_q31	1	327
	nds32_sqrt_q15	1	328

2.1. Basic Functions

2.1.1. Absolute Functions

Absolute functions get absolute values from elements of a source vector and write them one-by-one into a destination vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = (src[n] < 0) ? -src[n] : src[n];
```

Andes DSP library supports distinct absolute functions for the following data types: floating-point, Q31, Q15 and Q7. These functions are introduced in the subsections below.

2.1.1.1 nds32_abs_f32

Declaration:

```
void nds32_abs_f32 (float32_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.1.1.2 nds32_abs_q31

Declaration:

```
void nds32_abs_q31 (q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. The range of output results is [0, 0x7FFFFFFF].
2. If the input value is INT32_MIN (0x80000000), then the output value is INT32_MAX (0x7FFFFFFF).

2.1.1.3 nds32_abs_q15

Declaration:

```
void nds32_abs_q15 (q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. The range of output results is [0, 0x7FFF].
2. If the input value is INT16_MIN (0x8000), then the output value is INT16_MAX (0x7FFF).

2.1.1.4 nds32_abs_q7

Declaration:

```
void nds32_abs_q7 (q7_t *src, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. The range of output results is [0, 0x7F].
2. If the input value is INT8_MIN (0x80), then the output value is INT8_MAX (0x7F).

2.1.2. Addition Functions

Addition functions add two elements from separate source vectors and write the results one-by-one into a destination vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = src1[n] + src2[n];
```

Andes DSP library supports distinct addition functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.

2.1.2.1 nds32_add_f32

Declaration:

```
void nds32_add_f32 (float32_t *src1, float32_t *src2, float32_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size size of elements in a vector

Return Value:

None

2.1.2.2 nds32_add_q31

Declaration:

```
void nds32_add_q31 (q31_t *src1, q31_t *src2, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The addition results will be saturated to the Q31 range [0x80000000, 0x7FFFFFFF].

2.1.2.3 nds32_add_q15

Declaration:

```
void nds32_add_q15 (q15_t *src1, q15_t *src2, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The addition results will be saturated to the Q15 range [0x8000, 0x7FFF].

2.1.2.4 nds32_add_q7

Declaration:

```
void nds32_add_q7 (q7_t *src1, q7_t *src2, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The addition results will be saturated to the Q7 range [0x80, 0x7F].

2.1.2.5 nds32_add_u8_u16

Declaration:

```
void nds32_add_u8_u16(uint8_t *src1, uint8_t *src2, uint16_t *dst, uint32_t  
size);
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The addition result of two `uint8_t` data will be saved in `uint16_t` format.

2.1.3. Division Functions

Division functions divide two elements from separate source vectors/scalars and write the results one-by-one into a destination vector/scalar. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = src1[n] / src2[n];
```

Andes DSP library supports distinct division functions for floating-point, Q31 and other data types. The functions are introduced in the subsections below.

2.1.3.1 nds32_div_f32

Declaration:

```
void nds32_div_f32 (float32_t *src1, float32_t *src2, float32_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector (dividend vector)
- [in] *src2 pointer of the second input vector (divisor vector)
- [out] *dst pointer of the destination vector (quotient vector)
- [in] size number of elements in a vector

Return Value:

None

2.1.3.2 nds32_div_q31

Declaration:

q31_t nds32_div_q31 (q31_t src1, q31_t src2)

Parameters:

- [in] src1 dividend value
- [in] src2 divisor value

Return Value:

Quotient value

Note:

1. The absolute value of **src1** should be smaller than that of **src2**.
2. This function deals with scalars instead of vectors.

2.1.3.3 nds32_div_u64_u32

Declaration:

`q31_t nds32_div_u64_u32 (uint64_t src1, uint32_t src2)`

Parameters:

- `[in] src1` dividend value
- `[in] src2` divisor value

Return Value:

Quotient value

Note:

1. The value of `src1` right-shifted by 31 bits should be smaller than that of `src2`.
Otherwise, the result is not valid.
2. This function deals with scalars instead of vectors.

2.1.3.4 nds32_div_s64_u32

Declaration:

```
q31_t nds32_div_s64_u32 (q63_t src1, uint32_t src2)
```

Parameters:

- [in] src1 dividend value
- [in] src2 divisor value

Return Value:

Quotient value

Note:

1. The absolute value of `src1` right-shifted by 31 bits should be smaller than that of `src2`. Otherwise, the result is not valid.
2. This function deals with scalars instead of vectors.

2.1.4. Dot Product Functions

Dot product functions calculate the dot product of two vectors and return the result values. The definition of dot production is as follows:

```
for (output = 0, n = 0; n < size; n++)  
    output += src1[n] * src2[n];
```

Andes DSP library supports distinct dot product functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.

2.1.4.1 nds32_dprod_f32

Declaration:

```
float32_t nds32_dprod_f32 (float32_t *src1, float32_t *src2, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [in] size number of elements in a vector

Return Value:

Dot product of the two input vectors

2.1.4.2 nds32_dprod_q31

Declaration:

```
q63_t nds32_dprod_q31 (q31_t *src1, q31_t *src2, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [in] size number of elements in a vector

Return Value:

Dot product of the two input vectors

Note:

The format of the return values is Q48 and its type is `q63_t`.

2.1.4.3 nds32_dprod_q15

Declaration:

```
q63_t nds32_dprod_q15 (q15_t *src1, q15_t *src2, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [in] size number of elements in a vector

Return Value:

Dot product of the two input vectors

Note:

The format of the return values is Q48 and its type is `q63_t`.

2.1.4.4 nds32_dprod_q7

Declaration:

```
q31_t nds32_dprod_q7 (q7_t *src1, q7_t *src2, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [in] size number of elements in a vector

Return Value:

Dot product of the two input vectors

Note:

The format of the return values is Q14 and its type is `q31_t`.

2.1.4.5 nds32_dprod_u8

Declaration:

```
uint32_t nds32_dprod_u8(uint8_t *src1, uint8_t *src2, uint32_t size)
```

Parameters:

- `[in] *src1` pointer of the first input vector
- `[in] *src2` pointer of the second input vector
- `[in] size` number of elements in a vector

Return Value:

Dot product of the two input vectors

Note:

This function multiplies a `uint8_t` datum with another `uint8_t` datum and then adds the result to a `uint32_t` accumulator.

2.1.4.6 nds32_dprod_u8xq15

Declaration:

```
q31_t nds32_dprod_u8xq15(uint8_t *src1, q15_t *src2, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [in] size number of elements in a vector

Return Value:

Dot product of the two input vectors

Note:

This function multiplies a `uint8_t` datum with a `q15_t` datum and then adds the result to a `q31_t` accumulator.

2.1.5. Multiplication Functions

Multiplication functions multiply two elements from separate vectors and write the results one-by-one into a destination vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = src1[n] * src2[n];
```

Andes DSP library supports distinct multiplication functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.

2.1.5.1 nds32_mul_f32

Declaration:

```
void nds32_mul_f32 (float32_t *src1, float32_t *src2, float32_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

2.1.5.2 nds32_mul_q31

Declaration:

```
void nds32_mul_q31 (q31_t *src1, q31_t *src2, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The multiplication results will be saturated to the Q31 range [0x80000000, 0x7FFFFFFF].

2.1.5.3 nds32_mul_q15

Declaration:

```
void nds32_mul_q15 (q15_t *src1, q15_t *src2, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The multiplication results will be saturated to the Q15 range [0x8000, 0x7FFF].

2.1.5.4 nds32_mul_q7

Declaration:

```
void nds32_mul_q7 (q7_t *src1, q7_t *src2, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The multiplication results will be saturated to the Q7 range [0x80, 0x7F].

2.1.5.5 nds32_mul_u8_u16

Declaration:

```
void nds32_mul_u8_u16(uint8_t *src1, uint8_t *src2, uint16_t *dst, uint32_t  
size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the destination vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The multiplication results will be saved to `uint16_t *dst`.

2.1.6. Negate Functions

Negate functions negate elements of a source vector and write the results one-by-one into a destination vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = -src[n];
```

Andes DSP library supports distinct negate functions for the following data types: floating-point, Q31, Q15 and Q7. These functions are introduced in the subsections below.

2.1.6.1 nds32_neg_f32

Declaration:

```
void nds32_neg_f32 (float32_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.1.6.2 nds32_neg_q31

Declaration:

```
void nds32_neg_q31 (q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. The range of output results is [0x80000001, 0x7FFFFFFF].
2. If the input value is INT32_MIN (0x80000000), then the output value is INT32_MAX (0x7FFFFFFF).

2.1.6.3 nds32_neg_q15

Declaration:

```
void nds32_neg_q15 (q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. The range of output results is [0x8001, 0x7FFF].
2. If the input value is INT16_MIN (0x8000), then the output value is INT16_MAX (0x7FFF).

2.1.6.4 nds32_neg_q7

Declaration:

```
void nds32_neg_q7 (q7_t *src, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. The range of output results is [0x81, 0x7F].
2. If the input value is INT8_MIN (0x80), then the output value is INT8_MAX (0x7F).

2.1.7. Offset Functions

Offset functions add each element of a source vector with a constant number and write the results one-by-one into a destination vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = src[n] + offset;
```

Andes DSP library supports distinct offset functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.

2.1.7.1 nds32_offset_f32

Declaration:

```
void nds32_offset_f32 (float32_t *src, float32_t offset, float32_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] offset constant offset value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.1.7.2 nds32_offset_q31

Declaration:

```
void nds32_offset_q31 (q31_t *src, q31_t offset, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] offset constant offset value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The result value will be saturated to the Q31 range [0x80000000, 0x7FFFFFFF].

2.1.7.3 nds32_offset_q15

Declaration:

```
void nds32_offset_q15 (q15_t *src, q15_t offset, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] offset constant offset value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The result value will be saturated to the Q15 range [0x8000, 0x7FFF].

2.1.7.4 nds32_offset_q7

Declaration:

```
void nds32_offset_q7(q7_t *src, q7_t offset, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] offset constant offset value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The result value will be saturated to the Q7 range [0x80, 0x7F].

2.1.7.5 nds32_offset_u8

Declaration:

```
void nds32_offset_u8(uint8_t *src, q7_t offset, uint8_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] offset constant offset value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The result value will be saturated to the `uint8_t` range `[0x00, 0xFF]`.

2.1.8. Scale Functions

Scale functions multiply each element in a source vector by a constant scaling value and write the results one-by-one into a destination vector. In general, its behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = src[n] * scaling_value;
```

In cases of fractional presentation, a parameter `shift` is introduced to adjust the range of result values. For details about the parameter, please refer to functions with fractional data types (Section 2.1.8.2 to Section 2.1.8.5).

Andes DSP library supports distinct scale functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.

2.1.8.1 nds32_scale_f32

Declaration:

```
void nds32_scale_f32(float32_t *src, float32_t scale, float32_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] scale constant scaling value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Example:

```
#define size        2  
float32_t src[size] = {0.8167, 0.5};  
float32_t dst[size] = {0};  
float32_t scale = 0.4;  
  
nds32_scale_f32(src, scale, dst, size);
```

This example also serves as a reference for examples of Q31, Q15, or Q7 scale functions.

2.1.8.2 nds32_scale_q31

Declaration:

```
void nds32_scale_q31 (q31_t *src, q31_t scalefract, int8_t shift, q31_t *dst,
uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] scalefract constant fractional scaling value
- [in] shift bits for result adjustment (Please refer to Note 1. and 2. below)
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. Due to fractional presentation, the results should fall within the Q31 range. You can use the input parameter, `shift`, to adjust the result values. The value of `shift` can range from 0 to 31 in this function.
2. The behavior of this function can be roughly presented as follows:

```
dst[n] = (src[n] * scalefract) >> (31 - shift)
Where 0 <= n < size and 0 <= shift <= 31.
```

2.1.8.3 nds32_scale_q15

Declaration:

```
void nds32_scale_q15(q15_t *src, q15_t scalefract, int8_t shift, q15_t *dst,
uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] scalefract constant fractional scaling value
- [in] shift bits for result adjustment (Please refer to Note 1. and 2. below)
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. Due to fractional presentation, the results should fall within the Q15 range. You can use the input parameter, `shift`, to adjust the result values. The value of `shift` can range from 0 to 15 in this function.
2. The behavior of this function can be roughly presented as follows:

```
dst[n] = (src[n] * scalefract) >> (15 - shift)
Where 0 <= n < size and 0 <= shift <= 15.
```


2.1.8.4 nds32_scale_q7

Declaration:

```
void nds32_scale_q7(q7_t *src, q7_t scalefract, int8_t shift, q7_t *dst,
uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] scalefract constant fractional scaling value
- [in] shift bits for result adjustment (Please refer to Note 1. and 2. below)
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. Due to fractional presentation, the results should fall within the Q7 range. You can use the input parameter, `shift`, to adjust the result values. The value of `shift` can range from 0 to 7 in this function.
2. The behavior of this function can be roughly presented as follows:

```
dst[n] = (src[n] * scalefract) >> (7 - shift)
Where 0 <= n < size and 0 <= shift <= 7.
```

2.1.8.5 nds32_scale_u8

Declaration:

```
void nds32_scale_u8(uint8_t *src, q7_t scalefract, int8_t shift, uint8_t *dst,
uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] scalefract constant fractional scaling value
- [in] shift bits for result adjustment (Please refer to Note 1. and 2. below)
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. Since the data type for the destination vector is `uint8_t`, the scaled results will be saturated to the `uint8_t` range `[0x00, 0xFF]`. You can also use the input parameter, `shift`, to adjust the result values. The value of `shift` can range from 0 to 7 in this function.
2. The behavior of this function can be roughly presented as follows:

```
dst[n] = (src[n] * scalefract) >> (7 - shift)
Where 0 <= n < size and 0 <= shift <= 7.
```

2.1.9. Shift Functions

Shift functions shift each element in a source vector by a constant shift value and write the results one-by-one into a destination vector. The positive/negative shift value decides the left/right direction elements will be shifted to. In general, the behavior of shift functions can be defined as follows:

```
for (n = 0; n < size; n++)
{
    if (shift < 0)
        dst[n] = src[n] >> (-shift);
    else
        dst[n] = src[n] << (shift);
}
```

Andes DSP library supports distinct shift functions for Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.

2.1.9.1 nds32_shift_q31

Declaration:

```
void nds32_shift_q31(q31_t *src, int8_t shift, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] shift signed shift value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. Due to performance consideration, the shift value must be no greater than 32; otherwise, the result may be incorrect.
2. The result value will be saturated to the Q31 range [0x80000000, 0x7FFFFFFF].

Example:

```
#define size      1024
q31_t src[size] = {...};
q31_t dst[size];
q31_t shift = 1;
```

```
nds32_shift_q31(src, shift, dst, size);
```

This example also serves as a reference for examples of Q15 or Q7 shift functions.

2.1.9.2 nds32_shift_q15

Declaration:

```
void nds32_shift_q15 (q15_t *src, int8_t shift, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] shift signed shift value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. Due to performance consideration, the shift value must be no greater than 16; otherwise, the result may be incorrect.
2. The result value will be saturated to the Q15 range [0x8000, 0x7FFF].

2.1.9.3 nds32_shift_q7

Declaration:

```
void nds32_shift_q7(q7_t *src, int8_t shift, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] shift signed shift value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. Due to performance consideration, the shift value must be no greater than 24; otherwise, the result may be incorrect.
2. The result value will be saturated to the Q7 range [0x80, 0x7F].

2.1.9.4 nds32_shift_u8

Declaration:

```
void nds32_shift_u8(uint8_t *src, int8_t shift, uint8_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] shift signed shift value
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. Due to performance consideration, the shift value must be no greater than 24; otherwise, the result may be incorrect.
2. The result value will be saturated to the `uint8_t` range `[0x00, 0xFF]`.

2.1.10. Subtraction Functions

Subtraction functions subtract two elements from separate source vectors and write the results one-by-one into a destination vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = src1[n] - src2[n];
```

Andes DSP library supports distinct subtraction functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.

2.1.10.1 nds32_sub_f32

Declaration:

```
void nds32_sub_f32 (float32_t *src1, float32_t *src2, float32_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.1.10.2 nds32_sub_q31

Declaration:

```
void nds32_sub_q31 (q31_t *src1, q31_t *src2, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The results will be saturated to the Q31 range [0x80000000, 0x7FFFFFFF].

2.1.10.3 nds32_sub_q15

Declaration:

```
void nds32_sub_q15 (q15_t *src1, q15_t *src2, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The results will be saturated to the Q15 range [0x8000, 0x7FFF].

2.1.10.4 nds32_sub_q7

Declaration:

```
void nds32_sub_q7 (q7_t *src1, q7_t *src2, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The results will be saturated to the Q7 range [0x80, 0x7F].

2.1.10.5 nds32_sub_u8_q7

Declaration:

```
void nds32_sub_u8_q7(uint8_t *src1, uint8_t *src2, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] *src2 pointer of the second input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The results will be saturated to the Q7 range [0x80, 0x7F].

2.2. Complex Functions

Complex functions provide calculations for complex vectors. In Andes DSP library, the elements of complex vectors should be arranged as real and imaginary parts interleaved. That is, the memory layout should look like [real, imaginary, real, imaginary, ... , real, imaginary].

2.2.1. Complex Conjugate Functions

Complex conjugate functions calculate conjugation values of complex numbers from a source vector and write the results into a destination vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)
    dst[2 * n] = src[2 * n];
    dst[2 * n + 1] = -src[2 * n + 1];
```

Andes DSP library supports distinct complex conjugate functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.2.1.1 nds32_cconj_f32

Declaration:

```
void nds32_cconj_f32 (const float32_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output complex vector
- [in] size number of complex elements in a vector

Return Value:

None

Example:

With a set of complex vector containing three complex elements like $\{3 + 4i, 4 - 7i, -3 + 5i\}$, the complex conjugation is as follows:

```
//Complex vector is arranged with real and imaginary parts interleaved.
#define size      3
float32_t src[2 * size] = {3.0, 4.0, 4.0, -7.0, -3.0, 5.0};
float32_t dst[2 * size];
nds32_cconj_f32(src, dst, size);
```

This example also serves as a reference for examples of Q31 or Q15 complex conjugate functions.

2.2.1.2 nds32_cconj_q31

Declaration:

```
void nds32_cconj_q31 (const q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output complex vector
- [in] size number of complex elements in a vector

Return Value:

None

Note:

1. The real parts and imaginary parts in complex vectors are both in the Q31 range.
2. When negating the input imaginary part where the value is `INT32_MIN` (0x80000000), the output imaginary part will be `INT32_MAX` (0x7FFFFFFF) after complex conjugation.

2.2.1.3 nds32_cconj_q15

Declaration:

```
void nds32_cconj_q15 (const q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output complex vector
- [in] size number of complex elements in a vector

Return Value:

None

Note:

1. The real parts and imaginary parts in complex vectors are both in the Q15 range.
2. When negating the input imaginary part where the value is `INT16_MIN` (0x8000), the output imaginary part will be `INT16_MAX` (0x7FFF) after complex conjugation.

2.2.2. Complex Dot Product Functions

Complex dot product functions calculate the dot product of two complex vectors and write the results into a destination vector. There are two kinds of complex dot product functions in Andes DSP library.

The first type of complex dot product functions includes `nds32_cdprod_f32`, `nds32_cdprod_q31` and `nds32_cdprod_q15`. Their behavior can be described as follows:

```
for (n = 0; n < size; n++)
    a = src1[2 * n];
    b = src1[2 * n + 1];
    c = src2[2 * n];
    d = src2[2 * n + 1];
    dst[2 * n] = a * c + b * d;
    dst[2 * n + 1] = a * d - b * c;
```

The second type of complex dot product functions, named as complex dot product type 2 functions, includes `nds32_cdprod_typ2_f32`, `nds32_cdprod_typ2_q31` and `nds32_cdprod_typ2_q15`. Their behavior can be described as follows:

```
for (n = 0; n < size; n++)
    a = src1[2 * n];
    b = src1[2 * n + 1];
    c = src2[2 * n];
    d = src2[2 * n + 1];
    real_sum += a * c - b * d;
    image_sum += a * d + b * c;
```

Details of each complex dot product functions are introduced in the subsections below.

2.2.2.1 nds32_cdprod_f32

Declaration:

```
void nds32_cdprod_f32(const float32_t *src1, const float32_t *src2, uint32_t
size, float32_t *dst)
```

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [in] size number of complex elements in a vector
- [out] *dst pointer of the output complex vector

Return Value:

None

Example:

With two sets of complex vector each containing three complex numbers like $\{3 + 4i, 4 - 7i, -3 + 5i\}$ and $\{1 - 2i, 5 - 1i, -4 + 3i\}$, the complex dot production is as follows:

```
#define size            3
float32_t src1[2*size] = {3, 4, 4, -7, -3, 5};
float32_t src2[2*size] = {1, -2, 5, -1, -4, 3};
float32_t dst[2*size];
nds32_cdprod_f32(src1, src2, size, dst);
```

This example also serves as a reference for examples of Q31 or Q15 complex dot product functions.

2.2.2.2 nds32_cdprod_q31

Declaration:

```
void nds32_cdprod_q31 (const q31_t *src1, const q31_t *src2, uint32_t size,  
q31_t *dst)
```

Official
Release

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [in] size number of complex elements in a vector
- [out] *dst pointer of the output complex vector

Return Value:

None

Note:

The values written into the destination vector is in Q29 format.

2.2.2.3 nds32_cdprod_q15

Declaration:

```
void nds32_cdprod_q15 (const q15_t *src1, const q15_t *src2, uint32_t size, q15_t *dst)
```

Official
Release

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [in] size number of complex elements in a vector
- [out] *dst pointer of the output complex vector

Return Value:

None

Note:

The values written into the destination vector is in Q13 format.

2.2.2.4 nds32_cdprod_typ2_f32

Declaration:

```
void nds32_cdprod_typ2_f32(const float32_t *src1, const float32_t *src2,
uint32_t size, float32_t *rout, float32_t *iout)
```

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [in] size number of complex elements in a vector
- [out] *rout pointer of the real output
- [out] *iout pointer of the image output

Return Value:

None

Example:

With two sets of complex vector each containing three complex numbers like $\{3 + 4i, 4 - 7i, -3 + 5i\}$ and $\{1 - 2i, 5 - 1i, -4 + 3i\}$, the complex dot production of this function is as follows:

```
#define size      3
float32_t src1[2*size] = {3, 4, 4, -7, -3, 5};
float32_t src2[2*size] = {1, -2, 5, -1, -4, 3};
float32_t real_out, image_out;
nds32_cdprod_typ2_f32(src1, src2, size, &real_out, &image_out);
```

This example also serves as a reference for examples of Q31 or Q15 complex dot product type 2 functions.

2.2.2.5 nds32_cdprod_typ2_q31

Declaration:

```
void nds32_cdprod_typ2_q31(const q31_t *src1, const q31_t *src2,  
uint32_t size, q63_t *rout, q63_t *iout)
```

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [in] size number of complex elements in a vector
- [out] *rout pointer of the real output
- [out] *iout pointer of the image output

Return Value:

None

Note:

The values written into the two output variables are in Q48 format.

2.2.2.6 nds32_cdprod_typ2_q15

Declaration:

```
void nds32_cdprod_typ2_q15(const q15_t *src1, const q15_t *src2,  
uint32_t size, q31_t *rout, q31_t *iout)
```

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [in] size number of complex elements in a vector
- [out] *rout pointer of the real output
- [out] *iout pointer of the image output

Return Value:

None

Note:

The values written into the two output variables are in Q24 format.

2.2.3. Complex Magnitude Functions

Complex magnitude functions compute the magnitude of a complex vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = sqrt(src[2 * n]2 + src[2 * n + 1]2);
```

Andes DSP library supports distinct complex magnitude functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.2.3.1 nds32_cmag_f32

Declaration:

```
void nds32_cmag_f32 (const float32_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output vector
- [in] size number of complex elements in a vector

Return Value:

None

2.2.3.2 nds32_cmag_q31

Declaration:

```
void nds32_cmag_q31 (const q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output vector
- [in] size number of complex elements in a vector

Return Value:

None

Note:

The values written into the destination vector is in Q29 format.

2.2.3.3 nds32_cmag_q15

Declaration:

```
void nds32_cmag_q15 (const q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output vector
- [in] size number of complex elements in a vector

Return Value:

None

Note:

The values written into the destination vector is in Q13 format.

2.2.4. Complex Magnitude-Squared Function

Complex magnitude-squared functions compute the magnitude squared of complex numbers from a complex vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = src[2 * n]2 + src[2 * n + 1]2;
```

Andes DSP library supports distinct complex magnitude-squared functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.2.4.1 nds32_cmag_sqr_f32

Declaration:

```
void nds32_cmag_sqr_f32(const float32_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output vector
- [in] size number of complex elements in a vector

Return Value:

None

2.2.4.2 nds32_cmag_sqr_q31

Declaration:

```
void nds32_cmag_sqr_q31 (const q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output vector
- [in] size number of complex elements in a vector

Return Value:

None

Note:

The values written into the destination vector is in Q29 format.

2.2.4.3 nds32_cmag_sqr_q15

Declaration:

```
void nds32_cmag_sqr_q15 (const q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [out] *dst pointer of the output vector
- [in] size number of complex elements in a vector

Return Value:

None

Note:

The values written into the destination vector is in Q13 format.

2.2.5. Complex-Multiply-Complex Functions

Complex-multiply-complex functions compute the multiplication of two complex vectors and write the results into a destination vector. The behavior can be defined as follows:

A large, light blue, rounded rectangular watermark with the words "Official" and "Release" stacked vertically in a serif font is centered over the code block.

```
for (n = 0; n < size; n++)
    a = src1[2 * n];
    b = src1[2 * n + 1];
    c = src2[2 * n];
    d = src2[2 * n + 1];
    dst[2 * n] = a * c - b * d;
    dst[2 * n + 1] = a * d + b * c;
```

Andes DSP library supports distinct complex-multiply-complex functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.2.5.1 nds32_cmul_f32

Declaration:

```
void nds32_cmul_f32(const float32_t *src1, const float32_t *src2, float32_t
*dst, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [out] *dst pointer of the output complex vector
- [in] size number of complex elements in a vector

Return Value:

None

Example:

With two sets of complex vector each containing three complex numbers like $[3 + 4i, 4 - 7i, -3 + 5i]$ and $[1 - 2i, 5 - 1i, -4 + 3i]$, the multiplication of the two complex vectors is as follows:

```
#define size      3
float32_t src1[2*size] = {3, 4, 4, -7, -3, 5};
float32_t src2[2*size] = {1, -2, 5, -1, -4, 3};
float32_t dst[2*size];
nds32_cmul_f32(src1, src2, dst, size);
```

This example also serves as a reference for examples of Q31 or Q15 complex-multiply-complex functions.

2.2.5.2 nds32_cmul_q31

Declaration:

```
void nds32_cmul_q31 (const q31_t *src1, const q31_t *src2, q31_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [out] *dst pointer of the output complex vector
- [in] size number of complex elements in a vector

Return Value:

None

Note:

The values written into the destination vector is in Q29 format.

2.2.5.3 nds32_cmul_q15

Declaration:

```
void nds32_cmul_q15 (const q15_t *src1, const q15_t *src2, q15_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input complex vector
- [in] *src2 pointer of the second input complex vector
- [out] *dst pointer of the output complex vector
- [in] size number of complex elements in a vector

Return Value:

None

Note:

The values written into the destination vector is in Q13 format.

2.2.6. Complex-Multiply-Real Functions

Complex-multiply-real functions compute the multiplication of a complex vector by a real vector and write the results into a destination complex vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    r = real[n];  
    dst[2 * n] = src[2 * n] * r;  
    dst[2 * n + 1] = src[2 * n + 1] * r;
```

Andes DSP library supports distinct complex-multiply-real functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.2.6.1 nds32_cmul_real_f32

Declaration:

```
void nds32_cmul_real_f32 (const float32_t *src, const float32_t *real,
float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [in] *real pointer of the input real vector
- [out] *dst pointer of the output complex vector
- [in] size number of elements in a vector

Return Value:

None

Example:

With a complex vector containing three elements like $[3 + 4i, 4 - 7i, -3 + 5i]$ and a real vector containing three elements like $[1, -3, 2]$, the multiplication of the two vectors is as follows:

```
#define size      3
float32_t src[2*size] = {3, 4, 4, -7, -3, 5};
float32_t real[size] = {1, -3, 2};
float32_t dst[2*size];
nds32_cmul_real_f32(src, real, dst, size);
```

This example also serves as a reference for examples of Q31 or Q15 complex-multiply-real functions.

2.2.6.2 nds32_cmul_real_q31

Declaration:

```
void nds32_cmul_real_q31 (const q31_t *src, const q31_t *real, q31_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [in] *real pointer of the input real vector
- [out] *dst pointer of the output complex vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The results will be saturated to the Q31 range [0x80000000, 0x7FFFFFFF].

2.2.6.3 nds32_cmul_real_q15

Declaration:

```
void nds32_cmul_real_q15 (const q15_t *src, const q15_t *real, q15_t *dst,  
uint32_t size)
```

Parameters:

- [in] *src pointer of the input complex vector
- [in] *real pointer of the input real vector
- [out] *dst pointer of the output complex vector
- [in] size number of elements in a vector

Return Value:

None

Note:

The results will be saturated to the Q15 range [0x8000, 0x7FFF].

2.3. Controller Functions

2.3.1. Clarke Transform Functions

Clarke transform functions translate quantities from a three-phase system into a two-axis reference frame (alpha- and beta-axis), as illustrated in Figure 1:

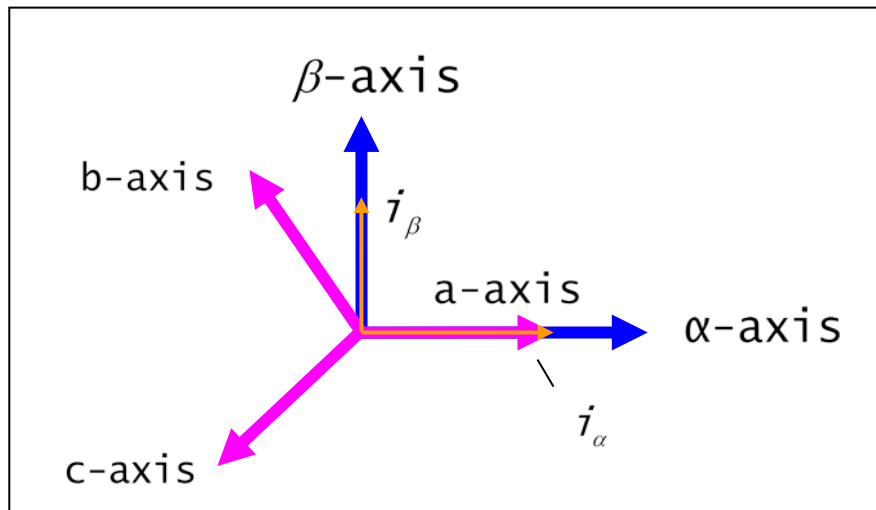


Figure 1. Clarke Transform Diagram

The behavior of Clarke transform can be defined as follows:

```
let  $I_a + I_b + I_c = 0$ 
such that,
isqrt3 =  $1 / \sqrt{3}$ ;
 $i_\alpha = I_a$ ;
 $i_\beta = \text{isqrt3} * I_a + 2 * \text{isqrt3} * I_b$ 
```

Andes DSP library supports distinct Clarke transform functions for floating-point and Q31 data types. The two functions are introduced in the subsections below.

2.3.1.1 nds32_clarke_f32

Declaration:

```
void nds32_clarke_f32(float32_t a, float32_t b, float32_t *alpha, float32_t *beta)
```

Official
Release

Parameters:

- [in] a quantity a of a three-phase system
- [in] b quantity b of a three-phase system
- [out] *alpha axis alpha of a two-phase reference frame
- [out] *beta axis beta of a two-phase reference frame

Return Value:

None

2.3.1.2 nds32_clarke_q31

Declaration:

```
void nds32_clarke_q31 (q31_t a, q31_t b, q31_t *alpha, q31_t *beta)
```

Parameters:

- [in] a quantity a of a three-phase system
- [in] b quantity b of a three-phase system
- [out] *alpha axis alpha of a two-phase reference frame
- [out] *beta axis beta of a two-phase reference frame

Return Value:

None

2.3.2. Inverse Clarke Transform Functions

Inverse Clarke transform functions translate a two-axis reference frame (alpha- and beta-axis) back to a three-phase system a, b and c. The behavior can be defined as follows:

```

$$I_a = i_\alpha;$$

$$I_b = -0.5 * i_\alpha + 0.5 * \sqrt{3} * i_\beta;$$

```

Andes DSP library supports distinct inverse Clarke transform functions for floating-point and Q31 data types. The two functions are introduced in the subsections below.

2.3.2.1 nds32_inv_clarke_f32

Declaration:

```
void nds32_inv_clarke_f32 (float32_t alpha, float32_t beta, float32_t *a,  
float32_t *b)
```

Parameters:

- [in] alpha axis alpha of a two-phase reference frame
- [in] beta axis beta of a two-phase reference frame
- [out] *a quantity a of a three-phase system
- [out] *b quantity b of a three-phase system

Return Value:

None

2.3.2.2 nds32_inv_clarke_q31

Declaration:

```
void nds32_inv_clarke_q31 (q31_t alpha, q31_t beta, q31_t *a, q31_t *b)
```

Parameters:

- [in] alpha axis alpha of a two-phase reference frame
- [in] beta axis beta of a two-phase reference frame
- [out] *a quantity a of a three-phase system
- [out] *b quantity b of a three-phase system

Return Value:

None

2.3.3. PID Functions

PID (Proportional-Integral-Derivative) controller functions provide a continuous-loop feedback mechanism to control output errors according to the inputs periodically. The state of PID controller is shown in the following diagram:

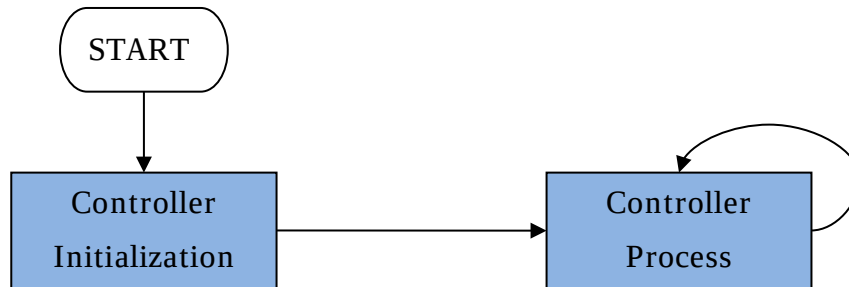


Figure 2. State of PID Controller

At the controller process when current data is input each time, the behavior of PID controller is as follows:

```

output = Kp * current_data + Ki * state0 + Kd * state1 + state2
state2 = output;
state1 = state0;
state0 = current_data;
  
```

Andes DSP library supports distinct PID initialization and PID functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.3.3.1 nds32_init_pid_f32

Declaration:

```
void nds32_init_pid_f32 (nds32_pid_f32_t *instance, int32_t set)
```

Parameters:

- `nds32_pid_f32_t` instance structure of the floating-point PID function, defined as follows:

```
typedef struct {
    float32_t gain1;
    float32_t gain2;
    float32_t gain3;
    float32_t state[3];
    float32_t Kp;
    float32_t Ki;
    float32_t Kd;
} nds32_pid_f32_t;
```

Where

- `gain1` is equal to $Kp + Ki + Kd$
 - `gain2` is equal to $-Kp - 2Kd$
 - `gain3` is equal to Kd
 - `state[3]` state buffer
 - `Kp` proportional value
 - `Ki` integral value
 - `Kd` derivative value
- `[in] *instance` pointer of the instance structure
 - `[in] set` do nothing if its value is zero; otherwise, clear the state buffer.

Return Value:

None

Note:

1. This function should be called to initialize the controller before the function `nds32_pid_f32` is called.
2. While parameters `gain1`, `gain2`, `gain3` and `state[3]` are managed by the controller, you have to input `Kp`, `Ki` and `Kd` on your own.

Example:

```
nds32_pid_f32_t instance;
instance.Kp = 0.02F;
instance.Ki = 0.03F;
instance.Kd = 0.014F;

//initialize controller
nds32_init_pid_f32(&instance, 1);

//Begin controller
while (1)
{
    f32_t output;
    f32_t input = ... read data from input port ...
    output = nds32_pid_f32(&instance, input);
    ...
}
```

This example also serves as a reference for examples of Q31 or Q15 PID functions.

2.3.3.2 nds32_pid_f32

Declaration:

```
float32_t nds32_pid_f32 (nds32_pid_f32_t *instance, float32_t src)
```

Parameters:

- **nds32_pid_f32_t** instance structure of the floating-point PID function, defined as follows:

```
typedef struct {
    float32_t gain1;
    float32_t gain2;
    float32_t gain3;
    float32_t state[3];
    float32_t Kp;
    float32_t Ki;
    float32_t Kd;
} nds32_pid_f32_t;
```

Where

- **gain1** is equal to $K_p + K_i + K_d$
- **gain2** is equal to $-K_p - 2K_d$
- **gain3** is equal to K_d
- **state[3]** state buffer
- **Kp** proportional value
- **Ki** integral value
- **Kd** derivative value
- **[in] *instance** pointer of the instance structure
- **[in] src** current data

Return Value:

Output of controller

Note:

The **nds32_init_pid_f32** function must be called to initialize the controller first, then the **nds32_pid_f32** function can be executed to do loop process of adjusting the controller's outputs with the current data. Please refer to Section 2.3.3.1 for an example.

2.3.3.3 nds32_init_pid_q31

Declaration:

```
void nds32_init_pid_q31 (nds32_pid_q31_t *instance, int32_t set)
```

Parameters:

- **nds32_pid_q31_t** instance structure of the Q31 PID function, defined as follows:

```
typedef struct {
    q31_t gain1;
    q31_t gain2;
    q31_t gain3;
    q31_t state[3];
    q31_t Kp;
    q31_t Ki;
    q31_t Kd;
} nds32_pid_q31_t;
```

Where

- **gain1** is equal to $K_p + K_i + K_d$
 - **gain2** is equal to $-K_p - 2K_d$
 - **gain3** is equal to K_d
 - **state[3]** state buffer
 - **Kp** proportional value
 - **Ki** integral value
 - **Kd** derivative value
- **[in] *instance** pointer of the instance structure
 - **[in] set** do nothing if its value is zero; otherwise, clear the state buffer.

Return Value:

None

Note:

1. This function should be called to initialize the controller before the function **nds32_pid_q31** is called.
2. While parameters **gain1**, **gain2**, **gain3** and **state[3]** are managed by the controller, you have to input **Kp**, **Ki** and **Kd** on your own.

2.3.3.4 nds32_pid_q31

Declaration:

```
q31_t nds32_pid_q31 (nds32_pid_q31_t *instance, q31_t src)
```

Parameters:

- **nds32_pid_q31_t** instance structure of the Q31 PID function, defined as follows:

```
typedef struct {
    q31_t gain1;
    q31_t gain2;
    q31_t gain3;
    q31_t state[3];
    q31_t Kp;
    q31_t Ki;
    q31_t Kd;
} nds32_pid_q31_t;
```

Where

- **gain1** is equal to $K_p + K_i + K_d$
- **gain2** is equal to $-K_p - 2K_d$
- **gain3** is equal to K_d
- **state[3]** state buffer
- **Kp** proportional value
- **Ki** integral value
- **Kd** derivative value
- **[in] *instance** pointer of the instance structure
- **[in] src** current data

Return Value:

Output of controller

Note:

The **nds32_init_pid_q31** function must be called to initialize the controller first, then the **nds32_pid_q31** function can be executed to do loop process of adjusting the controller's outputs with the current data. Please refer to Section 2.3.3.1 for an example.

2.3.3.5 nds32_init_pid_q15

Declaration:

```
void nds32_init_pid_q15 (nds32_pid_q15_t *instance, int32_t set)
```

Parameters:

- `nds32_pid_q15_t` instance structure of the Q15 PID function, defined as follows:

```
typedef struct {
    q15_t gain1;
    q15_t gain2;
    q15_t gain3;
    q15_t state[3];
    q15_t Kp;
    q15_t Ki;
    q15_t Kd;
} nds32_pid_q15_t;
```

Where

- `gain1` is equal to $Kp + Ki + Kd$
 - `gain2` is equal to $-Kp - 2Kd$
 - `gain3` is equal to Kd
 - `state[3]` state buffer
 - `Kp` proportional value
 - `Ki` integral value
 - `Kd` derivative value
- `[in] *instance` pointer of the instance structure
 - `[in] set` do nothing if its value is zero; otherwise, clear the state buffer.

Return Value:

None

Note:

1. This function should be called to initialize the controller before the function `nds32_pid_q15` is called.
2. While parameters `gain1`, `gain2`, `gain3` and `state[3]` are managed by the controller, you have to input `Kp`, `Ki` and `Kd` on your own.

2.3.3.6 nds32_pid_q15

Declaration:

```
q15_t nds32_pid_q15 (nds32_pid_q15_t *instance, q15_t src)
```

Parameters:

- `nds32_pid_q15_t` instance structure of the Q15 PID function, defined as follows:

```
typedef struct {
    q15_t gain1;
    q15_t gain2;
    q15_t gain3;
    q15_t state[3];
    q15_t Kp;
    q15_t Ki;
    q15_t Kd;
} nds32_pid_q15_t;
```

Where

- `gain1` is equal to $K_p + K_i + K_d$
- `gain2` is equal to $-K_p - 2K_d$
- `gain3` is equal to K_d
- `state[3]` state buffer
- `Kp` proportional value
- `Ki` integral value
- `Kd` derivative value
- `[in] *instance` pointer of the instance structure
- `[in] src` current data

Return Value:

Output of controller

Note:

The `nds32_init_pid_q15` function must be called to initialize the controller first, then the `nds32_pid_q15` function can be executed to do loop process of adjusting the controller's outputs with the current data. Please refer to Section 2.3.3.1 for an example.

2.4. Filtering Functions

2.4.1. Biquadratic Cascade Filter (Direct Form I) Functions

Biquadratic cascade filter functions provide biquadratic filters. Andes DSP library supports direct form I biquadratic cascade filters using the following equation:

$$\text{dst}[n] = b0 * \text{src}[n] + b1 * \text{src}[n-1] + b2 * \text{src}[n-2] + a1 * \text{dst}[n-1] + a2 * \text{dst}[n-2];$$

In the equation, 5 coefficients and 4 state variables are used per stage. The coefficients **b0**, **b1** and **b2** determine zeros; the coefficients **a1** and **a2** determine the positions of poles.

Andes DSP library supports distinct direct form I biquadratic cascade filter functions and instance structures for floating-point, Q31, Q15 and also 32x64 Q31 to meet high precision. Instance structures are used to store information like stages and coefficients, and save the state in each stage. Before you use biquadratic cascade filter functions, make sure you initialize the corresponding instance structures first. These functions and their instance structures are introduced in the subsections below.

2.4.1.1 nds32_bq_df1_f32

Declaration:

```
void nds32_bq_df1_f32 (const nds32_bq_df1_f32_t *instance, float32_t *src,
float32_t *dst, uint32_t size)
```

Parameters:

- **nds32_bq_df1_f32_t** instance structure of the floating-point biquadratic cascade filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    float32_t *state;
    float32_t *coeff;
} nds32_bq_df1_f32_t;
```

Where

- **nstage** stages in the filter
- ***state** pointer of the state vector whose size is **4*nstage**
- ***coeff** pointer of the coefficient vector whose size is **5*nstage**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples in each stage

Return Value:

None

Example:

```
#define nstage      3
#define size        6
float32_t state[4 * nstage] = {0.0};
float32_t coeff[5 * nstage] = {0.40, 0.10, 0.24, -0.40, -0.34, 0.20, 0.06,
                                0.28, -0.04, -0.20, 0.08, 0.40, 0.60, -1.00, -0.14};
float32_t src[size] = {1.0, 0.5, 0.4, -0.1, -0.1, 0.3};
float32_t dst[size];
nds32_bq_df1_f32_t instance = {nstage, state, coeff};
nds32_bq_df1_f32(&instance, src, dst, size);
```


2.4.1.2 nds32_bq_df1_q31

Declaration:

```
void nds32_bq_df1_q31(const nds32_bq_df1_q31_t *instance, q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- **nds32_bq_df1_q31_t** instance structure of the Q31 biquadratic cascade filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    q31_t *state;
    q31_t *coeff;
    int8_t shift;
} nds32_bq_df1_q31_t;
```

Where

- **nstage** stages in the filter
- ***state** pointer of the state vector whose size is $4 \times \text{nstage}$
- ***coeff** pointer of the coefficient vector whose size is $5 \times \text{nstage}$
- **shift** shift bits
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples in each stage

Return Value:

None

Example:

```
#define shift      0
#define nstage     3
#define size       6
q31_t state_q31[4 * nstage] = {0} ;
q31_t coeff_q31[5 * nstage], src_q31[size], dst_q31[size];
nds32_bq_df1_q31_t instance_q31 = {nstage, state_q31, coeff_q31, shift};
float32_t coeff_f32[5 * nstage] = {0.40, 0.10, 0.24, -0.40, -0.34, 0.20, 0.06,
                                     0.28, -0.04, -0.20, 0.08, 0.40, 0.60, -1.00, -0.14};
```

```
float32_t src_f32[size] = {1.0, 0.5, 0.4, -0.1, -0.1, 0.3};  
nds32_convert_f32_q31(coeff_f32, coeff_q31, 5 * nstage);  
nds32_convert_f32_q31(src_f32, src_q31, size);  
nds32_bq_df1_q31(&instance_q31, src_q31, dst_q31, size);
```

This Q31 biquadratic cascade filter function example uses the `nds32_convert_f32_q31` function to convert the **F32** vector to **Q31**. You may also reference this example for the Q15 biquadratic cascade filter function, but make sure you replace `nds32_convert_f32_q31` with `nds32_convert_f32_q15`.

2.4.1.3 nds32_bq_df1_q15

Declaration:

```
void nds32_bq_df1_q15 (const nds32_bq_df1_q15_t *instance, q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- **nds32_bq_df1_q15_t** instance structure of the Q15 biquadratic cascade filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    q15_t *state;
    q15_t *coeff;
    int8_t shift;
} nds32_bq_df1_q15_t;
```

Where

- **nstage** stages in the filter
- ***state** pointer of the state vector whose size is **4*nstage**
- ***coeff** pointer of the coefficient vector whose size is **5*nstage** or **6*nstage** when using DSP extension. Please see “Note” below.
- **shift** shift bits
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples in each stage

Return Value:

None

Note:

The coefficient vector stores coefficients in the order {b0, b1, b2, a1, a2, ...}. The first 5 elements are for the first stage, the next 5 are for the next stage and so on. If using DSP extension, the order turns to be {b0, 0, b1, b2, a1, a2, ...} in which one zero (0) element is inserted between b0 and b1 for SIMD load/store instruction. In this case, the first 6 elements are for the first stage, the next 6 are for the next stage and so on.

2.4.1.4 nds32_bq_df1_32x64_q31

Declaration:

```
void nds32_bq_df1_32x64_q31 (const nds32_bq_df1_32x64_q31_t *instance,
q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- `nds32_bq_df1_32x64_q31_t` instance structure of the 32x64 Q31 biquadratic cascade filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    q63_t *state;
    q31_t *coeff;
    int8_t shift;
} nds32_bq_df1_32x64_q31_t;
```

Where

- `nstage` stages in the filter
- `*state` pointer of the state vector whose size is `4*nstage`
- `*coeff` pointer of the coefficient vector whose size is `5*nstage`
- `shift` shift bits
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples in each stage

Return Value:

None

2.4.2. Biquadratic Cascade Filter (Direct Form II Transposed) Functions

Biquadratic cascade filter functions provide biquadratic filters. Andes DSP library supports direct form II transposed biquadratic cascade filters using the following equation:



```
for (n = 0; n < size; n++)
    out[n] = b0 * in[n] + d0;
    d0 = b1 * in[n] + a1 * out[n] + d1;
    d1 = b2 * in[n] + a2 * out[n];
```

In the equation, 5 coefficients (**b0**, **b1**, **b2**, **a1** and **a2**) and 2 state variables (**d0** and **d1**) are used per stage. After the completion of each stage, the 2 state variables **d0** and **d1** are saved into a state vector.

Andes DSP library also supports a direct form II transposed biquadratic cascade filter for stereo (2 channels) signals. Its equation is as below:

```
for (n = 0; n < size; n+=2)
    out[n] = b0 * in[n] + d00;
    out[n + 1] = b0 * in[n + 1] + d01;
    d00 = b1 * in[n] + a1 * out[n] + d10;
    d01 = b1 * in[n + 1] + a1 * out[n + 1] + d11;
    d10 = b2 * in[n] + a2 * out[n];
    d11 = b2 * in[n + 1] + a2 * out[n + 1];
```

In this equation, 5 coefficients (**b0**, **b1**, **b2**, **a1** and **a2**) and 4 state variables (**d00**, **d01**, **d10** and **d11**) are used per stage. After the completion of each stage, the 4 state variables are saved into a state vector.

Andes DSP library supports distinct direct form II transposed biquadratic cascade filter functions and instance structures only for floating-point data types, F32 and F64. Instance structures are used to store information like stages and coefficients, and save the state in each stage. Before you use biquadratic cascade filter functions, make sure you initialize the corresponding instance structures first. These functions and their instance structures are introduced in the subsections below.

2.4.2.1 nds32_bq_df2T_f32

Declaration:

```
void nds32_bq_df2T_f32(const nds32_bq_df2T_f32_t *instance, float32_t *src,
float32_t *dst, uint32_t size)
```

Parameters:

- `nds32_bq_df2T_f32_t` instance structure of the F32 biquadratic cascade filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    float32_t *state;
    float32_t *coeff;
} nds32_bq_df2T_f32_t;
```

Where

- `nstage` stages in the filter
- `*state` pointer of the state vector whose size is $2 \times \text{nstage}$
- `*coeff` pointer of the coefficient vector whose size is $5 \times \text{nstage}$
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples in each stage

Return Value:

None

Example:

```
#define nstage      3
#define size        6
float32_t state[2 * nstage] = {0.0};
float32_t coeff[5 * nstage] = {0.40, 0.10, 0.24, -0.40, -0.34, 0.20, 0.06,
                                0.28, -0.04, -0.20, 0.08, 0.40, 0.60, -1.00, -0.14};
float32_t src[size] = {1, 0.5, 0.4, -0.1, -0.1, 0.3};
float32_t dst[size];
nds32_bq_df2T_f32_t inst = {nstage, state, coeff};
nds32_bq_df2T_f32(&inst, src, dst, size);
```

This example also serves as a reference for examples of the F64 direct form II transposed biquadratic cascade filter function.

2.4.2.2 nds32_bq_df2T_f64

Declaration:

```
void nds32_bq_df2T_f64(const nds32_bq_df2T_f64_t *instance, float64_t *src,
float64_t *dst, uint32_t size)
```

Parameters:

- `nds32_bq_df2T_f64_t` instance structure of the F64 biquadratic cascade filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    float64_t *state;
    float64_t *coeff;
} nds32_bq_df2T_f64_t;
```

Where

- `nstage` stages in the filter
- `*state` pointer of the state vector whose size is `2*nstage`
- `*coeff` pointer of the coefficient vector whose size is `5*nstage`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples in each stage

Return Value:

None

2.4.2.3 nds32_bq_stereo_df2T_f32

Declaration:

```
void nds32_bq_stereo_df2T_f32(const nds32_bq_stereo_df2T_f32_t *instance,
float32_t *src, float32_t *dst, uint32_t size);
```

Parameters:

- `nds32_bq_stereo_df2T_f32_t` instance structure of the F32 biquadratic cascade filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    float32_t *state;
    float32_t *coeff;
} nds32_bq_stereo_df2T_f32_t;
```

Where

- `nstage` stages in the filter
- `*state` pointer of the state vector whose size is `4*nstage`
- `*coeff` pointer of the coefficient vector whose size is `5*nstage`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples in each stage

Return Value:

None

Example:

```
#define nstage      3
#define size        1024
float32_t state[4 * nstage] = {0.0};
float32_t coeff[5 * nstage] = {0.40, 0.10, 0.24, -0.40, -0.34, 0.20, 0.06,
                                0.28, -0.04, -0.20, 0.08, 0.40, 0.60, -1.00, -0.14};
float32_t src[size * 2] = {...};
float32_t dst[size * 2];
nds32_bq_stereo_df2T_f32_t inst = {nstage, state, coeff};
nds32_bq_stereo_df2T_f32(&inst, src, dst, size);
```


2.4.3. Convolution Functions

Convolution functions combine two signals into one signal mathematically. The convolution equation can be presented as:

$$dst[n] = \sum_{k=0}^{len1} (src1[k] \times src2[n - k])$$

Where the size of `src1[n]` is `len1`, the size of `src2[n]` is `len2`, and the size of `dst[n]` is `len1 + len2 - 1`.

Andes DSP library supports distinct convolution functions for the following data types: floating-point, Q31, Q15 and Q7. These functions are introduced in the subsections below.

2.4.3.1 nds32_conv_f32

Declaration:

```
void nds32_conv_f32 (float32_t *src1, uint32_t len1, float32_t *src2,
uint32_t len2, float32_t *dst)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector

Return Value:

None

Example:

```
#define len1        6
#define len2        4
float32_t src1[len1] = {0.3, 0.2, -0.1, 0.2, 0.4, 0.2};
float32_t src2[len2] = {-0.1, 0.3, 0.2, -0.2};
uint32_t dst[len1 + len2 - 1];
nds32_conv_f32(src1, len1, src2, len2, dst);
```

This example also serves as a reference for examples of Q31, Q15 or Q7 convolution functions.

2.4.3.2 nds32_conv_q31

Declaration:

```
void nds32_conv_q31(q31_t *src1, uint32_t len1, q31_t *src2, uint32_t len2, q31_t *dst)
```

Official
Release

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector

Return Value:

None

2.4.3.3 nds32_conv_q15

Declaration:

```
void nds32_conv_q15(q15_t *src1, uint32_t len1, q15_t *src2, uint32_t len2, q15_t *dst)
```

Official
Release

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector

Return Value:

None

2.4.3.4 nds32_conv_q7

Declaration:

```
void nds32_conv_q7 (q7_t *src1, uint32_t len1, q7_t *src2, uint32_t len2,  
q7_t *dst)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector

Return Value:

None

2.4.4. Partial Convolution Functions

With a starting index and a size specified, partial convolution functions do the convolution partially for two signals. The partial convolution result will be generated to the destination vector in the range `[start_index, ..., start_index + size - 1]`. Thus, if the partial convolution result is not within the range `[0, ..., len1 + len2 - 2]`, a return value of `-1` will be given for the argument error.

Andes DSP library supports distinct partial convolution functions for the following data types: floating-point, Q31, Q15 and Q7. These functions are introduced in the subsections below.

2.4.4.1 nds32_conv_partial_f32

Declaration:

```
int32_t nds32_conv_partial_f32 (float32_t *src1, uint32_t len1, float32_t
*src2, uint32_t len2, float32_t *dst, uint32_t startindex, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector
- [in] startindex starting index of the partial convolution
- [in] size length of the partial convolution

Return Value:

- 0 success
- 1 failure

Example:

```
#define len1        6
#define len2        4
float32_t src1[len1] = {0.3, 0.2, -0.1, 0.2, 0.4, 0.2};
float32_t src2[len2] = {-0.1, 0.3, 0.2, -0.2};
uint32_t dst[len1 + len2 - 1];
nds32_conv_partial_f32(src1, len1, src2, len2, dst, 3, 4);
```

This example also serves as a reference for examples of Q31, Q15 or Q7 partial convolution functions.

2.4.4.2 nds32_conv_partial_q31

Declaration:

```
int32_t nds32_conv_partial_q31 (q31_t *src1, uint32_t len1, q31_t *src2,
uint32_t len2, q31_t *dst, uint32_t startindex, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector
- [in] startindex starting index of the partial convolution
- [in] size length of the partial convolution

Return Value:

- 0 success
- 1 failure

2.4.4.3 nds32_conv_partial_q15

Declaration:

```
int32_t nds32_conv_partial_q15(q15_t *src1, uint32_t len1, q15_t *src2,
uint32_t len2, q15_t *dst, uint32_t startindex, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector
- [in] startindex starting index of the partial convolution
- [in] size length of the partial convolution

Return Value:

- 0 success
- 1 failure

2.4.4.4 nds32_conv_partial_q7

Declaration:

```
int32_t nds32_conv_partial_q7(q7_t *src1, uint32_t len1, q7_t *src2, uint32_t len2, q7_t *dst, uint32_t startindex, uint32_t size)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector
- [in] startindex starting index of the partial convolution
- [in] size length of the partial convolution

Return Value:

- 0 success
- 1 failure

2.4.5. Correlation Functions

Correlation functions use two signals to form another signal (also called cross-correlation) mathematically. The mathematical operations of correlation functions are similar to those of convolution functions and can be presented as:

$$dst[n] = \sum_{k=0}^{len1} (src1[k] \times src2[k - n])$$

Where `src1` and `src2` are input vectors, and their lengths are `len1` and `len2` separately; `dst` is the output vector and its size is `2 * max (len1, len2) - 1`.

Andes DSP library supports distinct correlation functions for the following data types: floating-point, Q31, Q15 and Q7. These functions are introduced in the subsections below.

2.4.5.1 nds32_corr_f32

Declaration:

```
void nds32_corr_f32 (float32_t *src1, uint32_t len1, float32_t *src2,
uint32_t len2, float32_t *dst)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector

Return Value:

None

Example:

```
#define len1 7
#define len2 5
float32_t src1[len1] = {0.3, 0.2, -0.1, 0.2, 0.4, 0.2, -0.1};
float32_t src2[len2] = {-0.1, 0.3, 0.2, -0.2, -0.2};
float32_t dst[2 * len1 - 1];
nds32_conv_f32(src1, len1, src2, len2, dst);
```

This example also serves as a reference for examples of Q31, Q15 or Q7 correlation functions.

2.4.5.2 nds32_corr_q31

Declaration:

```
void nds32_corr_q31(q31_t *src1, uint32_t len1, q31_t *src2, uint32_t len2,  
q31_t *dst)
```

Official
Release

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector

Return Value:

None

2.4.5.3 nds32_corr_q15

Declaration:

```
void nds32_corr_q15(q15_t *src1, uint32_t len1, q15_t *src2, uint32_t len2, q15_t *dst)
```

Official
Release

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector

Return Value:

None

2.4.5.4 nds32_corr_q7

Declaration:

```
void nds32_corr_q7 (q7_t *src1, uint32_t len1, q7_t *src2, uint32_t len2, q7_t *dst)
```

Parameters:

- [in] *src1 pointer of the first input vector
- [in] len1 length of the first input vector
- [in] *src2 pointer of the second input vector
- [in] len2 length of the second input vector
- [out] *dst pointer of the output vector

Return Value:

None

2.4.6. Finite Impulse Response (FIR) Filter Fncions

Finite impulse response (FIR) filter functions can be presented by the following equation:

$$dst[n] = \sum_{k=0}^{Size} (b[k] * src[n - k])$$

Where $b[k]$ is the filter coefficient and $Size$ is the number of filter coefficients.

Andes DSP library supports distinct FIR filter functions and instance structures for the following data types: floating-point, Q31, Q15 and Q7. Before you use FIR filter functions, make sure you initialize the corresponding instance structures first. These functions and their instance structures are introduced in the subsections below.

2.4.6.1 nds32_fir_f32

Declaration:

```
void nds32_fir_f32 (const nds32_fir_f32_t *instance, float32_t *src,
float32_t *dst, uint32_t size)
```

Parameters:

- `nds32_fir_f32_t` instance structure of the floating-point FIR filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    float32_t *state;
    float32_t *coeff;
} nds32_fir_f32_t;
```

Where

- `coeff_size` number of coefficients
- `*state` pointer of the state vector whose size is `coeff_size + size - 1`
- `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples

Return Value:

None

Example:

```
#define coeff_size      4
#define size            6
float32_t state[coeff_size + size - 1];
float32_t coeff[coeff_size] = {0.3, -0.1, 0.4, 0.5};
nds32_fir_f32_t inst = {coeff_size, state, coeff};
float32_t src[size] = {0.1, -0.2, 0.2, 0.3, 0.4, 0.1};
float32_t dst[size];
nds32_fir_f32(&inst, src, dst, size);
```

This example also serves as a reference for examples of Q31, fast_Q31, Q15, fast_Q15 or Q7 FIR filter functions.

2.4.6.2 nds32_fir_q31

Declaration:

```
void nds32_fir_q31(const nds32_fir_q31_t *instance, q31_t *src, q31_t *dst,
uint32_t size)
```

Parameters:

- `nds32_fir_q31_t` instance structure of the Q31 FIR filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q31_t *state;
    q31_t *coeff;
} nds32_fir_q31_t;
```

Where

- `coeff_size` number of coefficients
- `*state` pointer of the state vector whose size is `coeff_size + size - 1`
- `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples

Return Value:

None

Note:

Both coefficients and state variables are represented in Q31 format and multiplications yield a Q2.62 result. These results are right-shifted by 31 bits, saturated to Q31 format and saved into buffer. Thus, in order to avoid overflow, the input signal must be scaled down by $\log_2(\text{coeff_size})$ bits before this function is called. When compared with `nds32_fir_fast_q31`, this function delivers higher precision and smaller code size.

2.4.6.3 nds32_fir_fast_q31

Declaration:

```
void nds32_fir_fast_q31 (const nds32_fir_q31_t *instance, q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- `nds32_fir_q31_t` instance structure of the Q31 FIR filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q31_t *state;
    q31_t *coeff;
} nds32_fir_q31_t;
```

Where

- `coeff_size` number of coefficients
 - `*state` pointer of the state vector whose size is `coeff_size + size - 1`
 - `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `[in] *instance` pointer of the instance structure
 - `[in] *src` pointer of the input vector
 - `[out] *dst` pointer of the output vector
 - `[in] size` number of samples

Return Value:

None

Note:

Both coefficients and state variables are represented in Q31 format and multiplications yield a Q1.31 result. These results are left-shifted by 1 bit and saved into buffer. Thus, in order to avoid overflow, the input signal must be scaled down by $\log_2(\text{coeff_size})$ bits before this function is called. When compared with `nds32_fir_q31`, this function delivers higher performance with lower precision and larger code size.

2.4.6.4 nds32_fir_q15

Declaration:

```
void nds32_fir_q15(const nds32_fir_q15_t *instance, q15_t *src, q15_t *dst,
uint32_t size)
```

Parameters:

- `nds32_fir_q15_t` instance structure of the Q15 FIR filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q15_t *state;
    q15_t *coeff;
} nds32_fir_q15_t;
```

Where

- `coeff_size` number of coefficients
- `*state` pointer of the state vector whose size is `coeff_size + size - 1`
- `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples

Return Value:

None

Note:

Both coefficients and state variables are represented in Q15 format and multiplications yield a Q30 result. The results are accumulated in a 64-bit accumulator in Q34.30 format and then truncated to Q34.15 format by discarding the low 15 bits. Lastly, the outputs are saturated to yield a result in Q1.15 format. When compared with `nds32_fir_fast_q15`, this function delivers higher precision and smaller code size.

2.4.6.5 nds32_fir_fast_q15

Declaration:

```
void nds32_fir_fast_q15 (const nds32_fir_q15_t *instance, q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- `nds32_fir_q15_t` instance structure of the Q15 FIR filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q15_t *state;
    q15_t *coeff;
} nds32_fir_q15_t;
```

Where

- `coeff_size` number of coefficients
 - `*state` pointer of the state vector whose size is `coeff_size + size - 1`
 - `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `[in] *instance` pointer of the instance structure
 - `[in] *src` pointer of the input vector
 - `[out] *dst` pointer of the output vector
 - `[in] size` number of samples

Return Value:

None

Note:

Both coefficients and state variables are represented in Q15 format and multiplications yield a Q30 result. The results are accumulated in a 32-bit accumulator in Q2.30 format and then saturated to yield a result in Q1.15 format. When compared with `nds32_fir_q15`, this function delivers higher performance with lower precision and larger code size.

2.4.6.6 nds32_fir_q7

Declaration:

```
void nds32_fir_q7 (const nds32_fir_q7_t *instance, q7_t *src, q7_t *dst,
uint32_t size)
```

Parameters:

- `nds32_fir_q7_t` instance structure of the Q7 FIR filter. defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q7_t *state;
    q7_t *coeff;
} nds32_fir_q7_t;
```

Where

- `coeff_size` number of coefficients
- `*state` pointer of the state vector whose size is `coeff_size + size - 1`.
- `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples

Return Value:

None

Note:

Both inputs are in Q7 format and multiplications yield a Q14 result. The intermediate results are accumulated in a 32-bit accumulator in Q18.14 format. The result is then converted to Q18.7 format by discarding the low 7 bits and then saturated to Q1.7 format.

2.4.7. Decimation FIR Filter Functions

Decimation FIR filter functions implement FIR filters with decimation by an integer factor **M**. They can be presented by the following equation:

$$dst[n] = \sum_{k=0}^{Size} (b[k] * src[n - k])$$

Where **b[k]** is the filter coefficient, **src** is the input data of the length **Size**, and **dst** is the output data of the length **Size/M**. **Size** must be a multiple of the decimation factor **M** so that the number of output samples can be an integer.

Andes DSP library supports distinct decimation FIR filter functions and instance structures for the following data types: floating-point, Q31 and Q15. Before you use decimation FIR filter functions, make sure you initialize the corresponding instance structure first. These functions and their instance structures are introduced in the subsections below.

2.4.7.1 nds32_dcmfir_f32

Declaration:

```
void nds32_dcmfir_f32 (const nds32_dcmfir_f32_t *instance, float32_t *src,
float32_t *dst, uint32_t size)
```

Parameters:

- `nds32_dcmfir_f32_t` instance structure of the floating-point decimation FIR filter, defined as follows:

```
typedef struct {
    uint32_t M;
    uint32_t coeff_size;
    float32_t *coeff;
    float32_t *state;
} nds32_dcmfir_f32_t;
```

Where

- `M` decimation factor
- `coeff_size` number of filter coefficients
- `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `*state` pointer of the state vector whose size is `coeff_size + size - 1`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples

Return Value:

None

Example:

With the input size as 24, the decimation factor `M` as 4 and the filter coefficient size as 6, set the instance of the decimation FIR filter as:

```
#define size          24
#define M             4
#define coeff_size    6
float32_t coeff[coeff_size] = {...};
```


Andes DSP Library V3 User Manual

```
float32_t state[coeff_size + size - 1];  
nds32_dcmfir_f32_t inst = {M, coeff_size, coeff, state};  
float32_t src[size] = {0.1, -0.2, 0.2, 0.3, 0.4, 0.1, ...};  
float32_t dst[size / M];  
nds32_dcmfir_f32(&inst, src, dst, size);
```

This example also serves as a reference for examples of Q31 or Q15 decimation FIR filter functions.

2.4.7.2 nds32_dcmfir_q31

Declaration:

```
void nds32_dcmfir_q31(const nds32_dcmfir_q31_t *instance, q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- **nds32_dcmfir_q31_t** instance structure of the Q31 decimation FIR filter, defined as follows:

```
typedef struct {
    uint32_t M;
    uint32_t coeff_size;
    q31_t *coeff;
    q31_t *state;
} nds32_dcmfir_q31_t;
```

Where

- **M** decimation factor
- **coeff_size** number of filter coefficients
- ***coeff** pointer of the time reversed coefficient vector whose size is **coeff_size**
- ***state** pointer of the state vector whose size is **coeff_size + size - 1**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples

Return Value:

None

Note:

Both coefficients and state variables are represented in Q31 format. A 64-bit accumulator is used to do multiply-accumulate operations with saturation to yield a Q1.62 result. After all operations are performed, the results are saturated to Q31 format.

2.4.7.3 nds32_dcmfir_q15

Declaration:

```
void nds32_dcmfir_q15(const nds32_dcmfir_q15_t *instance, q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- **nds32_dcmfir_q15_t** instance structure of the Q15 decimation FIR filter, defined as follows:

```
typedef struct {
    uint32_t M;
    uint32_t coeff_size;
    q15_t *coeff;
    q15_t *state;
} nds32_dcmfir_q15_t;
```

Where

- **M** decimation factor
- **coeff_size** number of filter coefficients
- ***coeff** pointer of the time reversed coefficient vector whose size is **coeff_size**
- ***state** pointer of the state vector whose size is **coeff_size + size - 1**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples

Return Value:

None

Note:

Both coefficients and state variables are represented in Q15 format. A 64-bit accumulator is used to do multiply-accumulate operations with saturation to yield a Q33.30 result. After all operations are performed, the results are saturated to Q15 format.

2.4.8. Lattice FIR Filter Functions

Lattice FIR filter functions can be represented by the following equations:

$$\begin{aligned} y_0[n] &= u_0[n] = x[n] \\ y_z[n] &= y_{z-1}[n] + k_z u_{z-1}[n-1] \\ u_z[n] &= k_z y_{z-1}[n] + u_{z-1}[n-1] \end{aligned}$$

Where x is the input, y is the output, u is the state of previous inputs, $0 \leq z < M$ and M is the stage number, and k_z is the reflection coefficients.

Andes DSP library supports distinct lattice FIR filter functions and instance structures for the following data types: floating-point, Q31 and Q15. Before you use lattice FIR filter functions, make sure you initialize the corresponding instance structure first. These functions and their instance structures are introduced in the subsections below.

2.4.8.1 nds32_lfir_f32

Declaration:

```
void nds32_lfir_f32 (const nds32_lfir_f32_t *instance, float32_t *src,
float32_t *dst, uint32_t size)
```

Parameters:

- `nds32_lfir_f32_t` instance structure of the floating-point lattice FIR filter, defined as follows:

```
typedef struct {
    uint32_t stage;
    float32_t *state;
    float32_t *coeff;
} nds32_lfir_f32_t;
```

Where

- `stage` number of filter stages
- `*state` pointer of the state vector whose size is `stage + size - 1`
- `*coeff` pointer of the coefficient vector whose size is `stage`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples

Return Value:

None

Example:

With the filter length as 6 and the input size as 8, set the instance of the lattice FIR filter as:

```
#define M      6
#define size   8
float32_t state[M + size - 1];
float32_t coeff[M] = {...};
float32_t src[size] = {...};
float32_t dst[size];
nds32_lfir_f32_t inst = {M, state, coeff};
nds32_lfir_f32 (&inst, src, dst, size);
```

This example also serves as a reference for examples of Q31 or Q15 lattice FIR filter functions.

2.4.8.2 nds32_lfir_q31

Declaration:

```
void nds32_lfir_q31(const nds32_lfir_q31_t *instance, q31_t *src, q31_t *dst,
uint32_t size)
```

Parameters:

- **nds32_lfir_q31_t** instance structure of the Q31 lattice FIR filter, defined as follows:

```
typedef struct {
    uint32_t stage;
    q31_t *state;
    q31_t *coeff;
} nds32_lfir_q31_t;
```

Where

- **stage** number of filter stages
- ***state** pointer of the state vector whose size is **stage + size - 1**
- ***coeff** pointer of the coefficient vector whose size is **stage**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples

Return Value:

None

2.4.8.3 nds32_lfir_q15

Declaration:

```
void nds32_lfir_q15(const nds32_lfir_q15_t *instance, q15_t *src, q15_t *dst,
uint32_t size)
```

Parameters:

- **nds32_lfir_q15_t** instance structure of the Q15 lattice FIR filter, defined as follows:

```
typedef struct {
    uint32_t stage;
    q15_t *state;
    q15_t *coeff;
} nds32_lfir_q15_t;
```

Where

- **stage** number of filter stages
- ***state** pointer of the state vector whose size is **stage + size - 1**
- ***coeff** pointer of the coefficient vector whose size is **stage**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples

Return Value:

None

2.4.9. Lattice Infinite Impulse Response (IIR) Filter Functions

Lattice infinite impulse response (IIR) filter functions can be represented as follows:

Official Release

```

y0[n] = x[n]
yz-1[n] = yz[n] - kz * uz-1[n-1]
uz[n] = kz * yz-1[n] + uz-1[n-1]
and w[n] = vN * uN[n] + vN-1 * uN-1[n] + ... + v0 * u0[n]

```

Where **N** is the number of states and **z** = **N**, **N-1**, ..., **1**, coefficients **k** = {**r_N**, **r_{N-1}**, ..., **r₁**}, **v_N** is the ladder coefficients and **u** is the state.

Andes DSP library supports distinct lattice IIR filter functions and instance structures for the following data types: floating-point, Q31 and Q15. Before you use lattice IIR filter functions, make sure you initialize the corresponding instance structure first. These functions and their instance structures are introduced in the subsections below.

2.4.9.1 nds32_liir_f32

Declaration:

```
void nds32_liir_f32 (const nds32_liir_f32_t *instance, float32_t *src,
float32_t *dst, uint32_t size)
```

Parameters:

- **nds32_liir_f32_t** instance structure of the floating-point lattice IIR filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    float32_t *state;
    float32_t *rcoeff;
    float32_t *lcoeff;
} nds32_liir_f32_t;
```

Where

- **nstage** number of filter stages
 - ***state** pointer of the state vector whose size is **nstage + size**
 - ***rcoeff** pointer of the time reversed reflection coefficient vector whose size is **nstage**
 - ***lcoeff** pointer of the time reversed ladder coefficient vector whose size is **nstage + 1**
- **[in] *instance** pointer of the instance structure
 - **[in] *src** pointer of the input vector
 - **[out] *dst** pointer of the output vector
 - **[in] size** number of samples

Return Value:

None

Example:

```
#define size      8
#define nstage    6
float32_t state[nstage + size];
float32_t rcoeff[nstage] = {...};
float32_t lcoeff[nstage + 1] = {...};
```

```
nds32_liir_f32_t inst = {nstage, state, rcoeff, lcoeff}.  
float32_t src[size] = {...};  
float32_t dst[size];  
nds32_liir_f32(&inst, src, dst, size);
```

This example also serves as a reference for examples of Q31, fast_Q31, Q15 or fast_Q15 lattice IIR filter functions.

2.4.9.2 nds32_liir_q31

Declaration:

```
void nds32_liir_q31(const nds32_liir_q31_t *instance, q31_t *src, q31_t *dst,
uint32_t size)
```

Parameters:

- **nds32_liir_q31_t** instance structure of the Q31 lattice IIR filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    q31_t *state;
    q31_t *rcoeff;
    q31_t *lcoeff;
} nds32_liir_q31_t;
```

Where

- **nstage** number of filter stages
 - ***state** pointer of the state vector whose size is **nstage + size**
 - ***rcoeff** pointer of the time reversed reflection coefficient vector whose size is **nstage**
 - ***lcoeff** pointer of the time reversed ladder coefficient vector whose size is **nstage + 1**
- **[in] *instance** pointer of the instance structure
 - **[in] *src** pointer of the input vector
 - **[out] *dst** pointer of the output vector
 - **[in] size** number of samples

Return Value:

None

Note:

When compared with `nds32_liir_fast_q31`, this function delivers smaller code size with lower performance and the output has the same accuracy.

2.4.9.3 nds32_liir_fast_q31

Declaration:

```
void nds32_liir_fast_q31(const nds32_liir_q31_t *instance, q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- **nds32_liir_q31_t** instance structure of the Q31 lattice IIR filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    q31_t *state;
    q31_t *rcoeff;
    q31_t *lcoeff;
} nds32_liir_q31_t;
```

Where

- **nstage** number of filter stages
 - ***state** pointer of the state vector whose size is **nstage + size**
 - ***rcoeff** pointer of the time reversed reflection coefficient vector whose size is **nstage**
 - ***lcoeff** pointer of the time reversed ladder coefficient vector whose size is **nstage + 1**
- **[in] *instance** pointer of the instance structure
 - **[in] *src** pointer of the input vector
 - **[out] *dst** pointer of the output vector
 - **[in] size** number of samples whose size must be a multiple of 2

Return Value:

None

Note:

When compared with `nds32_liir_q31`, this function delivers higher performance with larger code size and the output has the same accuracy. In order to avoid overflow, the reflection coefficient must be scaled down by 2.

2.4.9.4 nds32_liir_q15

Declaration:

```
void nds32_liir_q15(const nds32_liir_q15_t *instance, q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- **nds32_liir_q15_t** instance structure of the Q15 lattice IIR filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    q15_t *state;
    q15_t *rcoeff;
    q15_t *lcoeff;
} nds32_liir_q15_t;
```

Where

- **nstage** number of filter stages
 - ***state** pointer of the state vector whose size is **nstage + size**
 - ***rcoeff** pointer of the time reversed reflection coefficient vector whose size is **nstage**
 - ***lcoeff** pointer of the time reversed ladder coefficient vector whose size is **nstage + 1**
- **[in] *instance** pointer of the instance structure
 - **[in] *src** pointer of the input vector
 - **[out] *dst** pointer of the output vector
 - **[in] size** number of samples

Return Value:

None

Note:

When compared with `nds32_liir_fast_q15`, this function delivers smaller code size with lower performance and the output has the same accuracy.

2.4.9.5 nds32_liir_fast_q15

Declaration:

```
void nds32_liir_fast_q15(const nds32_liir_q15_t *instance, q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- **nds32_liir_q15_t** instance structure of the Q15 lattice IIR filter, defined as follows:

```
typedef struct {
    uint32_t nstage;
    q15_t *state;
    q15_t *rcoeff;
    q15_t *lcoeff;
} nds32_liir_q15_t;
```

Where

- **nstage** number of filter stages
 - ***state** pointer of the state vector whose size is **nstage + size**
 - ***rcoeff** pointer of the time reversed reflection coefficient vector whose size is **nstage**
 - ***lcoeff** pointer of the time reversed ladder coefficient vector whose size is **nstage + 1**
- **[in] *instance** pointer of the instance structure
 - **[in] *src** pointer of the input vector
 - **[out] *dst** pointer of the output vector
 - **[in] size** number of samples whose size must be a multiple of 2

Return Value:

None

Note:

When compared with **nds32_liir_q15**, this function delivers higher performance with larger code size and the output has the same accuracy.

2.4.10. Least Mean Square (LMS) Filter Functions

Least mean square (LMS) filter functions are adaptive filters which perform the following equations to update coefficients and get the minimum mean square error:

$$y[n] = h[0] * x[n] + h[1] * x[n - 1] + \dots + h[L - 1] * x[n - L + 1];$$

$$e[n] = d[n] - y[n];$$

$$h[k] = h[k - 1] + \mu * e[n] * x[n - k];$$

Where

- y is the output vector.
- h is the coefficients vector.
- x is the input vector.
- L is the filter order.
- e is the error vector.
- d is the desired vector.
- μ is the step size which can adjust coefficients.

Andes DSP library supports distinct least mean square filter functions and instance structures for the following data types: floating-point, Q31 and Q15. Before you use least mean square filter functions, make sure you initialize the corresponding instance structure first. These functions and their instance structures are introduced in the subsections below.

2.4.10.1 nds32_lms_f32

Declaration:

```
void nds32_lms_f32 (const nds32_lms_f32_t *instance, float32_t *src,
float32_t *ref, float32_t *dst, float32_t *err, uint32_t size)
```

Parameters:

- **nds32_lms_f32_t** instance structure of the floating-point LMS filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    float32_t *state;
    float32_t *coeff;
    float32_t mu;
} nds32_lms_f32_t;
```

Where

- **coeff_size** number of filter coefficients
- ***state** pointer of the state vector whose size is **coeff_size + size + 1**
- ***coeff** pointer of the time reversed coefficient vector whose size is **coeff_size**
- **mu** step size which can adjust coefficients
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[in] *ref** pointer of the desired vector
- **[out] *dst** pointer of the output vector
- **[out] *err** pointer of the error vector
- **[in] size** number of samples

Return Value:

None

2.4.10.2 nds32_lms_q31

Declaration:

```
void nds32_lms_q31(const nds32_lms_q31_t *instance, q31_t *src, q31_t *ref,
q31_t *dst, q31_t *err, uint32_t size)
```

Parameters:

- `nds32_lms_q31_t` instance structure of the Q31 LMS filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q31_t *state;
    q31_t *coeff;
    q31_t mu;
    q31_t shift;
} nds32_lms_q31_t;
```

Where

- `coeff_size` number of filter coefficients
- `*state` pointer of the state vector whose size is `coeff_size + size + 1`
- `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `mu` step size which can adjust coefficients
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[in] *ref` pointer of the desired vector
- `[out] *dst` pointer of the output vector
- `[out] *err` pointer of the error vector
- `[in] size` number of samples

Return Value:

None

2.4.10.3 nds32_lms_q15

Declaration:

```
void nds32_lms_q15 (const nds32_lms_q15_t *instance, q15_t *src, q15_t *ref,
q15_t *dst, q15_t *err, uint32_t size)
```

Parameters:

- `nds32_lms_q15_t` instance structure of the Q15 LMS filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q15_t *state;
    q15_t *coeff;
    q15_t mu;
    q15_t shift;
} nds32_lms_q15_t;
```

Where

- `coeff_size` number of filter coefficients
- `*state` pointer of the state vector whose size is `coeff_size + size + 1`
- `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
- `mu` step size which can adjust coefficients
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[in] *ref` pointer of the desired vector
- `[out] *dst` pointer of the output vector
- `[out] *err` pointer of the error vector
- `[in] size` number of samples

Return Value:

None

2.4.11. Normalized Least Mean Square (NLMS) Filter Function

The Normalized Least Mean Square (NLMS) filter function is an extension of LMS filter functions. It can be represented by the following equations:

```
y[n] = h[0] * x[n] + h[1] * x[n - 1] + ... + h[L - 1] * x[n - L + 1];
e[n] = d[n] - y[n];
μ[n] = 1 / (x[n]^2);
h[n] = h[n - 1] + μ[n] * e[n] * x[n];
```

Where

- y is the output vector.
- h is the coefficients vector.
- x is the input vector.
- L is the filter order.
- e is the error vector.
- d is the desired vector.
- μ is the step size which can adjust coefficients and is related to the normalized input signals.

Andes DSP library supports distinct NLMS filter function and instance structure for the following data types: floating point, Q31 and Q15. Before you use the NLMS filter function, make sure you initialize the corresponding instance structure first. These functions and their instance structures are introduced in the subsections below.

2.4.11.1 nds32_nlms_f32

Declaration:

```
void nds32_nlms_f32 (nds32_nlms_f32_t *instance, float32_t *src, float32_t *ref, float32_t *dst, float32_t *err, uint32_t size)
```

Parameters:

- **nds32_nlms_f32_t** instance structure of the floating-point NLMS filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    float32_t *state;
    float32_t *coeff;
    float32_t mu;
    float32_t energy
} nds32_nlms_f32_t;
```

Where

- **coeff_size** number of filter coefficients
- ***state** pointer of the state vector whose size is **coeff_size + size + 1**
- ***coeff** pointer of the time reversed coefficient vector whose size is **coeff_size**
- **mu** step size which can adjust coefficients
- **energy** energy of the previous frame
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[in] *ref** pointer of the desired vector
- **[out] *dst** pointer of the output vector
- **[out] *err** pointer of the error vector
- **[in] size** number of samples

Return Value:

None

2.4.11.2 nds32_nlms_q31

Declaration:

```
void nds32_nlms_q31(nds32_nlms_q31_t *instance, q31_t *src, q31_t *ref, q31_t *dst, q31_t *err, uint32_t size);
```

Parameters:

- **nds32_nlms_q31_t** instance structure of the Q31 NLMS filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q31_t *state;
    q31_t *coeff;
    q31_t mu;
    uint8_t postshift;
    q31_t energy;
    q31_t x0;
} nds32_nlms_q31_t;
```

Where

- **coeff_size** number of filter coefficients
- ***state** pointer of the state vector whose size is **coeff_size + size + 1**
- ***coeff** pointer of the time reversed coefficient vector whose size is **coeff_size**
- **mu** step size which can adjust coefficients
- **postshift** number of coefficient bits to be shifted
- **energy** energy of the previous frame
- **x0** previous input sample
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[in] *ref** pointer of the desired vector
- **[out] *dst** pointer of the output vector
- **[out] *err** pointer of the error vector
- **[in] size** number of samples

Return Value:

None

2.4.11.3 nds32_nlms_q15

Declaration:

```
void nds32_nlms_q15(nds32_nlms_q15_t *instance, q15_t *src, q15_t *ref, q15_t *dst, q15_t *err, uint32_t size);
```

Parameters:

- `nds32_nlms_q15_t` instance structure of the Q15 NLMS filter, defined as follows:

```
typedef struct {
    uint32_t coeff_size;
    q15_t *state;
    q15_t *coeff;
    q15_t mu;
    uint8_t postshift;
    q15_t energy;
    q15_t x0;
} nds32_nlms_q15_t;
```

Where

- `coeff_size` number of filter coefficients
 - `*state` pointer of the state vector whose size is `coeff_size + size + 1`
 - `*coeff` pointer of the time reversed coefficient vector whose size is `coeff_size`
 - `mu` step size which can adjust coefficients
 - `postshift` number of coefficient bits to be shifted
 - `energy` energy of the previous frame
 - `x0` previous input sample
-
- `[in] *instance` pointer of the instance structure
 - `[in] *src` pointer of the input vector
 - `[in] *ref` pointer of the desired vector
 - `[out] *dst` pointer of the output vector
 - `[out] *err` pointer of the error vector
 - `[in] size` number of samples

Return Value:

None

2.4.12. Sparse FIR Filter Functions

Sparse FIR filter functions are FIR filters with the majority of coefficients being zero. Due to this characteristic, sparse FIR filter only keeps the non-zero coefficients and ignores the multiplication operation of signal and zero coefficients.

Andes DSP library supports distinct sparse FIR filter functions and instance structures for the following data types: floating-point, Q31, Q15 and Q7. Before you use sparse FIR filter functions, make sure you initialize the corresponding instance structures first. These functions and their instance structures are introduced in the subsections below.

2.4.12.1 nds32_spafir_f32

Declaration:

```
void nds32_spafir_f32 (nds32_spafir_f32_t *instance, float32_t *src,
float32_t *dst, float32_t *buf, uint32_t size)
```

Parameters:

- **nds32_spafir_f32_t** instance structure of the floating-point sparse FIR filter, defined as follows:

```
typedef struct {
    uint16_t coeff_size;
    uint16_t index;
    float32_t *state;
    float32_t *coeff;
    uint16_t delay;
    int32_t *nezdelay;
} nds32_spafir_f32_t;
```

Where

- **coeff_size** number of filter coefficients
- **index** index of the state buffer
- ***state** pointer of the state vector whose size is **delay + size**
- ***coeff** pointer of the coefficient vector whose size is **coeff_size**
- **delay** maximum value in the nezdelay vector
- ***nezdelay** vector which stores non-zero coefficients and the size is **coeff_size**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] *buf** pointer of the temporary vector
- **[in] size** number of samples

Return Value:

None

Note:

The size of temporary buffer should be equal to **size**, size of the input vector.

2.4.12.2 nds32_spafir_q31

Declaration:

```
void nds32_spafir_q31(nds32_spafir_q31_t *instance, q31_t *src, q31_t *dst,
q31_t *buf, uint32_t size)
```

Parameters:

- **nds32_spafir_q31_t** instance structure of the Q31 sparse FIR filter, defined as follows:

```
typedef struct {
    uint16_t coeff_size;
    uint16_t index;
    q31_t *state;
    q31_t *coeff;
    uint16_t delay;
    int32_t *nezdelay;
} nds32_spafir_q31_t;
```

Where

- **coeff_size** number of filter coefficients
- **index** index of the state buffer
- ***state** pointer of the state vector whose size is **delay + size**
- ***coeff** pointer of the coefficient vector whose size is **coeff_size**
- **delay** maximum value in the nezdelay vector
- ***nezdelay** vector which stores non-zero coefficients and the size is **coeff_size**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] *buf** pointer of the temporary vector
- **[in] size** number of samples

Return Value:

None

Note:

The size of temporary buffer should be equal to **size**, size of the input vector.

2.4.12.3 nds32_spafir_q15

Declaration:

```
void nds32_spafir_q15(nds32_spafir_q15_t *instance, q15_t *src, q15_t *dst,
q15_t *buf1, q31_t *buf2, uint32_t size)
```

Parameters:

- `nds32_spafir_q15_t` instance structure of the Q15 sparse FIR filter, defined as follows:

```
typedef struct {
    uint16_t coeff_size;
    uint16_t index;
    q15_t *state;
    q15_t *coeff;
    uint16_t delay;
    int32_t *nezdelay;
} nds32_spafir_q15_t;
```

Where

- `coeff_size` number of filter coefficients
- `index` index of the state buffer
- `*state` pointer of the state vector whose size is `delay + size`
- `*coeff` pointer of the coefficient vector whose size is `coeff_size`
- `delay` maximum value in the `nezdelay` vector
- `*nezdelay` vector which stores non-zero coefficients and the size is `coeff_size`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] *buf1` pointer of the temporary buffer 1
- `[in] *buf2` pointer of the temporary buffer 2
- `[in] size` number of samples

Return Value:

None

Note:

The temporary buffer 1 and 2 should both have a size of `size` (size of the input vector).

2.4.12.4 nds32_spafir_q7

Declaration:

```
void nds32_spafir_q7(nds32_spafir_q7_t *instance, q7_t *src, q7_t *dst, q7_t
*buf1, q31_t *buf2, uint32_t size)
```

Parameters:

- `nds32_spafir_q7_t` instance structure of the Q7 sparse FIR filter, defined as follows:

```
typedef struct {
    uint16_t coeff_size;
    uint16_t index;
    q7_t *state;
    q7_t *coeff;
    uint16_t delay;
    int32_t *nezdelay;
} nds32_spafir_q7_t;
```

Where

- `coeff_size` number of filter coefficients
- `index` index of the state buffer
- `*state` pointer of the state vector whose size is `delay + size`
- `*coeff` pointer of the coefficient vector whose size is `coeff_size`
- `delay` maximum value in the `nezdelay` vector
- `*nezdelay` vector which stores non-zero coefficients and the size is `coeff_size`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] *buf1` pointer of the temporary buffer 1
- `[in] *buf2` pointer of the temporary buffer 2
- `[in] size` number of samples

Return Value:

None

Note:

The temporary buffer 1 and 2 should both have a size of `size` (size of the input vector).

2.4.13. Upsampling FIR Filter Functions

Upsampling FIR filter functions upsample incoming signals to generate multirate outputs. They are usually used in audio/sound or images processing like increasing the resolutions of images or adjusting the audio output sample rate from 16kbps to 32kbps.

Andes DSP library supports distinct upsampling FIR filter functions and instance structures for the following data types: floating-point, Q31 and Q15. Before you use upsampling filter functions, make sure you initialize the corresponding instance structure first. These functions and their instance structures are introduced in the subsections below.

2.4.13.1 nds32_upsplfir_f32

Declaration:

```
void nds32_upsplfir_f32 (const nds32_upsplfir_f32_t *instance, float32_t
*src, float32_t *dst, uint32_t size)
```

Parameters:

- **nds32_upsplfir_f32_t** instance structure of the floating-point upsampling FIR filter, defined as follows:

```
typedef struct {
    uint32_t L;
    uint32_t plen;
    float32_t *coeff;
    float32_t *state;
} nds32_upsplfir_f32_t;
```

Where

- **L** factor of the upsampling filter
- **plen** number of coefficients
- ***coeff** pointer of the time reversed coefficient vector whose size is **L * plen**
- ***state** pointer of the state vector whose size is **plen + size - 1**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples

Return Value:

None

2.4.13.2 nds32_upsplfir_q31

Declaration:

```
void nds32_upsplfir_q31 (const nds32_upsplfir_q31_t *instance, q31_t *src,
q31_t *dst, uint32_t size)
```

Parameters:

- `nds32_upsplfir_q31_t` instance structure of the Q31 upsampling FIR filter, defined as follows:

```
typedef struct {
    uint32_t L;
    uint32_t plen;
    q31_t *coeff;
    q31_t *state;
} nds32_upsplfir_q31_t;
```

Where

- `L` factor of the upsampling filter
- `plen` number of coefficients
- `*coeff` pointer of the time reversed coefficient vector whose size is `L * plen`
- `*state` pointer of the state vector whose size is `plen + size - 1`
- `[in] *instance` pointer of the instance structure
- `[in] *src` pointer of the input vector
- `[out] *dst` pointer of the output vector
- `[in] size` number of samples

Return Value:

None

Note:

Both coefficients and state variables are represented in 1.31 format and multiplications yield a 2.62 result. The 2.62 results are accumulated in a 64-bit accumulator. After all operations are performed, the 2.62 accumulator is truncated to 1.32 format and then saturated to 1.31 format.

2.4.13.3 nds32_upsplfir_q15

Declaration:

```
void nds32_upsplfir_q15 (const nds32_upsplfir_q15_t *instance, q15_t *src,
q15_t *dst, uint32_t size)
```

Parameters:

- **nds32_upsplfir_q15_t** instance structure of the Q15 upsampling FIR filter, defined as follows:

```
typedef struct {
    uint32_t L;
    uint32_t plen;
    q15_t *coeff;
    q15_t *state;
} nds32_upsplfir_q15_t;
```

Where

- **L** factor of the upsampling filter
- **plen** number of coefficients
- ***coeff** pointer of the time reversed coefficient vector whose size is **L * plen**
- ***state** pointer of the state vector whose size is **plen + size - 1**
- **[in] *instance** pointer of the instance structure
- **[in] *src** pointer of the input vector
- **[out] *dst** pointer of the output vector
- **[in] size** number of samples

Return Value:

None

Note:

Both coefficients and state variables are represented in 1.15 format and multiplications yield a 2.30 result. The 2.30 results are accumulated in a 64-bit accumulator in 34.30 format and the results is truncated to 34.15 format by discarding low 15 bits. Lastly, the outputs are saturated to yield a result in 1.15 format.

2.5. Matrix Functions

A matrix can be seen as a rectangular array with m rows and n columns and of $m * n$ elements. In Andes DSP library, for programming convenience, matrices are stored into vectors separately in row-major order. That is, the size of a vector equals to the number of a matrix (size = $m * n$).

2.5.1. Matrix Addition Functions

Matrix addition functions add two elements from two source vectors and write the results one-by-one into a destination vector. The behavior can be defined as follows:

```
size = row * col;  
for (i = 0; i < size; i++)  
    dst[i] = src1[i] + src2[i];
```

Andes DSP library supports distinct matrix addition functions for the following data types: floating-point, Q31, and Q15. These functions are introduced in the subsections below.

2.5.1.1 nds32_mat_add_f32

Declaration:

```
void nds32_mat_add_f32 (const float32_t *src1, const float32_t *src2,
float32_t *dst, uint32_t row, uint32_t col)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

Example:

The following equation shows the addition of two matrices and the result.

$$\begin{bmatrix} 0.1 & 0.4 \\ -0.2 & 0.1 \end{bmatrix} + \begin{bmatrix} -0.2 & -0.1 \\ 0.3 & 0.5 \end{bmatrix} = \begin{bmatrix} -0.1 & 0.3 \\ 0.1 & 0.6 \end{bmatrix}$$

Its code example is as follows:

```
#define row    2
#define col    2
float32_t src1[row * col] = {0.1, 0.4, -0.2, 0.1};
float32_t src2[row * col] = {-0.2, -0.1, 0.3, 0.5};
float32_t dst[row * col];
nds32_mat_add_f32(src1, src2, dst, row, col);
```

This example also serves as a reference for examples of Q31 or Q15 matrix addition functions.

2.5.1.2 nds32_mat_add_q31

Declaration:

```
void nds32_mat_add_q31 (const q31_t *src1, const q31_t *src2, q31_t *dst,  
uint32_t row, uint32_t col)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

Note:

The results will be saturated to the Q31 range [0x80000000, 0x7FFFFFFF].

2.5.1.3 nds32_mat_add_q15

Declaration:

```
void nds32_mat_add_q15 (const q15_t *src1, const q15_t *src2, q15_t *dst,  
uint32_t row, uint32_t col)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

Note:

The results will be saturated to the Q15 range [0x8000, 0x7FFF].

2.5.2. Matrix Inverse Function

The matrix inverse function returns the inverse of a square matrix. The square matrix has the same dimension in the horizontal and vertical direction. Therefore, the matrix inverse function only accepts one parameter (i.e. size) to specify the number of row and that of column. It not only avoids receiving two different numbers for row and column but also slightly improves performance.

The matrix inverse function implements Gauss-Jordan elimination to find the inverse of a matrix. First, the output matrix is filled with 0 and 1 to form an identity matrix. Next, elementary row operations are performed on the input and output matrices simultaneously to make the input matrix an identity matrix. If the original input matrix is invertible, the output matrix will eventually be its inverse and 0 will be returned. The behavior can be defined as follows:

$$[A \mid I] \rightarrow \text{elementary row operations} \rightarrow [I \mid A^{-1}]$$

Where A is the input matrix and A^{-1} is its inverse matrix. I is an identity matrix generated in the output matrix before elementary row operations and found in the input matrix after the operations.

If the pivot value is zero during elementary row operations, the function needs to look for a non-zero pivot from following rows. Once it is found, swap the two rows and perform elementary row operations again until the input matrix is transformed into an identity matrix. If a non-zero pivot is not available, return -1, indicating the input matrix is singular.

Andes DSP library supports distinct matrix inverse functions for the following data types: single- and double-precision floating-point .

2.5.2.1 nds32_mat_inv_f32

Declaration:

```
int32_t nds32_mat_inv_f32 (float32_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input matrix
- [out] *dst pointer of the output matrix
- [in] size number of rows/columns in a matrix (i.e. size = rows = columns)

Return Value:

- 0 success
- 1 failure

Note:

1. If the return value is 0, the input matrix is invertible and transformed into an identity matrix; if the return value is -1, the input matrix is singular.
2. No matter what value is returned, the content of the input and of the output matrix will both be changed after execution of the matrix inverse function.

Example:

A square matrix (A) and its inverse are listed below:

$$A = \begin{bmatrix} 0.1 & 0.4 \\ -0.2 & -0.3 \end{bmatrix} \quad \text{inv}(A) = \begin{bmatrix} -6 & -8 \\ 4 & 2 \end{bmatrix}$$

The code example for getting the inverse of matrix A is as follows:

```
#define size      2
float32_t src[size * size] = {0.1, 0.4, -0.2, -0.3};
float32_t dst[size * size];
if (nds32_mat_inv_f32(src, dst, size) == 0)
    Success...
else
    Fail...
```

This example also serves as a reference for examples of the F64 matrix inverse function.

2.5.2.2 nds32_mat_inv_f64

Declaration:

```
int32_t nds32_mat_inv_f64 (float64_t *src, float64_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input matrix
- [out] *dst pointer of the output matrix
- [in] size number of rows/columns in a matrix (i.e. size = rows = columns)

Return Value:

- 0 success
- 1 failure

2.5.3. Matrix Multiplication Functions

Matrix multiplication functions compute the multiplication of two source matrices and write the result into a destination matrix. Figure 3 shows the multiplication of matrix A by matrix B, in which both matrices have 2 rows and 2 columns.

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11} \times B_{11} + A_{12} \times B_{21} & A_{11} \times B_{12} + A_{12} \times B_{22} \\ A_{21} \times B_{11} + A_{22} \times B_{21} & A_{21} \times B_{12} + A_{22} \times B_{22} \end{bmatrix}$$

Figure 3. Matrix A Multiply Matrix B

According to the definition of matrix multiplication, the number of rows in matrix B should be identical with the number of columns in matrix A. That is,

$$C_{ik} = A_{ij} * B_{jk}$$

where A is a matrix with i rows and j columns, B is a matrix with j rows and k columns, and C is a matrix with i rows and k columns.

Andes DSP library supports distinct matrix multiplication functions for the following data types: single- and double-precision floating-point, Q31, and Q15. These functions are introduced in the subsections below.

2.5.3.1 nds32_mat_mul_f32

Declaration:

```
void nds32_mat_mul_f32 (const float32_t *src1, const float32_t *src2,
float32_t *dst, uint32_t row, uint32_t col, uint32_t col2)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in the first matrix
- [in] col2 number of columns in the second matrix

Return Value:

None

Example:

The following equation shows the multiplication of two matrices:

$$A_{2,3} \times B_{3,2} = C_{2,2}$$

Given each element with a value, the result is like

$$\begin{bmatrix} 0.1 & -0.1 & 0.1 \\ 0.2 & -0.2 & 0.3 \end{bmatrix} \times \begin{bmatrix} 0.2 & 0.2 \\ -0.1 & 0.3 \\ -0.7 & -0.2 \end{bmatrix} = \begin{bmatrix} -0.04 & -0.03 \\ -0.15 & -0.08 \end{bmatrix}$$

and can be presented in code as follows:

```
#define Arow      2
#define Acol      3
#define Bcol      2
float32_t src1[Arow * Acol] = {0.1, -0.1, 0.1, 0.2, -0.2, 0.3};
float32_t src2[Acol * Bcol] = {0.2, 0.2, -0.1, 0.3, -0.7, -0.2};
float32_t dst[Arow * Bcol];
nds32_mat_mul_f32 (src1, src2, dst, Arow, Acol, Bcol);
```

This example also serves as a reference for examples of F64, Q31 or Q15 matrix multiplication functions.

2.5.3.2 nds32_mat_mul_f64

Declaration:

```
void nds32_mat_mul_f64 (const float64_t *src1, const float64_t *src2,  
float64_t *dst, uint32_t row, uint32_t col, uint32_t col2)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in the first matrix
- [in] col2 number of columns in the second matrix

Return Value:

None

2.5.3.3 nds32_mat_mul_q31

Declaration:

```
void nds32_mat_mul_q31 (const q31_t *src1, const q31_t *src2, q31_t *dst,  
uint32_t row, uint32_t col, uint32_t col2)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in the first matrix
- [in] col2 number of columns in the second matrix

Return Value:

None

2.5.3.4 nds32_mat_mul_q15

Declaration:

```
void nds32_mat_mul_q15(const q15_t *src1, const q15_t *src2, q15_t *dst,  
uint32_t row, uint32_t col, uint32_t col2)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in the first matrix
- [in] col2 number of columns in the second matrix

Return Value:

None

2.5.4. Matrix Scale Functions

Matrix scale functions multiply a matrix by a constant scaling value and write the result into a destination matrix. Since vectors are used as containers of matrices in Andes DSP library, the behavior of matrix scale functions can be defined as follows:

```
size = m * n;  
for (n = 0; n < size; n++)  
    dst[i] = src[i] * scale;
```

In cases of fractional presentation, a parameter **shift** is introduced to adjust the range of result values. For details about the parameter, please refer to functions with fractional data types (Section 2.5.4.2 and 2.5.4.3).

Andes DSP library supports distinct matrix scale functions for the following data types: floating-point, Q31, and Q15. These functions are introduced in the subsections below.

2.5.4.1 nds32_mat_scale_f32

Declaration:

```
void nds32_mat_scale_f32 (const float32_t *src, float32_t scale, float32_t
*dst, uint32_t row, uint32_t col)
```

Parameters:

- [in] *src pointer of the input matrix
- [in] scale constant scaling value
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

Example:

Given a matrix and a scale value 0.2, the matrix scale equation is as follows:

$$\begin{bmatrix} 0.1 & 0.4 \\ -0.2 & 0.1 \end{bmatrix} \times scale = \begin{bmatrix} 0.02 & 0.08 \\ -0.04 & 0.02 \end{bmatrix}$$

and its code example is like:

```
#define row    2
#define col    2
float32_t src[row * col] = {0.1, 0.4, -0.2, 0.1};
float32_t scale = 0.2;
float32_t dst[row * col];
nds32_mat_scale_f32 (src1, scale, dst, row, col);
```

This example also serves as a reference for examples of Q31 or Q15 matrix scale functions.

2.5.4.2 nds32_mat_scale_q31

Declaration:

```
void nds32_mat_scale_q31(const q31_t *src, q31_t scale_fract, int32_t shift,
q31_t *dst, uint32_t row, uint32_t col)
```

Parameters:

- [in] *src pointer of the input matrix
- [in] scale_fract constant fractional scaling value
- [in] shift shift bits
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

2.5.4.3 nds32_mat_scale_q15

Declaration:

```
void nds32_mat_scale_q15 (const q15_t *src, q15_t scale_fract, int32_t shift,  
q15_t *dst, uint32_t row, uint32_t col)
```

Parameters:

- [in] *src pointer of the input matrix
- [in] scale_fract constant fractional scaling value
- [in] shift shift bits
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

2.5.5. Matrix Subtraction Functions

Matrix subtraction functions subtract two matrices which have same dimensions and write the result into a destination matrix. The behavior can be defined as follows:

```
size = m * n;  
for (i = 0; i < size; i++)  
    dst[i] = src1[i] - src2[i];
```

Andes DSP library supports distinct matrix subtraction functions for the following data types: floating-point, Q31, and Q15. These functions are introduced in the subsections below.

2.5.5.1 nds32_mat_sub_f32

Declaration:

```
void nds32_mat_sub_f32 (const float32_t *src1, const float32_t *src2,
float32_t *dst, uint32_t row, uint32_t col)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

Example:

Given two matrices and their subtraction like below,

$$\begin{bmatrix} 0.1 & 0.4 \\ -0.2 & 0.1 \end{bmatrix} - \begin{bmatrix} -0.2 & -0.1 \\ 0.3 & 0.5 \end{bmatrix} = \begin{bmatrix} 0.3 & 0.5 \\ -0.5 & -0.4 \end{bmatrix}$$

the code example of matrix subtraction is as follows:

```
#define row      2
#define col      2
float32_t src1[row * col] = {0.1, 0.4, -0.2, 0.1};
float32_t src2[row * col] = {-0.2, -0.1, 0.3, 0.5};
float32_t dst[row * col];
nds32_mat_sub_f32 (src1, src2, dst, row, col);
```

This example also serves as a reference for examples of Q31 or Q15 matrix subtraction functions.

2.5.5.2 nds32_mat_sub_q31

Declaration:

```
void nds32_mat_sub_q31 (const q31_t *src1, const q31_t *src2, q31_t *dst,  
uint32_t row, uint32_t col)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

Note:

The results will be saturated to the Q31 range [0x80000000, 0x7FFFFFFF].

2.5.5.3 nds32_mat_sub_q15

Declaration:

```
void nds32_mat_sub_q15(const q15_t *src1, const q15_t *src2, q15_t *dst,  
uint32_t row, uint32_t col)
```

Parameters:

- [in] *src1 pointer of the first input matrix
- [in] *src2 pointer of the second input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

Note:

The results will be saturated to the Q15 range [0x8000, 0x7FFF].

2.5.6. Matrix Transpose Functions

Matrix transpose functions transpose a matrix then write the result into a destination matrix. The behavior can be defined as follows:

$\text{dst}[n, m] = \text{src}[m, n]$, where $0 \leq m < \text{row}$, $0 \leq n < \text{col}$

Andes DSP library supports distinct matrix transpose functions for floating-point, Q31, Q15 and other data types. These functions are introduced in the subsections below.

2.5.6.1 nds32_mat_trans_f32

Declaration:

```
void nds32_mat_trans_f32 (const float32_t *src, float32_t *dst, uint32_t row,
uint32_t col)
```

Parameters:

- [in] *src pointer of the input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

Example:

Given an equation of matrix transposing like below,

$$\begin{bmatrix} 0.1 & -0.1 & 0.1 \\ 0.2 & -0.2 & 0.3 \end{bmatrix}^T = \begin{bmatrix} 0.1 & 0.2 \\ -0.1 & -0.2 \\ 0.1 & 0.3 \end{bmatrix}$$

its code example is as follows:

```
#define row    2
#define col    3
float32_t src[row * col] = {0.1, -0.1, 0.1, 0.2, -0.2, 0.3};
float32_t dst[col * row];
nds32_mat_trans_f32 (src, dst, row, col);
```

This example also serves as a reference for examples of Q31 or Q15 matrix transpose functions.

2.5.6.2 nds32_mat_trans_q31

Declaration:

```
void nds32_mat_trans_q31(const q31_t *src, q31_t *dst, uint32_t row, uint32_t col)
```

Parameters:

- [in] *src pointer of the input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

2.5.6.3 nds32_mat_trans_q15

Declaration:

```
void nds32_mat_trans_q15 (const q15_t *src, q15_t *dst, uint32_t row, uint32_t col)
```

Parameters:

- [in] *src pointer of the input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

2.5.6.4 nds32_mat_trans_u8

Declaration:

```
void nds32_mat_trans_u8 (const uint8_t *src, uint8_t *dst, uint32_t row,  
uint32_t col)
```

Parameters:

- [in] *src pointer of the input matrix
- [out] *dst pointer of the output matrix
- [in] row number of rows in a matrix
- [in] col number of columns in a matrix

Return Value:

None

2.5.7. Matrix Power 2 Function

The matrix power 2 function powers a square matrix with 2 and then writes the result to a destination matrix. The behavior can be defined as follows:

`dst = src^2`, where `src` is a square matrix

Andes DSP library only supports the matrix power 2 function for double precision floating-point data.

2.5.7.1 nds32_mat_pwr2_cache_f64

Declaration:

```
int32_t nds32_mat_pwr2_cache_f64(const float64_t *src, float64_t *dst,
uint32_t size)
```

Parameters:

- [in] *src pointer of the input matrix
- [out] *dst pointer of the output matrix
- [in] size number of rows or columns in a matrix

Return Value:

- 0 success
- 1 failure

Note:

The input matrix must be a square matrix with the number of rows or columns equal to `size`. In addition, `size` must be a multiple of four (such as 28, 32, 64, 1024, etc). Otherwise, a return value of `-1` is given for the error.

This function performs a block matrix multiplication. The block size is the size of one cache line for utilizing each cache line fetched from the memory and reducing cache misses.

When `size` is smaller than 40, you are advised to use `nds32_mat_mul_f64` for better performance.

2.6. Statistics Functions

2.6.1. Maximum Functions

Maximum functions compare the values in a vector then return the maximum one and its index.

Andes DSP library supports distinct maximum functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.

2.6.1.1 nds32_max_f32

Declaration:

`float32_t nds32_max_f32 (const float32_t *src, uint32_t size, uint32_t *index)`

Parameters:

- `[in] *src` pointer of the input vector
- `[in] size` number of elements in a vector
- `[out] *index` index of the maximum value

Return Value:

Maximum value

2.6.1.2 nds32_max_q31

Declaration:

`q31_t nds32_max_q31 (const q31_t *src, uint32_t size, uint32_t *index)`

Parameters:

- `[in] *src` pointer of the input vector
- `[in] size` number of elements in a vector
- `[out] *index` index of the maximum value

Return Value:

Maximum value

2.6.1.3 nds32_max_q15

Declaration:

```
q15_t nds32_max_q15 (const q15_t *src, uint32_t size, uint32_t *index)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector
- [out] *index index of the maximum value

Return Value:

Maximum value

2.6.1.4 nds32_max_q7

Declaration:

`q7_t nds32_max_q7 (const q7_t *src, uint32_t size, uint32_t *index)`

Parameters:

- `[in] *src` pointer of the input vector
- `[in] size` number of elements in a vector
- `[out] *index` index of the maximum value

Return Value:

Maximum value

2.6.1.5 nds32_max_u8

Declaration:

```
uint8_t nds32_max_u8 (const uint8_t *src, uint32_t size, uint32_t *index)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector
- [out] *index index of the maximum value

Return Value:

Maximum value

2.6.2. Mean Functions

Mean functions calculate the arithmetic mean of a vector.

Andes DSP library supports distinct mean functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.



2.6.2.1 nds32_mean_f32

Declaration:

```
float32_t nds32_mean_f32 (const float32_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Mean value

2.6.2.2 nds32_mean_q31

Declaration:

```
q31_t nds32_mean_q31 (const q31_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Mean value

2.6.2.3 nds32_mean_q15

Declaration:

```
q15_t nds32_mean_q15 (const q15_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Mean value

2.6.2.4 nds32_mean_q7

Declaration:

```
q7_t nds32_mean_q7 (const q7_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Mean value

2.6.2.5 nds32_mean_u8

Declaration:

```
uint8_t nds32_mean_u8 (const uint8_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Mean value

2.6.3. Minimum Functions

Minimum functions compare the values in a vector then return the minimum one and its index.

Andes DSP library supports distinct minimum functions for floating-point, Q31, Q15, Q7 and other data types. These functions are introduced in the subsections below.



2.6.3.1 nds32_min_f32

Declaration:

`float32_t nds32_min_f32 (const float32_t *src, uint32_t size, uint32_t *index)`

Parameters:

- `[in] *src` pointer of the input vector
- `[in] size` number of elements in a vector
- `[out] *index` index of the minimum value

Return Value:

Minimum value

2.6.3.2 nds32_min_q31

Declaration:

```
q31_t nds32_min_q31 (const q31_t *src, uint32_t size, uint32_t *index)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector
- [out] *index index of the minimum value

Return Value:

Minimum value

2.6.3.3 nds32_min_q15

Declaration:

```
q15_t nds32_min_q15 (const q15_t *src, uint32_t size, uint32_t *index)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector
- [out] *index index of the minimum value

Return Value:

Minimum value

2.6.3.4 nds32_min_q7

Declaration:

```
q7_t nds32_min_q7 (const q7_t *src, uint32_t size, uint32_t *index)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector
- [out] *index index of the minimum value

Return Value:

Minimum value

2.6.3.5 nds32_min_u8

Declaration:

```
uint8_t nds32_min_u8 (const uint8_t *src, uint32_t size, uint32_t *index)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector
- [out] *index index of the minimum value

Return Value:

Minimum value

2.6.4. Root Mean Square (RMS) Functions

Root mean square (RMS) functions compute root mean square of a vector. The behavior can be defined as follows:

$$\text{RMS} = \sqrt{(\text{src}[0]^2 + \text{src}[1]^2 + \text{src}[2]^2 + \dots + \text{src}[\text{size}-1]^2) / \text{size}}$$

For `sqrt()`, please refer to square root functions in Section 2.8.7.

Andes DSP library supports distinct RMS functions for the following data types: floating-point, Q31, and Q15. These functions are introduced in the subsections below.

2.6.4.1 nds32_rms_f32

Declaration:

```
float32_t nds32_rms_f32 (const float32_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

RMS value

2.6.4.2 nds32_rms_q31

Declaration:

```
q31_t nds32_rms_q31 (const q31_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

RMS value

2.6.4.3 nds32_rms_q15

Declaration:

```
q15_t nds32_rms_q15 (const q15_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

RMS value

2.6.5. Power Functions

Power functions compute the sum of squares of a vector. The behavior can be defined as follows:

$$SOS = src[0]^2 + src[1]^2 + src[2]^2 + \dots + src[size-1]^2$$

Andes DSP library supports distinct sum of squares functions for the following data types: floating-point, Q31, Q15 and Q7. These functions are introduced in the subsections below.

2.6.5.1 nds32_pwr_f32

Declaration:

```
float32_t nds32_pwr_f32 (const float32_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Sum of squares

2.6.5.2 nds32_pwr_q31

Declaration:

```
q63_t nds32_pwr_q31 (const q31_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Sum of squares

Note:

The return value is in Q48 format.

2.6.5.3 nds32_pwr_q15

Declaration:

```
q63_t nds32_pwr_q15 (const q15_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Sum of squares

Note:

The return value is in Q30 format.

2.6.5.4 nds32_pwr_q7

Declaration:

```
q31_t nds32_pwr_q7 (const q7_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Sum of squares

Note:

The return value is in Q14 format.

2.6.6. Standard Deviation Functions

Standard deviation functions compute the standard deviation value of a vector. The behavior can be defined as follows:

A large, light blue, rounded rectangular watermark with the words "Official" and "Release" stacked vertically in a serif font is centered over the code block.

```
sos = src[0]2 + src[1]2 + ... + src[size-1]2;  
sqrsum = (src[0] + src[1] + ... + src[size-1])2;  
std = sqrt((sos - sqrsum / size) / (size - 1));
```

For `sqrt()`, please refer to square root functions in Section 2.8.7.

Andes DSP library supports distinct standard deviation functions for floating-point, Q31, Q15 and other data types. These functions are introduced in the subsections below.

2.6.6.1 nds32_std_f32

Declaration:

```
float32_t nds32_std_f32 (const float32_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Standard deviation value

2.6.6.2 nds32_std_q31

Declaration:

```
q31_t nds32_std_q31 (const q31_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Standard deviation value

2.6.6.3 nds32_std_q15

Declaration:

```
q15_t nds32_std_q15 (const q15_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Standard deviation value

2.6.6.4 nds32_std_u8

Declaration:

```
q15_t nds32_std_u8 (const uint8_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Standard deviation value

2.6.7. Variance Functions

Variance functions compute the variance value of a vector. The behavior can be defined as follows:

Official
Release

```
sos = src[0]2 + src[1]2 + ... + src[size-1]2;  
sqrsum = (src[0] + src[1] + ... + src[size-1])2;  
v = (sos - sqrsum / size) / (size - 1);
```

Andes DSP library supports distinct variance functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.6.7.1 nds32_var_f32

Declaration:

```
float32_t nds32_var_f32 (const float32_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Variance value

2.6.7.2 nds32_var_q31

Declaration:

```
q63_t nds32_var_q31 (const q31_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Variance value

2.6.7.3 nds32_var_q15

Declaration:

```
q31_t nds32_var_q15 (const q15_t *src, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [in] size number of elements in a vector

Return Value:

Variance value

2.7. Transform Functions

In Andes DSP library, the resolution is set as 1024 (the logarithm of 1024 to base 2 is 10) for default configuration. This default value affects the resolution of cosine and sine lookup tables (i.e. the number of elements in tables). It also affects transform functions since they use these tables to look up cosine or sine values. Thus, some suggested values for the input vector size are provided for transform functions in this chapter. Please find them from the function parameter `m` or the compilation option `FFT_LOGN` in examples. Using input vector size values that are not suggested, though still workable, may be at the risk of worse accuracy.

Note that overflow may occur in Q31 and Q15 transform functions. To avoid the problem, you may need to perform an arithmetic right shift operation before calling these functions. The shift amount is different according to the input format, which may vary depending on the input size. For details about input formats, please refer to the note of each Q31 and Q15 transform function in subsections of this chapter.

2.7.1. Radix-2 Complex FFT Functions

Radix-2 complex fast Fourier transform (CFFT) and inverse fast Fourier transform (CIFFT) functions implement the famous Cooley-Tukey algorithm to transform signals from time domain to frequency domain. The complex data in the input vector are arranged as [real, imaginary, real, imaginary..., real, imaginary].

Andes DSP library supports distinct Radix-2 CFFT and CIFFT functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below. For Q31 and Q15 Radix-2 CFFT and CIFFT functions, you may need to do an arithmetic right shift before calling them.

Note:

In this section, the parameter m represents the base 2 logarithm value of the input sample number and must be a number in the range [3, 10]. That is, if the input vector has 128 samples, then m is equal to 7 (i.e. $\log_2(128) = 7$).

2.7.1.1 nds32_cfft_rd2_f32

Declaration:

```
int32_t nds32_cfft_rd2_f32 (float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Example:

Given 128 samples (that is, `FFT_LOGN = 7`), the example of floating-point Radix-2 CFFT and CIFFT is as follows:

```
#define FFT_LOGN      7
float32_t src[2* (1 << FFT_LOGN)] = {...};
int32_t ret;
ret = nds32_cfft_rd2_f32(src, FFT_LOGN);
if (ret == 0)
    Success
Else
    Fail

ret = nds32_cifft_rd2_f32(src, FFT_LOGN);
if (ret == 0)
    Success
Else
    Fail
```

This example also serves as a reference for examples of Q31 and Q15 Radix-2 CFFT and CIFFT functions.

2.7.1.2 nds32_cifft_rd2_f32

Declaration:

```
int32_t nds32_cifft_rd2_f32 (float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

2.7.1.3 nds32_cfft_rd2_q31

Declaration:

```
int32_t nds32_cfft_rd2_q31 (q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.30	Q6.26
4	16	Q2.30	Q7.25
5	32	Q2.30	Q8.24
6	64	Q2.30	Q9.23
7	128	Q2.30	Q10.22
8	256	Q2.30	Q11.21
9	512	Q2.30	Q12.20
10	1024	Q2.30	Q13.19

2.7.1.4 nds32_cifft_rd2_q31

Declaration:

```
int32_t nds32_cifft_rd2_q31 (q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q6.26	Q2.30
4	16	Q7.25	Q2.30
5	32	Q8.24	Q2.30
6	64	Q9.23	Q2.30
7	128	Q10.22	Q2.30
8	256	Q11.21	Q2.30
9	512	Q12.20	Q2.30
10	1024	Q13.19	Q2.30

2.7.1.5 nds32_cfft_rd2_q15

Declaration:

```
int32_t nds32_cfft_rd2_q15 (q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.14	Q6.10
4	16	Q2.14	Q7.9
5	32	Q2.14	Q8.8
6	64	Q2.14	Q9.7
7	128	Q2.14	Q10.6
8	256	Q2.14	Q11.5
9	512	Q2.14	Q12.4
10	1024	Q2.14	Q13.3

2.7.1.6 nds32_cifft_rd2_q15

Declaration:

```
int32_t nds32_cifft_rd2_q15 (q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q6.10	Q2.14
4	16	Q7.9	Q2.14
5	32	Q8.8	Q2.14
6	64	Q9.7	Q2.14
7	128	Q10.6	Q2.14
8	256	Q11.5	Q2.14
9	512	Q12.4	Q2.14
10	1024	Q13.3	Q2.14

2.7.2. Radix-4 Complex FFT Functions

Radix-4 complex fast Fourier transform (CFFT) and inverse fast Fourier transform (CFFT) functions implement the famous Cooley-Tukey algorithm to transform signals from time domain to frequency domain. The complex data in the input vector are arranged as [real, imaginary, real, imaginary..., real, imaginary].

Andes DSP library supports distinct Radix-4 CFFT and CFFT functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below. For Q31 and Q15 Radix-4 CFFT and CFFT functions, you may need to do an arithmetic right shift before calling them.

Note:

In this section, the parameter m represents the base 2 logarithm value of the input sample number and must be a multiple of two in the range [4, 10]. That is, if the input vector has 256 samples, then m is equal to 8 (i.e. $\log_2(256) = 8$).

2.7.2.1 nds32_cfft_rd4_f32

Declaration:

```
int32_t nds32_cfft_rd4_f32 (float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set as 4, 6, 8 or 10

Return Value:

- 0 success
- 1 failure

Example:

Given 256 samples (that is, `FFT_LOGN = 8`), the example of floating-point Radix-4 CFFT and CIFFT is as follows:

```
#define FFT_LOGN      8
float32_t src[2* (1 << FFT_LOGN)] = {...};
int32_t ret;
ret = nds32_cfft_rd4_f32(src, FFT_LOGN);
if (ret == 0)
    Success
Else
    Fail

ret = nds32_cifft_rd4_f32(src, FFT_LOGN);
if (ret == 0)
    Success
Else
    Fail
```

This example also serves as a reference for examples of Q31 or Q15 Radix-4 CFFT and CIFFT functions.

2.7.2.2 nds32_cifft_rd4_f32

Declaration:

```
int32_t nds32_cifft_rd4_f32 (float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set as 4, 6, 8 or 10

Return Value:

- 0 success
- 1 failure

Example:

Please refer to Section 2.7.2.1 for an example.

2.7.2.3 nds32_cfft_rd4_q31

Declaration:

```
int32_t nds32_cfft_rd4_q31 (q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set as 4, 6, 8 or 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
4	16	Q2.30	Q7.25
6	64	Q2.30	Q9.23
8	256	Q2.30	Q11.21
10	1024	Q2.30	Q13.19

2.7.2.4 nds32_cifft_rd4_q31

Declaration:

```
int32_t nds32_cifft_rd4_q31 (q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set as 4, 6, 8 or 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
4	16	Q7.25	Q2.30
6	64	Q9.23	Q2.30
8	256	Q11.21	Q2.30
10	1024	Q13.19	Q2.30

2.7.2.5 nds32_cfft_rd4_q15

Declaration:

```
int32_t nds32_cfft_rd4_q15 (q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set as 4, 6, 8 or 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
4	16	Q2.14	Q7.9
6	64	Q2.14	Q9.7
8	256	Q2.14	Q11.5
10	1024	Q2.14	Q13.3

2.7.2.6 nds32_cifft_rd4_q15

Declaration:

```
int32_t nds32_cifft_rd4_q15 (q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set as 4, 6, 8 or 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
4	16	Q7.9	Q2.14
6	64	Q9.7	Q2.14
8	256	Q11.5	Q2.14
10	1024	Q13.3	Q2.14

2.7.3. Complex FFT Functions



Functions in this section internally call radix-2 or radix-4 CFFT/CIFFT functions introduced in Section 2.7.1 and 2.7.2 depending on the parameter m , which represents the base 2 logarithm value of the input sample number. If the value of m is a multiple of two such as 4 or 6, the radix-4 functions are used to provide a faster performance. Otherwise, radix-2 functions are used.

Andes DSP library supports distinct CFFT and CIFFT functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below. For Q31 and Q15 Radix-4 CFFT and CIFFT functions, you may need to do an arithmetic right shift before calling them.

2.7.3.1 nds32_cfft_f32

Declaration:

```
int32_t nds32_cfft_f32(float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Example:

Given 128 samples (that is, `FFT_LOGN = 7`), the example of floating-point CFFT and CIFFT is as follows:

```
#define FFT_LOGN      7
float32_t src[2* (1 << FFT_LOGN)] = {...};
int32_t ret;
ret = nds32_cfft_f32(src, FFT_LOGN);
if (ret == 0)
    Success
Else
    Fail

ret = nds32_cifft_f32(src, FFT_LOGN);
if (ret == 0)
    Success
Else
    Fail
```

This example also serves as a reference for examples of Q31 and Q15 CFFT and CIFFT functions.

2.7.3.2 nds32_cifft_f32

Declaration:

```
int32_t nds32_cifft_f32 (float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

2.7.3.3 nds32_cfft_q31

Declaration:

```
int32_t nds32_cfft_q31 (q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.30	Q6.26
4	16	Q2.30	Q7.25
5	32	Q2.30	Q8.24
6	64	Q2.30	Q9.23
7	128	Q2.30	Q10.22
8	256	Q2.30	Q11.21
9	512	Q2.30	Q12.20
10	1024	Q2.30	Q13.19

2.7.3.4 nds32_cifft_q31

Declaration:

```
int32_t nds32_cifft_q31 (q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q6.26	Q2.30
4	16	Q7.25	Q2.30
5	32	Q8.24	Q2.30
6	64	Q9.23	Q2.30
7	128	Q10.22	Q2.30
8	256	Q11.21	Q2.30
9	512	Q12.20	Q2.30
10	1024	Q13.19	Q2.30

2.7.3.5 nds32_cfft_q15

Declaration:

```
int32_t nds32_cfft_q15(q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.14	Q6.10
4	16	Q2.14	Q7.9
5	32	Q2.14	Q8.8
6	64	Q2.14	Q9.7
7	128	Q2.14	Q10.6
8	256	Q2.14	Q11.5
9	512	Q2.14	Q12.4
10	1024	Q2.14	Q13.3

2.7.3.6 nds32_cifft_q15

Declaration:

```
int32_t nds32_cifft_q15 (q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of the sample number and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q6.10	Q2.14
4	16	Q7.9	Q2.14
5	32	Q8.8	Q2.14
6	64	Q9.7	Q2.14
7	128	Q10.6	Q2.14
8	256	Q11.5	Q2.14
9	512	Q12.4	Q2.14
10	1024	Q13.3	Q2.14

2.7.4. DCT Type II Functions

Discrete cosine transform (DCT) type II functions implement DCT with the following equation:

$$y[k] = \sum_{n=0}^{N-1} x[n] * \cos(\pi * (2 * n + 1) * \frac{k}{(2 * N)})$$

and IDCT (DCT Type III, also called as the inverse of DCT II), with:

$$x[k] = \frac{2.0}{N} * \sum_{n=0}^{N-1} c[n] * y[n] * \cos(\pi * (2 * k + 1) * \frac{n}{(2 * N)})$$

where $c[0] = 1 / 2.0$ and $c[n] = 1$ for $n \neq 0$.

Andes DSP library supports distinct DCT type II and IDCT functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below. For Q31 and Q15 DCT type II and IDCT functions, you may need to do an arithmetic right shift before calling them.

2.7.4.1 nds32_dct_f32

Declaration:

```
void nds32_dct_f32(float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 8

Return Value:

None

Example:

Given 256 samples (that is, `FFT_LOGN = 8`), the example of floating-point (DCT) type II and IDCT is as follows:

```
#define FFT_LOGN      8
float32_t src[(1 << FFT_LOGN)] = {...};
nds32_dct_f32(src, FFT_LOGN);
nds32_idct_f32(src, FFT_LOGN);
```

This example also serves as a reference for examples of Q31 or Q15 DCT type II and IDCT functions.

2.7.4.2 nds32_idct_f32

Declaration:

```
void nds32_idct_f32(float32_t *src, uint32_t m)
```

Parameters:

- `[in, out] *src` pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- `[in] m` base 2 logarithm value of samples and it can be set from 3 to 8

Return Value:

None

Example:

Please refer to Section 2.7.4.1 for an example.

2.7.4.3 nds32_dct_q31

Declaration:

```
void nds32_dct_q31(q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 8

Return Value:

None

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.30	Q5.27
4	16	Q2.30	Q6.26
5	32	Q2.30	Q7.25
6	64	Q2.30	Q8.24
7	128	Q2.30	Q9.23
8	256	Q2.30	Q10.22

2.7.4.4 nds32_idct_q31

Declaration:

```
void nds32_idct_q31(q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 8

Return Value:

None

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q5.27	Q2.30
4	16	Q6.26	Q2.30
5	32	Q7.25	Q2.30
6	64	Q8.24	Q2.30
7	128	Q9.23	Q2.30
8	256	Q10.22	Q2.30

2.7.4.5 nds32_dct_q15

Declaration:

```
void nds32_dct_q15(q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 8

Return Value:

None

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.14	Q5.11
4	16	Q2.14	Q6.10
5	32	Q2.14	Q7.9
6	64	Q2.14	Q8.8
7	128	Q2.14	Q9.7
8	256	Q2.14	Q10.6

2.7.4.6 nds32_idct_q15

Declaration:

```
void nds32_idct_q15(q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 8

Return Value:

None

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q5.11	Q2.14
4	16	Q6.10	Q2.14
5	32	Q7.9	Q2.14
6	64	Q8.8	Q2.14
7	128	Q9.7	Q2.14
8	256	Q10.6	Q2.14

2.7.5. DCT Type IV Functions

Discrete cosine transform (DCT) type IV functions implement DCT transform with the following equation:

$$y[k] = \sum_{n=0}^{N-1} x[n] * \cos\left(\text{PI} * (2 * k + 1) * \frac{(2 * n + 1)}{(4 * N)}\right)$$

and IDCT with:

$$x[k] = \frac{2}{N} * \sum_{n=0}^{N-1} y[n] * \cos\left(\text{PI} * (2 * k + 1) * \frac{(2 * n + 1)}{(4 * N)}\right)$$

Andes DSP library supports distinct DCT and IDCT type IV functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below. For Q31 and Q15 DCT and IDCT type IV functions, you may need to do an arithmetic right shift before calling them.

2.7.5.1 nds32_dct4_f32

Declaration:

```
void nds32_dct4_f32(float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 7

Return Value:

None

Example:

Given 128 samples (that is, `FFT_LOGN = 7`), the example of floating-point DCT or IDCT type IV transform is as follows:

```
#define FFT_LOGN      8
float32_t src[(1 << FFT_LOGN)] = {...};
nds32_dct4_f32(src, FFT_LOGN);
nds32_idct4_f32(src, FFT_LOGN);
```

This example also serves as a reference for examples of Q31 or Q15 DCT and IDCT type IV functions.

2.7.5.2 nds32_idct4_f32

Declaration:

```
void nds32_idct4_f32(float32_t *src, uint32_t m)
```

Parameters:

- `[in, out] *src` pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- `[in] m` base 2 logarithm value of samples and it can be set from 3 to 7

Return Value:

None

Example:

Please refer to Section 2.7.5.1 for an example.

2.7.5.3 nds32_dct4_q31

Declaration:

```
void nds32_dct4_q31(q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 7

Return Value:

None

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.30	Q6.26
4	16	Q2.30	Q7.25
5	32	Q2.30	Q8.24
6	64	Q2.30	Q9.23
7	128	Q2.30	Q10.22

2.7.5.4 nds32_idct4_q31

Declaration:

```
void nds32_idct4_q31(q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 7

Return Value:

None

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q6.26	Q2.30
4	16	Q7.25	Q2.30
5	32	Q8.24	Q2.30
6	64	Q9.23	Q2.30
7	128	Q10.22	Q2.30

2.7.5.5 nds32_dct4_q15

Declaration:

```
void nds32_dct4_q15(q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 7

Return Value:

None

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.14	Q6.10
4	16	Q2.14	Q7.9
5	32	Q2.14	Q8.8
6	64	Q2.14	Q9.7
7	128	Q2.14	Q10.6

2.7.5.6 nds32_idct4_q15

Declaration:

```
void nds32_idct4_q15(q15_t *src, uint32_t m)
```

Parameters:

- **[in, out] *src** pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- **[in] m** base 2 logarithm value of samples and it can be set from 3 to 7

Return Value:

None

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q6.10	Q2.14
4	16	Q7.9	Q2.14
5	32	Q8.8	Q2.14
6	64	Q9.7	Q2.14
7	128	Q10.6	Q2.14

2.7.6. Real FFT Functions

Real fast Fourier transform (RFFT) and inverse fast Fourier transform (RIFFT) functions transform signals composed of real values from time domain to frequency domain.

Andes DSP library supports distinct RFFT and RIFFT functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below. For Q31 and Q15 RFFT and RIFFT functions, you may need to do an arithmetic right shift before calling them.

2.7.6.1 nds32_rfft_f32

Declaration:

```
int32_t nds32_rfft_f32(float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Example:

Given 128 samples (that is, `FFT_LOGN = 7`), the example of floating-point RFFT and RIFFT is as follows:

```
#define FFT_LOGN      7
float32_t src[(1 << FFT_LOGN)] = {...};
int32_t ret;
ret = nds32_rfft_f32(src, FFT_LOGN);
if (ret == 0)
    Success
else
    Fail

ret = nds32_rifft_f32(src, FFT_LOGN);
if (ret == 0)
    Success
else
    Fail
```

This example also serves as a reference for examples of Q31 or Q15 RFFT and RIFFT functions.

2.7.6.2 nds32_rifft_f32

Declaration:

```
int32_t nds32_rifft_f32(float32_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

2.7.6.3 nds32_rfft_q31

Declaration:

```
int32_t nds32_rfft_q31(q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.30	Q6.26
4	16	Q2.30	Q7.25
5	32	Q2.30	Q8.24
6	64	Q2.30	Q9.23
7	128	Q2.30	Q10.22
8	256	Q2.30	Q11.21
9	512	Q2.30	Q12.20
10	1024	Q2.30	Q13.19

2.7.6.4 nds32_rifft_q31

Declaration:

```
int32_t nds32_rifft_q31(q31_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q6.26	Q2.30
4	16	Q7.25	Q2.30
5	32	Q8.24	Q2.30
6	64	Q9.23	Q2.30
7	128	Q10.22	Q2.30
8	256	Q11.21	Q2.30
9	512	Q12.20	Q2.30
10	1024	Q13.19	Q2.30

2.7.6.5 nds32_rfft_q15

Declaration:

```
int32_t nds32_rfft_q15(q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q2.14	Q6.10
4	16	Q2.14	Q7.9
5	32	Q2.14	Q8.8
6	64	Q2.14	Q9.7
7	128	Q2.14	Q10.6
8	256	Q2.14	Q11.5
9	512	Q2.14	Q12.4
10	1024	Q2.14	Q13.3

2.7.6.6 nds32_rifft_q15

Declaration:

```
int32_t nds32_rifft_q15(q15_t *src, uint32_t m)
```

Parameters:

- [in, out] *src pointer of the input vector. After the function is executed, the output will be stored in the input vector.
- [in] m base 2 logarithm value of samples and it can be set from 3 to 10

Return Value:

- 0 success
- 1 failure

Note:

The input and output formats are listed below. To satisfy the input format corresponding to your input size, you may need to perform an arithmetic shift operation before calling this function.

m	Size	Input format	Output format
3	8	Q6.10	Q2.14
4	16	Q7.9	Q2.14
5	32	Q8.8	Q2.14
6	64	Q9.7	Q2.14
7	128	Q10.6	Q2.14
8	256	Q11.5	Q2.14
9	512	Q12.4	Q2.14
10	1024	Q13.3	Q2.14

2.8. Utils Functions

2.8.1. Arc Tangent Functions

Arc tangent functions are the inverse of trigonometric functions which return the inverse of tangent given an input value. The input values and output results of arc tangent functions are in radians and the output results are always in the range $[-\pi/2, \pi/2]$.

Andes DSP library supports distinct arc tangent functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.8.1.1 nds32_atan_f32

Declaration:

```
float32_t nds32_atan_f32 (float32_t src)
```

Parameters:

- [in] src input value (radian)

Return Value:

Radian value

Example:

```
float32_t src = 0.5;  
float32_t dst;  
dst = nds32_atan_f32 (src);
```

This example also serves as a reference for examples of Q31 or Q15 arc tangent functions.

2.8.1.2 nds32_atan_q31

Declaration:

q31_t nds32_atan_q31(q31_t src)

Parameters:

- [in] src input value (radian)

Return Value:

Radian value

2.8.1.3 nds32_atan_q15

Declaration:

q15_t nds32_atan_q15 (q15_t src)

Parameters:

- [in] src input value (radian)

Return Value:

Radian value

2.8.2. Arc Tangent 2 Functions

Arc tangent 2 functions are similar to arc tangent functions but accept two inputs y and x to compute the inverse tangent value. Normally, the output results are all in radians and always in the range $[-\pi, \pi]$. For convenient calculation, they are mapped to $[-1, 1)$ in Q number functions, but you can use conversion formulas to convert them to their floating-point counterparts.

Andes DSP library supports distinct arc tangent 2 functions for floating-point, Q31 and Q15 data types. These functions are introduced in the subsections below.

2.8.2.1 nds32_atan2_f32

Declaration:

q31_t nds32_atan2_f32 (float32_t y, float32_t x)

Parameters:

- [in] y input value y
- [in] x input value x

Return Value:

Radian value

Note:

The return value is in the floating-point format and always in the range $[-\pi, \pi]$.

2.8.2.2 nds32_atan2_q31

Declaration:

`q31_t nds32_atan2_q31 (q31_t y, q31_t x)`

Parameters:

- `[in] y` input value y
- `[in] x` input value x

Return Value:

Radian value

Note:

The return value is in the Q31 format and always in the range `[0x80000000, 0x7FFFFFFF]` (namely, `[-1, 1)`). To convert the value to the floating-point format, apply the following formula:

```
floating point value = (return value * π) / (2 ^ 31)
```

2.8.2.3 nds32_atan2_q15

Declaration:

q15_t nds32_atan2_q15 (q15_t y, q15_t x)

Parameters:

- [in] y input value y
- [in] x input value x

Return Value:

Radian value

Note:

The return value is in the Q15 format and always in the range [0x8000, 0x7FFF] (namely, [-1, 1)). To convert the value to the floating-point format, apply the following formula:

floating point value = (return value * π) / (2 ^ 15)

2.8.3. Cosine and Sine Functions

Cosine and sine functions compute the cosine and sine values.

The input values of these functions are in radians and their ranges are different between floating-point and Q number functions.

For floating-point functions, there is no limitation for the input range since the inputs, whatever values they are, will be pre-processed by the following snippet into the range $[-2\pi, 2\pi]$:

```
src = abs(src)
while (src >= 2π)
{
    src -= 2π;
}
```

where `src` is the input value.

For Q number (Q31 and Q15) functions, they only support the input range $[-1, 1)$ to represent the Q format range $[-\pi, \pi]$. Thus, the input values may need to be converted into the Q format range before these functions are called.

Andes DSP library supports distinct cosine and sine functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.8.3.1 nds32_cos_f32

Declaration:

```
float32_t nds32_cos_f32 (float32_t src)
```

Parameters:

- [in] src input value (radian)

Return Value:

Cosine value of the input

Example:

```
float32_t src = 0.5;  
float32_t dst;  
dst = nds32_cos_f32 (src);
```

This example also serves as a reference for examples of Q31 or Q15 cosine/sine functions.

2.8.3.2 nds32_cos_q31

Declaration:

q31_t nds32_cos_q31 (q31_t src)

Parameters:

- [in] src input value (radian)

Return Value:

Cosine value of the input

Note:

The input range is [0x80000000, 0x7FFFFFFF] and mapped to the range $[-\pi, \pi]$.

2.8.3.3 nds32_cos_q15

Declaration:

`q15_t nds32_cos_q15 (q15_t src)`

Parameters:

- `[in] src` input value (radian)

Return Value:

Cosine value of the input

Note:

The input range is `[0x8000, 0x7FFF]` and mapped to the range `[- $\pi$ ,  $\pi$ ]`.

2.8.3.4 nds32_sin_f32

Declaration:

`float32_t nds32_sin_f32 (float32_t src)`

Parameters:

- `[in] src` input value (radian)

Return Value:

Sine value of the input

2.8.3.5 nds32_sin_q31

Declaration:

q31_t nds32_sin_q31 (q31_t src)

Parameters:

- [in] src input value (radian)

Return Value:

Sine value of the input

Note:

The input range is [0x80000000, 0x7FFFFFFF] and mapped to the range $[-\pi, \pi]$.

2.8.3.6 nds32_sin_q15

Declaration:

`q15_t nds32_sin_q15 (q15_t src)`

Parameters:

- `[in] src` input value (radian)

Return Value:

Sine value of the input

Note:

The input range is `[0x8000, 0x7FFF]` and mapped to the range `[- $\pi$ ,  $\pi$ ]`.

2.8.4. Conversion Functions

Conversion functions convert element values in a source vector from one data type to another and write the results into a destination vector. There are following data type conversions:

float32_t → q31_t
float32_t → q15_t
float32_t → q7_t
q31_t → float32_t
q31_t → q15_t
q31_t → q7_t
q15_t → float32_t
q15_t → q31_t
q15_t → q7_t
q7_t → float32_t
q7_t → q31_t
q7_t → q15_t

These conversion functions are introduced in the subsections below.

2.8.4.1 nds32_convert_f32_q31

Declaration:

```
void nds32_convert_f32_q31 (float32_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. This function converts values from `float32_t` to `q31_t`.
2. The results will be saturated to the Q31 range `[0x80000000, 0x7FFFFFFF]`.

2.8.4.2 nds32_convert_f32_q15

Declaration:

```
void nds32_convert_f32_q15 (float32_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. This function converts values from `float32_t` to `q15_t`.
2. The results will be saturated to the Q15 range `[0x8000, 0x7FFF]`.

2.8.4.3 nds32_convert_f32_q7

Declaration:

```
void nds32_convert_f32_q7 (float32_t *src, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. This function converts values from `float32_t` to `q7_t`.
2. The results will be saturated to the Q7 range `[0x80, 0x7F]`.

2.8.4.4 nds32_convert_q31_f32

Declaration:

```
void nds32_convert_q31_f32 (q31_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

This function converts values from `q31_t` to `float32_t`.

2.8.4.5 nds32_convert_q31_q15

Declaration:

```
void nds32_convert_q31_q15 (q31_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. This function converts values from `q31_t` to `q15_t`.
2. There is no overflow situation since the values are simply right shifted with 16 bits.

2.8.4.6 nds32_convert_q31_q7

Declaration:

```
void nds32_convert_q31_q7 (q31_t *src, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

1. This function converts values from `q31_t` to `q7_t`.
2. There is no overflow situation since the values are simply right shifted with 24 bits.

2.8.4.7 nds32_convert_q15_f32

Declaration:

```
void nds32_convert_q15_f32 (q15_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

This function converts values from `q15_t` to `float32_t`.

2.8.4.8 nds32_convert_q15_q31

Declaration:

```
void nds32_convert_q15_q31 (q15_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

This function converts values from `q15_t` to `q31_t` (left-shifted with 16 bits).

2.8.4.9 nds32_convert_q15_q7

Declaration:

```
void nds32_convert_q15_q7 (q15_t *src, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

This function converts values from `q15_t` to `q7_t` (right-shifted with 8 bits).

2.8.4.10 nds32_convert_q7_f32

Declaration:

```
void nds32_convert_q7_f32 (q7_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

This function converts values from `q7_t` to `float32_t`.

2.8.4.11 nds32_convert_q7_q31

Declaration:

```
void nds32_convert_q7_q31 (q7_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

This function converts values from `q7_t` to `q31_t` (left-shifted with 24 bits).

2.8.4.12 nds32_convert_q7_q15

Declaration:

```
void nds32_convert_q7_q15 (q7_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

Note:

This function converts values from `q7_t` to `q15_t` (left-shifted with 8 bits).

2.8.5. Duplicate Functions

Duplicate functions copy element values in a source vector and write the values one-by-one into a destination vector. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = src[n];
```

Andes DSP library supports distinct duplicate functions for the following data types:
floating-point, Q31, Q15 and Q7. These functions are introduced in the subsections below.

2.8.5.1 nds32_dup_f32

Declaration:

```
void nds32_dup_f32 (float32_t *src, float32_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.8.5.2 nds32_dup_q31

Declaration:

```
void nds32_dup_q31 (q31_t *src, q31_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.8.5.3 nds32_dup_q15

Declaration:

```
void nds32_dup_q15 (q15_t *src, q15_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.8.5.4 nds32_dup_q7

Declaration:

```
void nds32_dup_q7 (q7_t *src, q7_t *dst, uint32_t size)
```

Parameters:

- [in] *src pointer of the input vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.8.6. Set Functions

Set functions set all elements in a destination vector to a constant value. The behavior can be defined as follows:

```
for (n = 0; n < size; n++)  
    dst[n] = value;
```

Andes DSP library supports distinct set functions for the following data types: floating-point, Q31, Q15 and Q7. These functions are introduced in the subsections below.

2.8.6.1 nds32_set_f32

Declaration:

```
void nds32_set_f32(float32_t val, float32_t *dst, uint32_t size)
```

Parameters:

- [in] val value to be written to the output vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.8.6.2 nds32_set_q31

Declaration:

```
void nds32_set_q31(q31_t val, q31_t *dst, uint32_t size)
```

Parameters:

- [in] val value to be written to the output vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.8.6.3 nds32_set_q15

Declaration:

```
void nds32_set_q15(q15_t val, q15_t *dst, uint32_t size)
```

Parameters:

- [in] val value to be written to the output vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.8.6.4 nds32_set_q7

Declaration:

```
void nds32_set_q7(q7_t val, q7_t *dst, uint32_t size)
```

Parameters:

- [in] val value to be written to the output vector
- [out] *dst pointer of the output vector
- [in] size number of elements in a vector

Return Value:

None

2.8.7. Square Root Functions

Square root functions return the square root value of the input.

Andes DSP library supports distinct square root functions for the following data types: floating-point, Q31 and Q15. These functions are introduced in the subsections below.

2.8.7.1 nds32_sqrt_f32

Declaration:

`float32_t nds32_sqrt_f32 (float32_t src)`

Parameters:

- `[in] src` input value

Return Value:

Square root of the input

2.8.7.2 nds32_sqrt_q31

Declaration:

q31_t nds32_sqrt_q31 (q31_t src)

Parameters:

- [in] src input value

Return Value:

Square root of the input

2.8.7.3 nds32_sqrt_q15

Declaration:

q15_t nds32_sqrt_q15 (q15_t src)

Parameters:

- [in] src input value

Return Value:

Square root of the input

3. Intrinsic Functions for Programming DSP Applications

Built-in to Andes compiler to deliver better code optimization, Andes intrinsic functions help users to use certain instructions as C-like functions without bothering to program in assembly. This chapter summarizes intrinsic functions which can be used to program DSP applications and groups them into following categories:

- SIMD Data Processing Intrinsic Functions
- Non-SIMD Data Processing Intrinsic Functions
- 64-bit Profile Intrinsic Functions
- Saturation ISA Extension Intrinsic Functions

Each intrinsic function is presented in the following format:

Computation Type

Function Syntax

Function Operation*

Mapped Instruction

*For definitions of shorthands used to explain function operations, please refer to Section 3.1 .

For some of these intrinsic functions, you may refer to *Andes Programming Guide for ISA V3*, *AndeStar™ Instruction Set Architecture Manual* and *AndeStar DSP ISA Extension Specification* for more detailed information.

3.1. Definitions of Shorthands for Explaining Function Operation

This section lists definitions of shorthands used to explain operations or behaviors of intrinsic functions.

- **Ra, Rb** and **Rc**: operands for parameters a, b and c.
- **Rt**: operand for the returned value.
- **[x]**: operator for bit x.
- **[x:y]**: operator for indicating the bit indices (i.e. From bit x to y).
- **<<**: signed/unsigned left shifter.
- **s>>**: signed right shifter.
- **u>>**: unsigned right shifter.
- **sr>>**: signed rounding right shifter (“rounding” means adding 1 to the most significant discarded bit).
- **ur>>**: unsigned rounding right shifter.
- **<, <=, >, >=**: signed comparison.
- **u<, u<=, u>, u>=**: unsigned comparison.
- **u***: unsigned multiplication.
- **SEm()**: sign-extend data to m-bit.
- **ZEm()**: zero-extend data to m-bit.
- **SAT.Qn()**: saturate to the range of $[-2^n, 2^n - 1]$. If saturation occurs, set PSW.OV.
- **SAT.Um()**: saturate to the range of $[0, 2^m - 1]$. If saturation occurs, set PSW.OV.
- **ABS.Qn()**: get the absolute value and saturate to the range of $[0, 2^n - 1]$. If saturation occurs, set PSW.OV.
- **REVB()**: reverse the bit position.

3.2. SIMD Data Processing Intrinsic Functions

3.2.1. 16-bit Addition & Subtraction Intrinsic Functions

16-bit Addition

Syntax `unsigned int __nds32__add16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32__v_uadd16(uint16x2_t a, uint16x2_t b);`
 `int16x2_t __nds32__v_sadd16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = Ra[31:16] + Rb[31:16];$
 $Rt[15:0] = Ra[15:0] + Rb[15:0];$

Mapped Instruction `ADD16`

16-bit Signed Halving Addition

Syntax `unsigned int __nds32__radd16(unsigned int a, unsigned int b);`
 `int16x2_t __nds32__v_radd16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = (Ra[31:16] + Rb[31:16]) \gg 1;$
 $Rt[15:0] = (Ra[15:0] + Rb[15:0]) \gg 1;$

Mapped Instruction `RADD16`

16-bit Unsigned Halving Addition

Syntax `unsigned int __nds32__uradd16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32__v_uradd16(uint16x2_t a, uint16x2_t b);`

Operation $Rt[31:16] = (Ra[31:16] + Rb[31:16]) \gg 1;$
 $Rt[15:0] = (Ra[15:0] + Rb[15:0]) \gg 1;$

Mapped Instruction `URADD16`

16-bit Signed Saturating Addition

Syntax `unsigned int __nds32__kadd16(unsigned int a, unsigned int b);`
 `int16x2_t __nds32__v_kadd16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = \text{SAT.Q15}(Ra[31:16] + Rb[31:16]);$
 $Rt[15:0] = \text{SAT.Q15}(Ra[15:0] + Rb[15:0]);$

Mapped Instruction KADD16

16-bit Unsigned Saturating Addition

Syntax `unsigned int __nds32__ukadd16(unsigned int a, unsigned int b);
uint16x2_t __nds32__v_ukadd16(uint16x2_t a, uint16x2_t b);`

Operation $Rt[31:16] = \text{SAT.U16}(Ra[31:16] + Rb[31:16]);$
 $Rt[15:0] = \text{SAT.U16}(Ra[15:0] + Rb[15:0]);$

Mapped Instruction UKADD16

16-bit Subtraction

Syntax `unsigned int __nds32__sub16(unsigned int a, unsigned int b);
uint16x2_t __nds32__v_usub16(uint16x2_t a, uint16x2_t b);
int16x2_t __nds32__v_ssub16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = Ra[31:16] - Rb[31:16];$
 $Rt[15:0] = Ra[15:0] - Rb[15:0];$

Mapped Instruction SUB16

16-bit Signed Halving Subtraction

Syntax `unsigned int __nds32__rsub16(unsigned int a, unsigned int b);
int16x2_t __nds32__v_rsub16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = (Ra[31:16] - Rb[31:16]) \gg 1;$
 $Rt[15:0] = (Ra[15:0] - Rb[15:0]) \gg 1;$

Mapped Instruction RSUB16

16-bit Unsigned Halving Subtraction

Syntax `unsigned int __nds32__ursub16(unsigned int a, unsigned int b);
uint16x2_t __nds32__v_ursub16(uint16x2_t a, uint16x2_t b);`

Operation $Rt[31:16] = (Ra[31:16] - Rb[31:16]) \gg 1;$
 $Rt[15:0] = (Ra[15:0] - Rb[15:0]) \gg 1;$

Mapped Instruction URSUB16

16-bit Signed Saturating Subtraction

Syntax `unsigned int __nds32_ksub16(unsigned int a, unsigned int b);`
 `int16x2_t __nds32_v_ksub16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = SAT.Q15(Ra[31:16] - Rb[31:16]);`
 `Rt[15:0] = SAT.Q15(Ra[15:0] - Rb[15:0]);`

Mapped Instruction `KSUB16`

16-bit Unsigned Saturating Subtraction

Syntax `unsigned int __nds32_uksb16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32_v_uksb16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = SAT.U16(Ra[31:16] - Rb[31:16]);`
 `Rt[15:0] = SAT.U16(Ra[15:0] - Rb[15:0]);`

Mapped Instruction `UKSUB16`

16-bit Cross Add & Sub

Syntax `unsigned int __nds32_cras16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32_v_ucras16(uint16x2_t a, uint16x2_t b);`
 `int16x2_t __nds32_v_scras16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = Ra[31:16] + Rb[15:0];`
 `Rt[15:0] = Ra[15:0] - Rb[31:16];`

Mapped Instruction `CRAS16`

16-bit Signed Halving Cross Addition & Subtraction

Syntax `unsigned int __nds32_rcras16(unsigned int a, unsigned int b);`
 `int16x2_t __nds32_v_rcras16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] + Rb[15:0]) s>> 1;`
 `Rt[15:0] = (Ra[15:0] - Rb[31:16]) s>> 1;`

Mapped Instruction `RCRAS16`

16-bit Unsigned Halving Cross Addition & Subtraction

Syntax `unsigned int __nds32_urcras16(unsigned int a, unsigned int b);`

```
uint16x2_t __nds32__v_urcras16(uint16x2_t a, uint16x2_t b);
```

Operation $Rt[31:16] = (Ra[31:16] + Rb[15:0]) \gg 1;$
 $Rt[15:0] = (Ra[15:0] - Rb[31:16]) \gg 1;$

Mapped Instruction URCRAS16

16-bit Signed Saturating Cross Addition & Subtraction

Syntax `unsigned int __nds32__kcras16(unsigned int a, unsigned int b);`
`int16x2_t __nds32__v_kcras16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = \text{SAT.Q15}(Ra[31:16] + Rb[15:0]);$
 $Rt[15:0] = \text{SAT.Q15}(Ra[15:0] - Rb[31:16]);$

Mapped Instruction KCRAS16

16-bit Unsigned Saturating Cross Addition & Subtraction

Syntax `unsigned int __nds32__ukcras16(unsigned int a, unsigned int b);`
`uint16x2_t __nds32__v_ukcras16(uint16x2_t a, uint16x2_t b);`

Operation $Rt[31:16] = \text{SAT.U16}(Ra[31:16] + Rb[15:0]);$
 $Rt[15:0] = \text{SAT.U16}(Ra[15:0] - Rb[31:16]);$

Mapped Instruction UKCRAS16

16-bit Cross Subtraction & Addition

Syntax `unsigned int __nds32__crsa16(unsigned int a, unsigned int b);`
`uint16x2_t __nds32__v_ucrsa16(uint16x2_t a, uint16x2_t b);`
`int16x2_t __nds32__v_scrsa16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = Ra[31:16] - Rb[15:0];$
 $Rt[15:0] = Ra[15:0] + Rb[31:16];$

Mapped Instruction CRSAS16

16-bit Signed Halving Cross Subtraction & Addition

Syntax `unsigned int __nds32__rcrsa16(unsigned int a, unsigned int b);`
`int16x2_t __nds32__v_rcrsa16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = (Ra[31:16] - Rb[15:0]) \gg 1;$
 $Rt[15:0] = (Ra[15:0] + Rb[31:16]) \gg 1;$

Mapped Instruction RCRSA16

16-bit Unsigned Halving Cross Subtraction & Addition

Syntax `unsigned int __nds32_urcrsa16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32_v_urcrsa16(uint16x2_t a, uint16x2_t b);`

Operation $Rt[31:16] = (Ra[31:16] - Rb[15:0]) \gg 1;$
 $Rt[15:0] = (Ra[15:0] + Rb[31:16]) \gg 1;$

Mapped Instruction URCRSA16

16-bit Signed Saturating Cross Subtraction & Addition

Syntax `unsigned int __nds32_kcrsa16(unsigned int a, unsigned int b);`
 `int16x2_t __nds32_v_kcrsa16(int16x2_t a, int16x2_t b);`

Operation $Rt[31:16] = \text{SAT.Q15}(Ra[31:16] - Rb[15:0]);$
 $Rt[15:0] = \text{SAT.Q15}(Ra[15:0] + Rb[31:16]);$

Mapped Instruction KCRSA16

16-bit Unsigned Saturating Cross Subtraction & Addition

Syntax `unsigned int __nds32_ukcrsa16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32_v_ukcrsa16(uint16x2_t a, uint16x2_t b);`

Operation $Rt[31:16] = \text{SAT.U16}(Ra[31:16] - Rb[15:0]);$
 $Rt[15:0] = \text{SAT.U16}(Ra[15:0] + Rb[31:16]);$

Mapped Instruction UKCRSA16

3.2.2. 8-bit Addition & Subtraction Intrinsic Functions

8-bit Addition

Syntax

```
unsigned int __nds32__add8(unsigned int a, unsigned int b);
uint8x4_t __nds32__v_uadd8(uint8x4_t a, uint8x4_t b);
int8x4_t __nds32__v_sadd8(int8x4_t a, int8x4_t b);
```

Operation

```
Rt[31:24] = Ra[31:24] + Rb[31:24];
Rt[23:16] = Ra[23:16] + Rb[23:16];
Rt[15:8] = Ra[15:8] + Rb[15:8];
Rt[7:0] = Ra[7:0] + Rb[7:0];
```

Mapped Instruction ADD8

8-bit Signed Halving Addition

Syntax

```
unsigned int __nds32__radd8(unsigned int a, unsigned int b);
int8x4_t __nds32__v_radd8(int8x4_t a, int8x4_t b);
```

Operation

```
Rt[31:24] = (Ra[31:24] + Rb[31:24]) s>> 1;
Rt[23:16] = (Ra[23:16] + Rb[23:16]) s>> 1;
Rt[15:8] = (Ra[15:8] + Rb[15:8]) s>> 1;
Rt[7:0] = (Ra[7:0] + Rb[7:0]) s>> 1;
```

Mapped Instruction RADD8

8-bit Unsigned Halving Addition

Syntax

```
unsigned int __nds32__uradd8(unsigned int a, unsigned int b);
uint8x4_t __nds32__v_uradd8(uint8x4_t a, uint8x4_t b);
```

Operation

```
Rt[31:24] = (Ra[31:24] + Rb[31:24]) u>> 1;
Rt[23:16] = (Ra[23:16] + Rb[23:16]) u>> 1;
Rt[15:8] = (Ra[15:8] + Rb[15:8]) u>> 1;
Rt[7:0] = (Ra[7:0] + Rb[7:0]) u>> 1;
```

Mapped Instruction URADD8

8-bit Signed Saturating Addition

Syntax `unsigned int __nds32__kadd8(unsigned int a, unsigned int b);
int8x4_t __nds32__v_kadd8(int8x4_t a, int8x4_t b);`

Operation `Rt[31:24] = SAT.Q7(Ra[31:24] + Rb[31:24]);
Rt[23:16] = SAT.Q7(Ra[23:16] + Rb[23:16]);
Rt[15:8] = SAT.Q7(Ra[15:8] + Rb[15:8]);
Rt[7:0] = SAT.Q7(Ra[7:0] + Rb[7:0]);`

Mapped Instruction `KADD8`

8-bit Unsigned Saturating Addition

Syntax `unsigned int __nds32__ukadd8(unsigned int a, unsigned int b);
uint8x4_t __nds32__v_ukadd8(uint8x4_t a, uint8x4_t b);`

Operation `Rt[31:24] = SAT.U8(Ra[31:24] + Rb[31:24]);
Rt[23:16] = SAT.U8(Ra[23:16] + Rb[23:16]);
Rt[15:8] = SAT.U8(Ra[15:8] + Rb[15:8]);
Rt[7:0] = SAT.U8(Ra[7:0] + Rb[7:0]);`

Mapped Instruction `UKADD8`

8-bit Subtraction

Syntax `unsigned int __nds32__sub8(unsigned int a, unsigned int b);
uint8x4_t __nds32__v_usub8(uint8x4_t a, uint8x4_t b);
int8x4_t __nds32__v_ssub8(int8x4_t a, int8x4_t b);`

Operation `Rt[31:24] = Ra[31:24] - Rb[31:24];
Rt[23:16] = Ra[23:16] - Rb[23:16];
Rt[15:8] = Ra[15:8] - Rb[15:8];
Rt[7:0] = Ra[7:0] - Rb[7:0];`

Mapped Instruction `SUB8`

8-bit Signed Halving Subtraction

Syntax `unsigned int __nds32_rsub8(unsigned int a, unsigned int b);
int8x4_t __nds32_v_rsub8(int8x4_t a, int8x4_t b);`

Operation `Rt[31:24] = (Ra[31:24] - Rb[31:24]) s>> 1;
Rt[23:16] = (Ra[23:16] - Rb[23:16]) s>> 1;
Rt[15:8] = (Ra[15:8] - Rb[15:8]) s>> 1;
Rt[7:0] = (Ra[7:0] - Rb[7:0]) s>> 1;`

Mapped Instruction `RSUB8`

8-bit Unsigned Halving Subtraction

Syntax `unsigned int __nds32_ursub8(unsigned int a, unsigned int b);
uint8x4_t __nds32_v_ursub8(uint8x4_t a, uint8x4_t b);`

Operation `Rt[31:24] = (Ra[31:24] - Rb[31:24]) u>> 1;
Rt[23:16] = (Ra[23:16] - Rb[23:16]) u>> 1;
Rt[15:8] = (Ra[15:8] - Rb[15:8]) u>> 1;
Rt[7:0] = (Ra[7:0] - Rb[7:0]) u>> 1;`

Mapped Instruction `URSUB8`

8-bit Signed Saturating Subtraction

Syntax `unsigned int __nds32_ksub8(unsigned int a, unsigned int b);
int8x4_t __nds32_v_ksub8(int8x4_t a, int8x4_t b);`

Operation `Rt[31:24] = SAT.Q7(Ra[31:24] - Rb[31:24]);
Rt[23:16] = SAT.Q7(Ra[23:16] - Rb[23:16]);
Rt[15:8] = SAT.Q7(Ra[15:8] - Rb[15:8]);
Rt[7:0] = SAT.Q7(Ra[7:0] - Rb[7:0]);`

Mapped Instruction `KSUB8`

8-bit Unsigned Saturating Subtraction

Syntax `unsigned int __nds32_uksub8(unsigned int a, unsigned int b);
uint8x4_t __nds32_v_uksub8(uint8x4_t a, uint8x4_t b);`

Operation `Rt[31:24] = SAT.U8(Ra[31:24] - Rb[31:24]);`

Andes DSP Library V3 User Manual

```
Rt[23:16] = SAT.U8(Ra[23:16] - Rb[23:16]);
```

```
Rt[15:8] = SAT.U8(Ra[15:8] - Rb[15:8]);
```

```
Rt[7:0] = SAT.U8(Ra[7:0] - Rb[7:0]);
```

Mapped Instruction UKSUB8



3.2.3. 16-bit Shift Intrinsic Functions

16-bit Shift Right Arithmetic

Syntax `unsigned int __nds32__sra16(unsigned int a, unsigned int b);
int16x2_t __nds32__v_sra16(int16x2_t a, unsigned int b);`

Operation `Rt[31:16] = Ra[31:16] s>> Rb[3:0];
Rt[15:0] = Ra[15:0] s>> Rb[3:0];`

Mapped Instruction SRA16, SRAI16

16-bit Rounding Shift Right Arithmetic

Syntax `unsigned int __nds32__sra16_u(unsigned int a, unsigned int b);
int16x2_t __nds32__v_sra16_u(int16x2_t a, unsigned int b);`

Operation `Rt[31:16] = Ra[31:16] sr>> Rb[3:0];
Rt[15:0] = Ra[15:0] sr>> Rb[3:0];`

Mapped Instruction SRA16.u, SRAI16.u

16-bit Shift Right Logical

Syntax `unsigned int __nds32__srl16(unsigned int a, unsigned int b);
uint16x2_t __nds32__v_srl16(uint16x2_t a, unsigned int b);`

Operation `Rt[31:16] = Ra[31:16] u>> Rb[3:0];
Rt[15:0] = Ra[15:0] u>> Rb[3:0];`

Mapped Instruction SRL16, SRLI16

16-bit Rounding Shift Right Logical

Syntax `unsigned int __nds32__srl16_u(unsigned int a, unsigned int b);
uint16x2_t __nds32__v_srl16_u(uint16x2_t a, unsigned int b);`

Operation `Rt[31:16] = Ra[31:16] ur>> Rb[3:0];
Rt[15:0] = Ra[15:0] ur>> Rb[3:0];`

Mapped Instruction SRL16.u, SRLI16.u

16-bit Shift Left Logical

Syntax `unsigned int __nds32__sll16(unsigned int a, unsigned int b);
uint16x2_t __nds32__v_sll16(uint16x2_t a, unsigned int b);`

Operation `Rt[31:16] = Ra[31:16] << Rb[3:0];
Rt[15:0] = Ra[15:0] << Rb[3:0];`

Mapped Instruction SLL16, SLLI16

16-bit Saturating Shift Left Logical

Syntax `unsigned int __nds32__ksll16(unsigned int a, unsigned int b);
int16x2_t __nds32__v_ksll16(int16x2_t a, unsigned int b);`

Operation `Rt[31:16] = SAT.Q15(Ra[31:16] << Rb[3:0]);
Rt[15:0] = SAT.Q15(Ra[15:0] << Rb[3:0]);`

Mapped Instruction KSLL16, KSLLI16

16-bit Shift Left Logical with Saturation & Shift Right Arithmetic

Syntax `unsigned int __nds32__kslra16(unsigned int a, unsigned int b);
int16x2_t __nds32__v_kslra16(int16x2_t a, unsigned int b);`

Operation `if (Rb[4:0] < 0) {
 sa = (Rb[4:0] == -16) ? 15 : -Rb[4:0];
 Rt[31:16] = Ra[31:16] s>> sa;
 Rt[15:0] = Ra[15:0] s>> sa;
} else {
 Rt[31:16] = SAT.Q15(Ra[31:16] << Rb[3:0]);
 Rt[15:0] = SAT.Q15(Ra[15:0] << Rb[3:0]);
}`

Mapped Instruction KSLRA16

16-bit Shift Left Logical with Saturation & Rounding Shift Right Arithmetic

Syntax `unsigned int __nds32__kslra16_u(unsigned int a, unsigned int b);
int16x2_t __nds32__v_kslra16_u(int16x2_t a, unsigned int b);`

Operation `if (Rb[4:0] < 0) {`

Andes DSP Library V3 User Manual

```
sa = (Rb[4:0] == -16) ? 15 : -Rb[4:0];  
Rt[31:16] = Ra[31:16] sr>> sa;  
Rt[15:0] = Ra[15:0] sr>> sa;  
} else {  
Rt[31:16] = SAT.Q15(Ra[31:16] << Rb[3:0]);  
Rt[15:0] = SAT.Q15(Ra[15:0] << Rb[3:0]);  
}
```

Mapped Instruction KSLRA16.u

3.2.4. 16-bit Compare Intrinsic Functions

16-bit Compare Equal

Syntax `unsigned int __nds32_cmpeq16(unsigned int a, unsigned int b);
uint16x2_t __nds32_v_cmpeq16(int16x2_t a, int16x2_t b);
uint16x2_t __nds32_v_ucmpeq16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] == Rb[31:16]) ? 0xffff : 0x0;
Rt[15:0] = (Ra[15:0] == Rb[15:0]) ? 0xffff : 0x0;`

Mapped Instruction CMPEQ16

16-bit Signed Compare Less Than

Syntax `unsigned int __nds32_scmplt16(unsigned int a, unsigned int b);
uint16x2_t __nds32_v_scmplt16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] < Rb[31:16]) ? 0xffff : 0x0;
Rt[15:0] = (Ra[15:0] < Rb[15:0]) ? 0xffff : 0x0;`

Mapped Instruction SCMPLT16

16-bit Signed Compare Less Than & Equal

Syntax `unsigned int __nds32_scmple16(unsigned int a, unsigned int b);
uint16x2_t __nds32_v_scmple16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] <= Rb[31:16]) ? 0xffff : 0x0;
Rt[15:0] = (Ra[15:0] <= Rb[15:0]) ? 0xffff : 0x0;`

Mapped Instruction SCMPLE16

16-bit Unsigned Compare Less Than

Syntax `unsigned int __nds32_ucmplt16(unsigned int a, unsigned int b);
uint16x2_t __nds32_v_ucmplt16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] u< Rb[31:16])? 0xffff : 0x0;
Rt[15:0] = (Ra[15:0] u< Rb[15:0]) ? 0xffff : 0x0;`

Mapped Instruction UCMPLT16

16-bit Unsigned Compare Less Than & Equal

Syntax	<pre> unsigned int __nds32__ucmple16(unsigned int a, unsigned int b); uint16x2_t __nds32__v_ucmple16(uint16x2_t a, uint16x2_t b); </pre>
Operation	<pre> Rt[31:16] = (Ra[31:16] u<= Rb[31:16])? 0xffff : 0x0; Rt[15:0] = (Ra[15:0] u<= Rb[15:0]) ? 0xffff : 0x0; </pre>
Mapped Instruction	UCMPLE16

3.2.5. 8-bit Compare Intrinsic Functions

8-bit Compare Equal

Syntax `unsigned int __nds32_cmpeq8(unsigned int a, unsigned int b);
uint8x4_t __nds32_v_cmpeq8(int8x4_t a, int8x4_t b);
uint8x4_t __nds32_v_ucmpeq8(uint8x4_t a, uint8x4_t b);`

Operation `Rt[31:24] = (Ra[31:24] == Rb[31:24]) ? 0xff : 0x0;
Rt[23:16] = (Ra[23:16] == Rb[23:16]) ? 0xff : 0x0;
Rt[15:8] = (Ra[15:8] == Rb[15:8]) ? 0xff : 0x0;
Rt[7:0] = (Ra[7:0] == Rb[7:0]) ? 0xff : 0x0;`

Mapped Instruction CMPEQ8

8-bit Signed Compare Less Than

Syntax `unsigned int __nds32_scmlt8(unsigned int a, unsigned int b);
uint8x4_t __nds32_v_scmlt8(int8x4_t a, int8x4_t b);`

Operation `Rt[31:24] = (Ra[31:24] < Rb[31:24]) ? 0xff : 0x0;
Rt[23:16] = (Ra[23:16] < Rb[23:16]) ? 0xff : 0x0;
Rt[15:8] = (Ra[15:8] < Rb[15:8]) ? 0xff : 0x0;
Rt[7:0] = (Ra[7:0] < Rb[7:0]) ? 0xff : 0x0;`

Mapped Instruction SCMLT8

8-bit Signed Compare Less Than & Equal

Syntax `unsigned int __nds32_scmlt8(unsigned int a, unsigned int b);
uint8x4_t __nds32_v_scmlt8(int8x4_t a, int8x4_t b);`

Operation `Rt[31:24] = (Ra[31:24] <= Rb[31:24]) ? 0xff : 0x0;
Rt[23:16] = (Ra[23:16] <= Rb[23:16]) ? 0xff : 0x0;
Rt[15:8] = (Ra[15:8] <= Rb[15:8]) ? 0xff : 0x0;
Rt[7:0] = (Ra[7:0] <= Rb[7:0]) ? 0xff : 0x0;`

Mapped Instruction SCMPLE8

8-bit Unsigned Compare Less Than

Syntax `unsigned int __nds32__ucmplt8(unsigned int a, unsigned int b);`
 `uint8x4_t __nds32__v_ucmplt8(uint8x4_t a, uint8x4_t b);`

Operation `Rt[31:24] = (Ra[31:24] < Rb[31:24]) ? 0xff : 0x0;`
 `Rt[23:16] = (Ra[23:16] < Rb[23:16]) ? 0xff : 0x0;`
 `Rt[15:8] = (Ra[15:8] < Rb[15:8]) ? 0xff : 0x0;`
 `Rt[7:0] = (Ra[7:0] < Rb[7:0]) ? 0xff : 0x0;`

Mapped Instruction `UCMPLT8`

8-bit Unsigned Compare Less Than & Equal

Syntax `unsigned int __nds32__ucmple8(unsigned int a, unsigned int b);`
 `uint8x4_t __nds32__v_ucmple8(uint8x4_t a, uint8x4_t b);`

Operation `Rt[31:24] = (Ra[31:24] <= Rb[31:24]) ? 0xff : 0x0;`
 `Rt[23:16] = (Ra[23:16] <= Rb[23:16]) ? 0xff : 0x0;`
 `Rt[15:8] = (Ra[15:8] <= Rb[15:8]) ? 0xff : 0x0;`
 `Rt[7:0] = (Ra[7:0] <= Rb[7:0]) ? 0xff : 0x0;`

Mapped Instruction `UCMPLE8`

3.2.6. 16-bit Miscellaneous Intrinsic Functions

16-bit Signed Minimum

Syntax `unsigned int __nds32__smin16(unsigned int a, unsigned int b);
int16x2_t __nds32__v_smin16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] < Rb[31:16]) ? Ra[31:16] : Rb[31:16];
Rt[15:0] = (Ra[15:0] < Rb[15:0]) ? Ra[15:0] : Rb[15:0];`

Mapped Instruction `SMIN16`

16-bit Unsigned Minimum

Syntax `unsigned int __nds32__umin16(unsigned int a, unsigned int b);
uint16x2_t __nds32__v_umin16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] u< Rb[31:16]) ? Ra[31:16] : Rb[31:16];
Rt[15:0] = (Ra[15:0] u< Rb[15:0]) ? Ra[15:0] : Rb[15:0];`

Mapped Instruction `UMIN16`

16-bit Signed Maximum

Syntax `unsigned int __nds32__smax16(unsigned int a, unsigned int b);
int16x2_t __nds32__v_smax16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] > Rb[31:16]) ? Ra[31:16] : Rb[31:16];
Rt[15:0] = (Ra[15:0] > Rb[15:0]) ? Ra[15:0] : Rb[15:0];`

Mapped Instruction `SMAX16`

16-bit Unsigned Maximum

Syntax `unsigned int __nds32__umax16(unsigned int a, unsigned int b);
uint16x2_t __nds32__v_umax16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = (Ra[31:16] u> Rb[31:16]) ? Ra[31:16] : Rb[31:16];
Rt[15:0] = (Ra[15:0] u> Rb[15:0]) ? Ra[15:0] : Rb[15:0];`

Mapped Instruction `UMAX16`

16-bit Signed Clip Value

Syntax `signed int __nds32_sclip16(signed int a, unsigned int b);
int16x2_t __nds32_v_sclip16(int16x2_t a, unsigned int b);`

Operation `n = Rb[3:0];
Rt[31:16] = SAT.Qn(Ra[31:16]);
Rt[15:0] = SAT.Qn(Ra[15:0]);`

Mapped Instruction `SCLIP16`

16-bit Unsigned Clip Value

Syntax `unsigned int __nds32_uclip16(unsigned int a, unsigned int b);
int16x2_t __nds32_v_uclip16(int16x2_t a, unsigned int b);`

Operation `m = Rb[3:0];
Rt[31:16] = SAT.Um(Ra[31:16]);
Rt[15:0] = SAT.Um(Ra[15:0]);`

Mapped Instruction `UCLIP16`

16-bit Signed Multiply

Syntax `unsigned int __nds32_khm16(unsigned int a, unsigned int b);
int16x2_t __nds32_v_khm16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = SAT.Q15((Ra[31:16] * Rb[31:16]) s>> 15);
Rt[15:0] = SAT.Q15((Ra[15:0] * Rb[15:0]) s>> 15);`

Mapped Instruction `KHM16`

16-bit Crossed Signed Multiply

Syntax `unsigned int __nds32_khmx16(unsigned int a, unsigned int b);
int16x2_t __nds32_v_khmx16(int16x2_t a, int16x2_t b);`

Operation `Rt[31:16] = SAT.Q15((Ra[31:16] * Rb[15:0]) s>> 15);
Rt[15:0] = SAT.Q15((Ra[15:0] * Rb[31:16]) s>> 15);`

Mapped Instruction `KHMX16`

16-bit Signed Multiply to 32-bit

Syntax `unsigned long long __nds32__smul16(unsigned int a, unsigned int b);`
 `int32x2_t __nds32__v_smul16(int16x2_t a, int16x2_t b);`

Operation $Rt[63:32] = Ra[31:16] * Rb[31:16];$
 $Rt[31:0] = Ra[15:0] * Rb[15:0];$

Mapped Instruction `SMUL16`

16-bit Signed Crossed Multiply to 32-bit

Syntax `unsigned long long __nds32__smulx16(unsigned int a, unsigned int b);`
 `int32x2_t __nds32__v_smulx16(int16x2_t a, int16x2_t b);`

Operation $Rt[63:32] = Ra[31:16] * Rb[15:0];$
 $Rt[31:0] = Ra[15:0] * Rb[31:16];$

Mapped Instruction `SMULX16`

16-bit Unsigned Multiply to 32-bit

Syntax `unsigned long long __nds32__umul16(unsigned int a, unsigned int b);`
 `uint32x2_t __nds32__v_umul16(uint16x2_t a, uint16x2_t b);`

Operation $Rt[63:32] = Ra[31:16] u* Rb[31:16];$
 $Rt[31:0] = Ra[15:0] u* Rb[15:0];$

Mapped Instruction `UMUL16`

16-bit Unsigned Crossed Multiply to 32-bit

Syntax `unsigned long long __nds32__umulx16(unsigned int a, unsigned int b);`
 `uint32x2_t __nds32__v_umulx16(uint16x2_t a, uint16x2_t b);`

Operation $Rt[63:32] = Ra[31:16] u* Rb[15:0];$
 $Rt[31:0] = Ra[15:0] u* Rb[31:16];$

Mapped Instruction `UMULX16`

16-bit Absolute Value

Syntax `unsigned int __nds32__kabs16(unsigned int a);`
 `int16x2_t __nds32__v_kabs16(int16x2_t a);`

Operation $Rt[31:16] = ABS.Q15(Ra[31:16]);$
 $Rt[15:0] = ABS.Q15(Ra[15:0]);$

Mapped Instruction KABS16



3.2.7. 8-bit Miscellaneous Intrinsic Functions

8-bit Signed Minimum

Syntax `unsigned int __nds32_smin8(unsigned int a, unsigned int b);
int8x4_t __nds32_v_smin8(int8x4_t a, int8x4_t b);`

Operation $Rt[31:24] = (Ra[31:24] < Rb[31:24]) ? Ra[31:24] : Rb[31:24];$
 $Rt[23:16] = (Ra[23:16] < Rb[23:16]) ? Ra[23:16] : Rb[23:16];$
 $Rt[15:8] = (Ra[15:8] < Rb[15:8]) ? Ra[15:8] : Rb[15:8];$
 $Rt[7:0] = (Ra[7:0] < Rb[7:0]) ? Ra[7:0] : Rb[7:0];$

Mapped Instruction `SMIN8`

8-bit Unsigned Minimum

Syntax `unsigned int __nds32_umin8(unsigned int a, unsigned int b);
uint8x4_t __nds32_v_umin8(uint8x4_t a, uint8x4_t b);`

Operation $Rt[31:24] = (Ra[31:24] u< Rb[31:24]) ? Ra[31:24] : Rb[31:24];$
 $Rt[23:16] = (Ra[23:16] u< Rb[23:16]) ? Ra[23:16] : Rb[23:16];$
 $Rt[15:8] = (Ra[15:8] u< Rb[15:8]) ? Ra[15:8] : Rb[15:8];$
 $Rt[7:0] = (Ra[7:0] u< Rb[7:0]) ? Ra[7:0] : Rb[7:0];$

Mapped Instruction `UMIN8`

8-bit Signed Maximum

Syntax `unsigned int __nds32_smax8(unsigned int a, unsigned int b);
int8x4_t __nds32_v_smax8(int8x4_t a, int8x4_t b);`

Operation $Rt[31:24] = (Ra[31:24] > Rb[31:24]) ? Ra[31:24] : Rb[31:24];$
 $Rt[23:16] = (Ra[23:16] > Rb[23:16]) ? Ra[23:16] : Rb[23:16];$
 $Rt[15:8] = (Ra[15:8] > Rb[15:8]) ? Ra[15:8] : Rb[15:8];$
 $Rt[7:0] = (Ra[7:0] > Rb[7:0]) ? Ra[7:0] : Rb[7:0];$

Mapped Instruction `SMAx8`

8-bit Unsigned Maximum

Syntax `unsigned int __nds32_umax8(unsigned int a, unsigned int b);`

```
uint8x4_t __nds32__v_umax8(uint8x4_t a, uint8x4_t b);
```

Operation

```
Rt[31:24] = (Ra[31:24] u> Rb[31:24]) ? Ra[31:24] : Rb[31:24];
Rt[23:16] = (Ra[23:16] u> Rb[23:16]) ? Ra[23:16] : Rb[23:16];
Rt[15:8]  = (Ra[15:8]  u> Rb[15:8]  ) ? Ra[15:8]  : Rb[15:8];
Rt[7:0]   = (Ra[7:0]   u> Rb[7:0]   ) ? Ra[7:0]   : Rb[7:0];
```

Mapped Instruction UMAX8

8-bit Absolute Value

Syntax

```
unsigned int __nds32__kabs8(unsigned int a);
int8x4_t __nds32__v_kabs8(int8x4_t a);
```

Operation

```
Rt[31:24] = ABS.Q7(Ra[31:24]);
Rt[23:16] = ABS.Q7(Ra[23:16]);
Rt[15:8]  = ABS.Q7(Ra[15:8]);
Rt[7:0]   = ABS.Q7(Ra[7:0]);
```

Mapped Instruction KABS8

3.2.8. 8-bit Unpacking Intrinsic Functions

Signed Unpacking Bytes 1 & 0

Syntax `unsigned int __nds32__sunpkd810(unsigned int a);`
 `int16x2_t __nds32__v_sunpkd810(int8x4_t a);`

Operation `Rt[31:16] = SE16(Ra[15:8]);`
 `Rt[15:0] = SE16(Ra[7:0]);`

Mapped Instruction `SUNPKD810`

Signed Unpacking Bytes 2 & 0

Syntax `unsigned int __nds32__sunpkd820(unsigned int a);`
 `int16x2_t __nds32__v_sunpkd820(int8x4_t a);`

Operation `Rt[31:16] = SE16(Ra[23:16]);`
 `Rt[15:0] = SE16(Ra[7:0]);`

Mapped Instruction `SUNPKD820`

Signed Unpacking Bytes 3 & 0

Syntax `unsigned int __nds32__sunpkd830(unsigned int a);`
 `int16x2_t __nds32__v_sunpkd830(int8x4_t a);`

Operation `Rt[31:16] = SE16(Ra[31:24]);`
 `Rt[15:0] = SE16(Ra[7:0]);`

Mapped Instruction `SUNPKD830`

Signed Unpacking Bytes 3 & 1

Syntax `unsigned int __nds32__sunpkd831(unsigned int a);`
 `int16x2_t __nds32__v_sunpkd831(int8x4_t a);`

Operation `Rt[31:16] = SE16(Ra[31:24]);`
 `Rt[15:0] = SE16(Ra[15:8]);`

Mapped Instruction `SUNPKD831`

Unsigned Unpacking Bytes 1 & 0

Syntax `unsigned int __nds32__zunpkd810(unsigned int a);
uint16x2_t __nds32__v_zunpkd810(uint8x4_t a);`

Operation `Rt[31:16] = ZE16(Ra[15:8]);
Rt[15:0] = ZE16(Ra[7:0]);`

Mapped Instruction `ZUNPKD810`

Unsigned Unpacking Bytes 2 & 0

Syntax `unsigned int __nds32__zunpkd820(unsigned int a);
uint16x2_t __nds32__v_zunpkd820(uint8x4_t a);`

Operation `Rt[31:16] = ZE16(Ra[23:16]);
Rt[15:0] = ZE16(Ra[7:0]);`

Mapped Instruction `ZUNPKD820`

Unsigned Unpacking Bytes 3 & 0

Syntax `unsigned int __nds32__zunpkd830(unsigned int a);
uint16x2_t __nds32__v_zunpkd830(uint8x4_t a);`

Operation `Rt[31:16] = ZE16(Ra[31:24]);
Rt[15:0] = ZE16(Ra[7:0]);`

Mapped Instruction `ZUNPKD830`

Unsigned Unpacking Bytes 3 & 1

Syntax `unsigned int __nds32__zunpkd831(unsigned int a);
uint16x2_t __nds32__v_zunpkd831(uint8x4_t a);`

Operation `Rt[31:16] = ZE16(Ra[31:24]);
Rt[15:0] = ZE16(Ra[15:8]);`

Mapped Instruction `ZUNPKD831`

3.3. Non-SIMD Data Processing Intrinsic Functions

3.3.1. 32-bit Addition/Subtraction Intrinsic Functions

32-bit Signed Halving Addition

Syntax `signed int __nds32__raddw(signed int a, signed int b);`

Operation `Rt[31:0] = (Ra[31:0] + Rb[31:0]) s>> 1;`

Mapped Instruction `RADDW`

32-bit Unsigned Halving Addition

Syntax `unsigned int __nds32__uraddw(unsigned int a, unsigned int b);`

Operation `Rt[31:0] = (Ra[31:0] + Rb[31:0]) u>> 1;`

Mapped Instruction `URADDW`

32-bit Signed Halving Subtraction

Syntax `signed int __nds32__rsubw(signed int a, signed int b);`

Operation `Rt[31:0] = (Ra[31:0] - Rb[31:0]) s>> 1;`

Mapped Instruction `RSUBW`

32-bit Unsigned Halving Subtraction

Syntax `unsigned int __nds32__ursubw(unsigned int a, unsigned int b);`

Operation `Rt[31:0] = (Ra[31:0] - Rb[31:0]) u>> 1;`

Mapped Instruction `URSUBW`

3.3.2. 32-bit Shift Intrinsic Functions

Rounding Shift Right Arithmetic

Syntax `int __nds32_sra_u(int a, unsigned int b);`

Operation `Rt[31:0] = Ra[31:0] sr >> Rb[4:0];`

Mapped Instruction `SRA.u, SRAI.u`

Saturating Shift Left Logical

Syntax `int __nds32_ksll(int a, unsigned int b);`

Operation `Rt[31:0] = SAT.Q31(Ra[31:0] << Rb[4:0]);`

Mapped Instruction `KSLI, KSLLI`

Shift Left Logical with Saturation & Rounding Shift Right Arithmetic

Syntax `int __nds32_kslraw_u(int a, int b);`

Operation

```

if (Rb[7:0] < 0) {
    sa = (Rb[7:0] < -31) ? 31 : -Rb[7:0];
    Rt[31:0] = Ra[31:0] sr >> sa;
} else {
    Rt[31:0] = SAT.Q31(Ra[31:0] << Rb[7:0]);
}

```

Mapped Instruction `KSLRAW.u`

3.3.3. 16-bit Packing Intrinsic Functions

Pack two 16-bit Data from Bottoms

Syntax `unsigned int __nds32__pkbb16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32__v_pkbb16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = Ra[15:0];`
 `Rt[15:0] = Rb[15:0];`

Mapped Instruction PKBB16

Pack two 16-bit Data Bottom & Top

Syntax `unsigned int __nds32__pkbt16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32__v_pkbt16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = Ra[15:0];`
 `Rt[15:0] = Rb[31:16];`

Mapped Instruction PKBT16

Pack two 16-bit Data Top & Bottom

Syntax `unsigned int __nds32__pktb16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32__v_pktb16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = Ra[31:16];`
 `Rt[15:0] = Rb[15:0];`

Mapped Instruction PKTB16

Pack two 16-bit Data from Tops

Syntax `unsigned int __nds32__pktt16(unsigned int a, unsigned int b);`
 `uint16x2_t __nds32__v_pktt16(uint16x2_t a, uint16x2_t b);`

Operation `Rt[31:16] = Ra[31:16];`
 `Rt[15:0] = Rb[31:16];`

Mapped Instruction PKTT16

3.3.4. Most Significant Word “32x32” Multiply & Add Intrinsic Functions

MSW “32 x 32” Signed Multiplication

Syntax `int __nds32_smmul(int a, int b);`

Operation `Rt[31:0] = (Ra[31:0] * Rb[31:0]) s>> 32;`

Mapped Instruction `SMMUL`

MSW “32 x 32” Signed Multiplication with Rounding

Syntax `int __nds32_smmul_u(int a, int b);`

Operation `Rt[31:0] = (Ra[31:0] * Rb[31:0]) sr>> 32;`

Mapped Instruction `SMMUL.u`

MSW “32 x 32” Signed Multiplication and Saturating Addition

Syntax `int __nds32_kmmac(int t, int a, int b);`

Operation `Rt[31:0] = SAT.Q31(Rt[31:0] + (Ra[31:0] * Rb[31:0]) s>> 32);`

Mapped Instruction `KMMAC`

MSW “32 x 32” Signed Multiplication and Saturating Addition with Rounding

Syntax `int __nds32_kmmac_u(int t, int a, int b);`

Operation `Rt[31:0] = SAT.Q31(Rt[31:0] + (Ra[31:0] * Rb[31:0]) sr>> 32);`

Mapped Instruction `KMMAC.u`

MSW “32 x 32” Signed Multiplication and Saturating Subtraction

Syntax `int __nds32_kmmsb(int t, int a, int b);`

Operation `Rt[31:0] = SAT.Q31(Rt[31:0] - (Ra[31:0] * Rb[31:0]) s>> 32);`

Mapped Instruction `KMMSB`

MSW “32 x 32” Signed Multiplication and Saturating Subtraction with Rounding

Syntax `int __nds32__kmmsb_u(int t, int a, int b);`

Operation `Rt[31:0] = SAT.Q31(Rt[31:0] - (Ra[31:0] * Rb[31:0]) sr>> 32);`

Mapped Instruction `KMMSB.u`

MSW “32 x 32” Signed Multiplication & Double

Syntax `int __nds32__kwmmul(int a, int b);`

Operation `Rt[31:0] = SAT.Q31((Ra[31:0] * Rb[31:0]) s>> 31);`

Mapped Instruction `KWMMUL`

MSW “32 x 32” Signed Multiplication & Double with Rounding

Syntax `int __nds32__kwmmul_u(int a, int b);`

Operation `Rt[31:0] = SAT.Q31((Ra[31:0] * Rb[31:0]) sr>> 31);`

Mapped Instruction `KWMMUL.u`

3.3.5. Most Significant Word “32x16” Multiply & Add Intrinsic Functions

MSW “32 x Bottom 16” Signed Multiplication

Syntax `int __nds32__smmwb(int a, unsigned int b);`
`int __nds32__v_smmwb(int a, int16x2_t b);`

Operation $Rt[31:0] = (Ra[31:0] * Rb[15:0]) s\gg 16;$

Mapped Instruction SMMWB

MSW “32 x Bottom 16” Signed Multiplication with Rounding

Syntax `int __nds32__smmwb_u(int a, unsigned int b);`
`int __nds32__v_smmwb_u(int a, int16x2_t b);`

Operation $Rt[31:0] = (Ra[31:0] * Rb[15:0]) sr\gg 16;$

Mapped Instruction SMMWB.u

MSW “32 x Top 16” Signed Multiplication

Syntax `int __nds32__smmwt(int a, unsigned int b);`
`int __nds32__v_smmwt(int a, int16x2_t b);`

Operation $Rt[31:0] = (Ra[31:0] * Rb[31:16]) s\gg 16;$

Mapped Instruction SMMWT

MSW “32 x Top 16” Signed Multiplication with Rounding (MSW 32 = 32x16)

Syntax `int __nds32__smmwt_u(int a, unsigned int b);`
`int __nds32__v_smmwt_u(int a, int16x2_t b);`

Operation $Rt[31:0] = (Ra[31:0] * Rb[31:16]) sr\gg 16;$

Mapped Instruction SMMWT.u

MSW “32 x Bottom 16” Signed Multiplication and Saturating Addition

Syntax `int __nds32__kmmawb(int t, int a, unsigned int b);`
`int __nds32__v_kmmawb(int t, int a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Rt[31:0] + (Ra[31:0] * Rb[15:0]) s\gg 16);$

Mapped Instruction KMMAWB

MSW “32 x Bottom 16” Signed Multiplication and Saturating Addition with Rounding

Syntax `int __nds32__kmmawb_u(int t, int a, unsigned int b);`
`int __nds32__v_kmmawb_u(int t, int a, int16x2_t b);`

Operation `Rt[31:0] = SAT.Q31(Rt[31:0] + (Ra[31:0] * Rb[15:0]) sr>> 16);`

Mapped Instruction KMMAWB.u

MSW “32 x Top 16” Signed Multiplication and Saturating Addition

Syntax `int __nds32__kmmawt(int t, int a, unsigned int b);`
`int __nds32__v_kmmawt(int t, int a, int16x2_t b);`

Operation `Rt[31:0] = SAT.Q31(Rt[31:0] + (Ra[31:0] * Rb[31:16]) s>> 16);`

Mapped Instruction KMMAWT

MSW “32 x Top 16” Signed Multiplication and Saturating Addition with Rounding

Syntax `int __nds32__kmmawt_u(int t, int a, unsigned int b);`
`int __nds32__v_kmmawt_u(int t, int a, int16x2_t b);`

Operation `Rt[31:0] = SAT.Q31(Rt[31:0] + (Ra[31:0] * Rb[31:16]) sr>> 16);`

Mapped Instruction KMMAWT.u

3.3.6. Signed 16-bit Multiply with 32-bit Add/Subtract Intrinsic Functions

Signed Multiply Bottom 16 & Bottom 16

Syntax `int __nds32_smbb(unsigned int a, unsigned int b);`
`int __nds32_v_smbb(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = Ra[15:0] * Rb[15:0];$

Mapped Instruction SMBB

Signed Multiply Bottom 16 & Top 16

Syntax `int __nds32_smbt(unsigned int a, unsigned int b);`
`int __nds32_v_smbt(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = Ra[15:0] * Rb[31:16];$

Mapped Instruction SMBT

Signed Multiply Top 16 & Top 16

Syntax `int __nds32_smtt(unsigned int a, unsigned int b);`
`int __nds32_v_smtt(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = Ra[31:16] * Rb[31:16];$

Mapped Instruction SMTT

Two “16x16” and Signed Addition

Syntax `int __nds32_kmda(unsigned int a, unsigned int b);`
`int __nds32_v_kmda(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Ra[31:16] * Rb[31:16] + Ra[15:0] * Rb[15:0]);$

Mapped Instruction KMDA

Two Crossed “16x16” and Signed Addition

Syntax `int __nds32_kmxda(unsigned int a, unsigned int b);`
`int __nds32_v_kmxda(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Ra[31:16] * Rb[15:0] + Ra[15:0] * Rb[31:16]);$

Mapped Instruction KMXDA

Two “16x16” and Signed Subtraction

Syntax `int __nds32__smds(unsigned int a, unsigned int b);`
`int __nds32__v_smds(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = Ra[31:16] * Rb[31:16] - Ra[15:0] * Rb[15:0];$

Mapped Instruction SMDS

Two “16x16” and Signed Reversed Subtraction

Syntax `int __nds32__smdrs(unsigned int a, unsigned int b);`
`int __nds32__v_smdrs(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = Ra[15:0] * Rb[15:0] - Ra[31:16] * Rb[31:16];$

Mapped Instruction SMDRS

Two Crossed “16x16” and Signed Subtraction

Syntax `int __nds32__smxds(unsigned int a, unsigned int b);`
`int __nds32__v_smxds(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = Ra[31:16] * Rb[15:0] - Ra[15:0] * Rb[31:16];$

Mapped Instruction SMXDS

“Bottom 16 x Bottom 16” with 32-bit Signed Addition

Syntax `int __nds32__kmabb(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmabb(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Rt[31:0] + Ra[15:0] * Rb[15:0]);$

Mapped Instruction KMABB

“Bottom 16 x Top 16” with 32-bit Signed Addition

Syntax `int __nds32__kmabt(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmabt(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Rt[31:0] + Ra[15:0] * Rb[31:16]);$

Mapped Instruction KMABT

“Top 16 x Top 16” with 32-bit Signed Addition

Syntax `int __nds32__kmatt(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmatt(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Rt[31:0] + Ra[31:16] * Rb[31:16]);$

Mapped Instruction KMATT

Two “16x16” with 32-bit Signed Double Addition

Syntax `int __nds32__kmada(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmada(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Rt[31:0] + Ra[31:16] * Rb[31:16] + Ra[15:0] * Rb[15:0]);$

Mapped Instruction KMADA

Two Crossed “16x16” with 32-bit Signed Double Addition

Syntax `int __nds32__kmaxda(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmaxda(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Rt[31:0] + Ra[31:16] * Rb[15:0] + Ra[15:0] * Rb[31:16]);$

Mapped Instruction KMAXDA

Two “16x16” with 32-bit Signed Addition and Subtraction

Syntax `int __nds32__kmads(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmads(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q31(Rt[31:0] + Ra[31:16] * Rb[31:16] - Ra[15:0] * Rb[15:0]);$

Mapped Instruction KMADS

Two “16x16” with 32-bit Signed Addition and Reversed Subtraction

Syntax `int __nds32__kmadrs(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmadrs(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q31}(Rt[31:0] + Ra[15:0] * Rb[15:0] - Ra[31:16] * Rb[31:16]);$

Mapped Instruction `KMADRS`

Two Crossed “16x16” with 32-bit Signed Addition and Subtraction

Syntax `int __nds32__kmaxds(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmaxds(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q31}(Rt[31:0] + Ra[31:16] * Rb[15:0] - Ra[15:0] * Rb[31:16]);$

Mapped Instruction `KMAXDS`

Two “16x16” with 32-bit Signed Double Subtraction

Syntax `int __nds32__kmsda(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmsda(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q31}(Rt[31:0] - Ra[31:16] * Rb[31:16] - Ra[15:0] * Rb[15:0]);$

Mapped Instruction `KMSDA`

Two Crossed “16x16” with 32-bit Signed Double Subtraction

Syntax `int __nds32__kmsxda(int t, unsigned int a, unsigned int b);`
`int __nds32__v_kmsxda(int t, int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q31}(Rt[31:0] - Ra[31:16] * Rb[15:0] - Ra[15:0] * Rb[31:16]);$

Mapped Instruction `KMSXDA`

3.3.7. Signed 16-bit Multiply with 64-bit Add/Subtract Intrinsic Functions

"16 x 16" with 64-bit Signed Addition

Syntax

```
long long __nds32__smal(long long a, unsigned int b);  
long long __nds32__v_smal(long long a, int16x2_t b);
```

Operation
$$Rt[63:0] = Ra[63:0] + Rb[31:16] * Rb[15:0];$$
Mapped Instruction SMAL

3.3.8. Miscellaneous Intrinsic Functions

Signed Clip Value

Syntax `int __nds32_clips(int a, const unsigned int imm5u);`

Operation `n = imm5u;`
`Rt[31:0] = SAT.Qn(Ra[31:0]);`

Mapped Instruction `SCLIP32`

Unsigned Clip Value

Syntax `unsigned int __nds32_clip(int a, const unsigned int imm5u);`

Operation `m = imm5u;`
`Rt[31:0] = SAT.Um(Ra[31:0]);`

Mapped Instruction `UCLIP32`

Bit Reverse

Syntax `unsigned int __nds32_bitrev(unsigned int a, unsigned int b);`

Operation `Rt[31:0] = ZE32(REVB(Ra[Rb[4:0] :0]));`

Mapped Instruction `BITREV, BITREVI`

Extract 32-bit from a 64-bit Value

Syntax `unsigned int __nds32_wext(long long a, unsigned int b);`

Operation `Rt[31:0] = Ra[63:0] u>> Rb[4:0];`

Mapped Instruction `WEXT, WEXTI`

Bit-wise Pick

Syntax `unsigned int __nds32_bpick(unsigned int a, unsigned int b, unsigned int c);`

Operation `for (i=0; i<=31; i++) {`
`Rt[i] = (Rc[i] == 1) ? Ra[i] : Rb[i];`
`}`

Mapped Instruction BPICK

Insert Byte

Syntax `unsigned int __nds32_insb(unsigned int t, unsigned int a, unsigned int b);`

Operation $Rt[8 * Rb[0:1] + 7 : 8 * Rb[0:1]] = Ra[7:0];$

Mapped Instruction INSB

Absolute Value

Syntax `int __nds32_abs(int a);`

Operation $Rt[31:0] = ABS.Q31(Ra[31:0]);$

Mapped Instruction KABS

Average Value of a and b

Syntax `int __nds32_ave(int a, int b);`

Operation $Rt[31:0] = (Ra[31:0] + Rb[31:0]) \text{ sr} \gg 1;$

Mapped Instruction AVE

Bit Clear

Syntax `unsigned int __nds32_bclr(unsigned int a, const unsigned int b);`

Operation $Rt[31:0] = Ra[31:0] \& (\sim(1 \ll Rb[4:0]));$

Mapped Instruction BCLR

Bit Set

Syntax `unsigned int __nds32_bset(unsigned int a, const unsigned int pos);`

Operation $Rt[31:0] = Ra[31:0] \mid (1 \ll Rb[4:0]);$

Mapped Instruction BSET

Bit Toggle

Andes DSP Library V3 User Manual

Syntax `unsigned int __nds32_btgl(unsigned int a, const unsigned int pos);`

Operation `Rt[31:0] = Ra[31:0] ^ (1 << Rb[4:0]);`

Mapped Instruction BTGL

Bit Test

Syntax `unsigned int __nds32_btst(unsigned int a, const unsigned int pos);`

Operation `Rt[31:0] = (Ra[31:0] & (1 << Rb[4:0])) ? 1 : 0;`

Mapped Instruction BTST

Count Leading Ones

Syntax `unsigned int __nds32_clo(unsigned int a);`

Operation

```

cnt = 0;
for (i=31; i<=0; i--) {
    if (Ra[i]) {
        cnt++;
    } else {
        break;
    }
}
Rt[31:0] = cnt;

```

Mapped Instruction CLO

Count Leading Zeros

Syntax `unsigned int __nds32_clz(unsigned int a);`

Operation

```

cnt = 0;
for (i=31; i<=0; i--) {
    if (!Ra[i]) {

```

```

        cnt++;
    } else {
        break;
    }
}
Rt[31:0] = cnt;

```

Official Release

Mapped Instruction CLZ

Bit Stream Extraction

Syntax `void __nds32__bse(unsigned int *t, unsigned int a, unsigned int *b);`

Operation Please refer to *Andes Programming Guide for ISA V3* and *AndeStar Instruction Set Architecture Manual*.

Mapped Instruction BSE

Bit Stream Packing

Syntax `void __nds32__bsp(unsigned int *t, unsigned int a, unsigned int *b);`

Operation Please refer to *Andes Programming Guide for ISA V3* and *AndeStar Instruction Set Architecture Manual*.

Mapped Instruction BSP

Parallel Byte Sum of Absolute Difference

Syntax `unsigned int __nds32__pbsad(unsigned int a, unsigned int b);`

Operation

```

sum = ABS.Q7(Ra[7:0] - Rb[7:0]);
sum += ABS.Q7(Ra[15:8] - Rb[15:8]);
sum += ABS.Q7(Ra[23:16] - Rb[23:16]);
sum += ABS.Q7(Ra[31:24] - Rb[31:24]);
Rt[31:0] = sum;

```

Mapped Instruction PBSAD

Parallel Byte Sum of Absolute Difference Accumulation

Syntax `unsigned int __nds32__pbsada(unsigned int acc, unsigned int a,
unsigned int b);`

Operation `sum = ABS.Q7(Ra[7:0] - Rb[7:0]);
sum += ABS.Q7(Ra[15:8] - Rb[15:8]);
sum += ABS.Q7(Ra[23:16] - Rb[23:16]);
sum += ABS.Q7(Ra[31:24] - Rb[31:24]);
Rt[31:0] += sum;`

Mapped Instruction PBSADA

3.4. 64-bit Profile Intrinsic Functions

3.4.1. 64-bit Addition & Subtraction Intrinsic Functions

64-bit Addition

Syntax `long long __nds32__sadd64(long long a, long long b);`
 `unsigned long long __nds32__uadd64(unsigned long long a, unsigned long long b);`

Operation $Rt[63:0] = Ra[63:0] + Rb[63:0];$

Mapped Instruction `ADD64`

64-bit Signed Halving Addition

Syntax `long long __nds32__radd64(long long a, long long b);`

Operation $Rt[63:0] = (Ra[63:0] + Rb[63:0]) \gg 1;$

Mapped Instruction `RADD64`

64-bit Unsigned Halving Addition

Syntax `unsigned long long __nds32__uradd64(unsigned long long a, unsigned long long b);`

Operation $Rt[63:0] = (Ra[63:0] + Rb[63:0]) \gg 1;$

Mapped Instruction `URADD64`

64-bit Signed Saturating Addition

Syntax `long long __nds32__kadd64(long long a, long long b);`

Operation $Rt[63:0] = SAT.Q63(Ra[63:0] + Rb[63:0]);$

Mapped Instruction `KADD64`

64-bit Unsigned Saturating Addition

Syntax `unsigned long long __nds32__ukadd64(unsigned long long a, unsigned long long b);`

Operation `Rt[63:0] = SAT.U64(Ra[63:0] + Rb[63:0]);`

Mapped Instruction `UKADD64`

64-bit Subtraction

Syntax `long long __nds32__ssub64(long long a, long long b);`
`unsigned long long __nds32__usub64(unsigned long long a, unsigned long long b);`

Operation `Rt[63:0] = Ra[63:0] - Rb[63:0];`

Mapped Instruction `SUB64`

64-bit Signed Halving Subtraction

Syntax `long long __nds32__rsub64(long long a, long long b);`

Operation `Rt[63:0] = (Ra[63:0] - Rb[63:0]) s>> 1;`

Mapped Instruction `RSUB64`

64-bit Unsigned Halving Subtraction

Syntax `unsigned long long __nds32__ursub64(unsigned long long a, unsigned long long b);`

Operation `Rt[63:0] = (Ra[63:0] - Rb[63:0]) u>> 1;`

Mapped Instruction `URSUB64`

64-bit Signed Saturating Subtraction

Syntax `long long __nds32__ksub64(long long a, long long b);`

Operation `Rt[63:0] = SAT.Q63(Ra[63:0] - Rb[63:0]);`

Mapped Instruction `KSUB64`

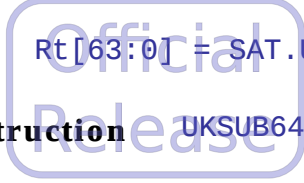
64-bit Unsigned Saturating Subtraction

Andes DSP Library V3 User Manual

Syntax unsigned long long __nds32__uksub64(unsigned long long a,
 unsigned long long b);

Operation Rt[63:0] = SAT.U64(Ra[63:0] - Rb[63:0]);

Mapped Instruction UKSUB64



3.4.2. 32-bit Multiply with 64-bit Add/Subtract Intrinsic Functions

32x32 with 64-bit Signed Addition

Syntax `long long __nds32__smar64(long long t, int a, int b);`

Operation $Rt[63:0] = Rt[63:0] + Ra[31:0] * Rb[31:0];$

Mapped Instruction SMAR64

32x32 with 64-bit Signed Subtraction

Syntax `long long __nds32__msr64(long long t, int a, int b);`

Operation $Rt[63:0] = Rt[63:0] - Ra[31:0] * Rb[31:0];$

Mapped Instruction SMSR64

32x32 with 64-bit Unsigned Addition

Syntax `unsigned long long __nds32__umar64(unsigned long long t, unsigned int a, unsigned int b);`

Operation $Rt[63:0] = Rt[63:0] + Ra[31:0] * Rb[31:0];$

Mapped Instruction UMAR64

32x32 with 64-bit Unsigned Subtraction

Syntax `unsigned long long __nds32__umsr64(unsigned long long t, unsigned int a, unsigned int b);`

Operation $Rt[63:0] = Rt[63:0] - Ra[31:0] * Rb[31:0];$

Mapped Instruction UMSR64

32x32 with Saturating 64-bit Signed Addition

Syntax `long long __nds32__kmar64(long long t, int a, int b);`

Operation $Rt[63:0] = SAT.Q63(Rt[63:0] + Ra[31:0] * Rb[31:0]);$

Mapped Instruction KMAR64

32x32 with Saturating 64-bit Signed Subtraction

Syntax `long long __nds32__kmsr64(long long t, int a, int b);`

Operation `Rt[63:0] = SAT.Q63(Rt[63:0] - Ra[31:0] * Rb[31:0]);`

Mapped Instruction `KMSR64`

32x32 with Saturating 64-bit Unsigned Addition

Syntax `unsigned long long __nds32__ukmar64(unsigned long long t, unsigned int a, unsigned int b);`

Operation `Rt[63:0] = SAT.U64(Rt[63:0] + Ra[31:0] * Rb[31:0]);`

Mapped Instruction `UKMAR64`

32x32 with Saturating 64-bit Unsigned Subtraction

Syntax `unsigned long long __nds32__ukmsr64(unsigned long long t, unsigned int a, unsigned int b);`

Operation `Rt[63:0] = SAT.U64(Rt[63:0] - Ra[31:0] * Rb[31:0]);`

Mapped Instruction `UKMSR64`

3.4.3. Signed 16-bit Multiply with 64-bit Add/Subtract Intrinsic Functions

“Bottom 16 x Bottom 16” with 64-bit Signed Addition

Syntax `long long __nds32__smalbb(long long t, unsigned int a, unsigned int b);`
 `long long __nds32__v_smalbb(long long t, int16x2_t a, int16x2_t b);`

Operation $Rt[63:0] = Rt[63:0] + Ra[15:0] * Rb[15:0];$

Mapped Instruction SMALBB

“Bottom 16 x Top 16” with 64-bit Signed Addition

Syntax `long long __nds32__smalbt(long long t, unsigned int a, unsigned int b);`
 `long long __nds32__v_smalbt(long long t, int16x2_t a, int16x2_t b);`

Operation $Rt[63:0] = Rt[63:0] + Ra[15:0] * Rb[31:16];$

Mapped Instruction SMALBT

“Top 16 x Top 16” with 64-bit Signed Addition

Syntax `long long __nds32__smalbt(long long t, unsigned int a, unsigned int b);`
 `long long __nds32__v_smalbt(long long t, int16x2_t a, int16x2_t b);`

Operation $Rt[63:0] = Rt[63:0] + Ra[31:16] * Rb[31:16];$

Mapped Instruction SMALTT

Two “16x16” with 64-bit Signed Double Addition

Syntax `long long __nds32__smalda(long long t, unsigned int a, unsigned int b);`
 `long long __nds32__v_smalda(long long t, int16x2_t a, int16x2_t b);`

Operation $Rt[63:0] = Rt[63:0] + Ra[31:16] * Rb[31:16] + Ra[15:0] * Rb[15:0];$

Mapped Instruction SMALDA

Two Crossed “16x16” with 64-bit Signed Double Addition

Syntax

```
long long __nds32__smalxda(long long t, unsigned int a, unsigned int b);
long long __nds32__v_smalxda(long long t, int16x2_t a, int16x2_t b);
```

Operation $Rt[63:0] = Rt[63:0] + Ra[31:16] * Rb[15:0] + Ra[15:0] * Rb[31:16];$

Mapped Instruction SMALXDA

Two “16x16” with 64-bit Signed Addition and Subtraction

Syntax

```
long long __nds32__smallds(long long t, unsigned int a, unsigned int b);
long long __nds32__v_smallds(long long t, int16x2_t a, int16x2_t b);
```

Operation $Rt[63:0] = Rt[63:0] + Ra[31:16] * Rb[31:16] - Ra[15:0] * Rb[15:0];$

Mapped Instruction SMALDS

Two “16x16” with 64-bit Signed Addition and Reversed Subtraction

Syntax

```
long long __nds32__smaldrs(long long t, unsigned int a, unsigned int b);
long long __nds32__v_smaldrs(long long t, int16x2_t a, int16x2_t b);
```

Operation $Rt[63:0] = Rt[63:0] + Ra[15:0] * Rb[15:0] - Ra[31:16] * Rb[31:16];$

Mapped Instruction SMALDRS

Two Crossed “16x16” with 64-bit Signed Addition and Subtraction

Syntax

```
long long __nds32__smalxds(long long t, unsigned int a, unsigned int b);
long long __nds32__v_smalxds(long long t, int16x2_t a, int16x2_t b);
```

Operation $Rt[63:0] = Rt[63:0] + Ra[31:16] * Rb[15:0] - Ra[15:0] * Rb[31:16];$

Mapped Instruction SMALXDS

Two “16x16” with 64-bit Signed Double Subtraction

Syntax

```
long long __nds32__smslda(long long t, unsigned int a, unsigned int b);
long long __nds32__v_smslda(long long t, int16x2_t a, int16x2_t b);
```

Operation $Rt[63:0] = Rt[63:0] - Ra[31:16] * Rb[31:16] - Ra[15:0] * Rb[15:0];$

Mapped Instruction SMSLDA

Two Crossed “16x16” with 64-bit Signed Double Subtraction

Syntax

```
long long __nds32__smslxda(long long t, unsigned int a, unsigned int b);
long long __nds32__v_smslxda(long long t, int16x2_t a, int16x2_t b);
```

Operation $Rt[63:0] = Rt[63:0] - Ra[31:16] * Rb[15:0] - Ra[15:0] * Rb[31:16];$

Mapped Instruction SMSLXDA

3.5. Saturation ISA Extension Intrinsic Functions

3.5.1. Q31 saturation Intrinsic Functions

Add with Q31 Saturation

Syntax `int __nds32__kaddw(int a, int b);`

Operation `Rt[31:0] = SAT.Q31(Ra[31:0] + Rb[31:0]);`

Mapped Instruction `KADDW`

Subtract with Q31 Saturation

Syntax `int __nds32__ksubw(int a, int b);`

Operation `Rt[31:0] = SAT.Q31(Ra[31:0] - Rb[31:0]);`

Mapped Instruction `KSUBW`

Logical Left Shift or Arithmetic Right Shift with Q31 Saturation

Syntax `int __nds32__kslraw(int a, int b);`

Operation

```

if (Rb[7:0] < 0) {
    sa = (Rb[7:0] < -31) ? 31 : -Rb[7:0];
    Rt[31:0] = Ra[31:0] s>> sa;
} else {
    Rt[31:0] = SAT.Q31(Ra[31:0] << Rb[7:0]);
}

```

Mapped Instruction `KSLRAW`

Multiply Q15 Numbers with Q31 Saturation in Bottom Parts of Two Registers

Syntax

```

int __nds32__kdmbb(unsigned int a, unsigned int b);
int __nds32__v_kdmbb(int16x2_t a, int16x2_t b);

```

Operation `Rt[31:0] = SAT.Q31((Ra[15:0] * Rb[15:0]) << 1);`

Mapped Instruction `KDMBB`

Multiply Q15 Numbers with Q31 Saturation in Top and Bottom Parts of Two Registers

Syntax `int __nds32__kdmtb(unsigned int a, unsigned int b);`
 `int __nds32__v_kdmtb(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q31}((Ra[31:16] * Rb[15:0]) \ll 1);$

Mapped Instruction KDMTB

Multiply Q15 Numbers with Q31 Saturation in Bottom and Top Parts of Two Registers

Syntax `int __nds32__kdmbt(unsigned int a, unsigned int b);`
 `int __nds32__v_kdmbt(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q31}((Ra[15:0] * Rb[31:16]) \ll 1);$

Mapped Instruction KDMBT

Multiply Q15 Numbers With Q31 Saturation in Top Parts of Two Registers

Syntax `int __nds32__kdmtt(unsigned int a, unsigned int b);`
 `int __nds32__v_kdmtt(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q31}((Ra[31:16] * Rb[31:16]) \ll 1);$

Mapped Instruction KDMTT

3.5.2. Q15 saturation Intrinsic Functions

Add with Q15 Saturation

Syntax `int __nds32__kaddh(int a, int b);`

Operation $Rt[31:0] = \text{SAT.Q15}(Ra[15:0] + Rb[15:0]);$

Mapped Instruction KADDH

Subtract with Q15 Saturation

Syntax `int __nds32__ksubh(int a, int b);`

Operation $Rt[31:0] = \text{SAT.Q15}(Ra[15:0] - Rb[15:0]);$

Mapped Instruction KSUBH

Multiply Q15 Numbers in Bottom Parts of Two Registers and Extract High Part with Q15 Saturation

Syntax `int __nds32__khmbb(unsigned int a, unsigned int b);`
`int __nds32__v_khmbb(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q15}((Ra[15:0] * Rb[15:0]) \gg 15);$

Mapped Instruction KHMBB

Multiply Q15 Numbers in Top and Bottom Parts of Two Registers and Extract High Part with Q15 Saturation

Syntax `int __nds32__khmtb(unsigned int a, unsigned int b);`
`int __nds32__v_khmtb(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = \text{SAT.Q15}((Ra[31:16] * Rb[15:0]) \gg 15);$

Mapped Instruction KHMTB

Multiply Q15 Numbers in Bottom and Top Parts of Two Registers and Extract High Part with Q15 Saturation

Syntax `int __nds32__khmbt(unsigned int a, unsigned int b);`
`int __nds32__v_khmbt(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q15((Ra[15:0] * Rb[31:16]) s \gg 15);$

Mapped Instruction KHMBT

Multiply Q15 Numbers in Top Parts of Two Registers and Extract High Part with Q15 Saturation

Syntax `int __nds32__khmtt(unsigned int a, unsigned int b);`
`int __nds32__v_khmtt(int16x2_t a, int16x2_t b);`

Operation $Rt[31:0] = SAT.Q15((Ra[31:16] * Rb[31:16]) s \gg 15);$

Mapped Instruction KHMTT

3.5.3. Overflow status manipulation Intrinsic Functions

Read PSW.OV to Rt

Syntax `unsigned int __nds32__rdov();`

Operation `Rt = ZE32(PSW.OV);`

Mapped Instruction `RDOV`

Clear PSW.OV Flag

Syntax `unsigned int __nds32__clrov();`

Operation `PSW.OV = 0;`

Mapped Instruction `CLR0V`